



Anwendungshandbuch für die Software SILAB[®] ESSENTIAL

Version 7.1

Mail: silab@wivw.de
Telefon: +49 931 78009-400



INHALTSVERZEICHNIS

1	VORWORT.....	5
2	EINFÜHRUNG UND ÜBERBLICK.....	5
2.1	Übersicht über SILAB.....	5
2.2	Ziel und Adressaten des Handbuchs.....	6
2.3	Inhalte des Handbuchs	6
3	BESTANDTEILE DER SOFTWARE	7
3.1	Programme von SILAB	7
3.1.1	SILABAdmin.....	7
3.1.2	SILABSessionEdit.....	8
3.1.3	Das Programm SILAB	8
3.2	Organisation von SILAB auf dem Operatorrechner	9
3.2.1	Operatorrechner.....	9
3.2.2	Installation	9
3.2.3	Installationsverzeichnis von SILAB.....	9
4	VERWALTUNG DER SIMULATIONSRECHNER	11
4.1	Benutzeroberfläche von SILABAdmin.....	11
4.2	Steuerung der Simulationsrechner.....	12
4.3	Installation von Updates auf den Simulationsrechnern.....	12
5	ZUSAMMENSTELLUNG VON SIMULATIONSSTRECKEN (SESSIONS) MIT SILABSESSIONEDIT .	14
5.1	Anwendung einer dynamischen Datenbasis	15
5.2	SILABSessionEdit-Benutzeroberfläche.....	16
5.3	Öffnen einer Szenarien-Datenbank.....	18
5.4	Festlegen einer Szenarien-Abfolge.....	19
5.4.1	Szenarien verknüpfen	21
5.4.2	Szenarienabfolge verändern	22
5.4.3	Fehlerbehandlung.....	22
5.5	Konfiguration bedingter Verknüpfungen.....	25
5.5.1	Bedingte Verknüpfungen erstellen.....	25
5.5.2	Bedingte Verknüpfungen bearbeiten	26

5.5.3	Reihenfolge der Bedingungen festlegen.....	27
5.6	Definition der Projekteinstellungen	28
5.6.1	Datenaufzeichnung.....	29
5.6.2	Konfiguration	30
5.6.3	Launch-Daten.....	30
5.7	Definition der Umweltbedingungen.....	31
5.8	Speichern einer Session.....	32
5.9	Öffnen einer Session	32
5.10	Exportieren einer Session für die Simulation	33
6	BEDIENUNG DER SIMULATION	34
6.1	Benutzeroberfläche von SILAB.....	34
6.2	Steuerung des Simulationsablaufs	35
6.2.1	Exkurs: Demo-Strecken.....	36
6.2.2	Anleitung zur Steuerung des Simulationsablaufs	38
6.3	Überwachung von Parametern	40
6.3.1	Data Processing Units (DPUs)	41
6.3.2	Auswahl und Anzeigeformen der Parameter	44
6.3.3	Abspeichern der ausgewählten Parameter als Arbeitsbereich	47
6.4	Veränderung von Parameterwerten während der Simulation.....	47
6.5	Überwachung der Simulationsrechner während der Simulation	48
6.6	Auto-Start der Simulation.....	49
6.7	Fernsteuerung der Simulation von anderen Rechnern	50
6.7.1	Funktionen und Benutzeroberfläche von SILABExplorer	50
6.7.2	Einrichten einer Fernsteuerung mit SILABExplorer	52
7	DURCHFÜHRUNG VON UNTERSUCHUNGEN MIT DEM FAHRSIMULATOR	54
8	HILFE BEI PROBLEMEN	55
9	ARBEITEN MIT EIGENEN KONFIGURATIONEN	56
9.1	Vorteile von Konfigurationsdateien	57
9.2	Bestandteile einer Konfigurationsdatei.....	58
9.3	Struktur einer Konfigurationsdatei	60
9.4	Bearbeitung der Konfigurationsdatei	61
9.4.1	Installation und Einrichten des Texteditors Geany	62

9.4.2	Regeln zur Bearbeitung der Konfigurationsdatei	63
9.5	Arbeiten mit der Standardkonfiguration	65
9.6	Systemkonfiguration	67
9.6.1	Launch-Prozess	68
9.6.2	Launch-Daten.....	68
9.6.2.1	Launch der Datenaufzeichnung.....	70
9.6.2.2	Launch des Aufsetzpunktes.....	70
9.6.3	Watchdog-Parameter	71
9.6.4	Systemsteuerungs-Parameter	72
9.7	Rechner-Konfiguration	72
9.8	DPU-Konfiguration	74
9.8.1	Aufbau und Inhalt einer DPU-Konfiguration.....	77
9.8.2	Integration von DPUs in eine bestehende DPU-Konfiguration.....	81
10	INTEGRATION VON HARDWARE.....	84
11	DATENAUFZEICHNUNG	85
11.1	Definition einer Aufzeichnung	85
11.2	Größe einer Aufzeichnungsdatei.....	86
11.3	Aufzeichnungsfrequenz	86
11.4	Aufbau einer Aufzeichnungsdatei	87
11.5	Benennung der Aufzeichnungsdatei.....	87
11.6	Import der Aufzeichnungsdatei in andere Programme	88
11.6.1	Import in Excel	88
11.6.2	Import in Statistica	90
11.6.3	Import in SPSS.....	91
12	ERSTELLUNG VON SCREENSHOTS UND VIDEOS	94
12.1	Screenshots	94
12.2	Videos	95
13	ERSTELLUNG VON DPUS MIT RUBY	98
13.1	Anwendungsbeispiel	98
14	GLOSSAR	102

1 VORWORT

Wir freuen uns, Ihnen mit SILAB die passende Software für Ihren Fahrsimulator zu liefern. SILAB ist die von der WIVW GmbH entwickelte Fahrsimulationssoftware, die sowohl für PKW-, als auch für LKW-, Motorrad- oder Fahrradsimulatoren verwendet werden kann.

SILAB wird seit über 15 Jahren kontinuierlich im engen Austausch mit Partnern und Kunden aus Industrie und Forschung weiterentwickelt und ist für die unterschiedlichsten Anwendungsbereiche optimiert. Der Einsatz der Software eignet sich für die Forschung und Entwicklung von Verkehrs- und Fahrzeugsystemen, für diagnostische Fragestellungen (z.B. Beurteilung der Fahrleistung), für Präsentationszwecke (z.B. Veranschaulichung von neuartigen Konzepten) sowie für Fahrtraining und Rehabilitation.

2 EINFÜHRUNG UND ÜBERBLICK

2.1 Übersicht über SILAB

SILAB ist eine Fahrsimulationssoftware, mit der Fahrsimulatoren mit sehr unterschiedlichen Ausbaustufen betrieben werden können. Die Software bietet die Möglichkeit, das Verhalten des simulierten Fahrzeugs und anderer Verkehrsteilnehmer, die virtuelle Umgebung sowie Fahrzeug- und Umgebungsgeräusche realistisch und flexibel zu simulieren. Zur Streckengestaltung können bereits erprobte und sofort einsetzbare Szenarien mithilfe einer bedienfreundlichen grafischen Benutzeroberfläche zusammengesetzt werden, deren Anwendung keinerlei Programmierkenntnisse erfordert. Darüber hinaus können Anwender bei Bedarf selber Szenarien entwerfen, die sie flexibel wiederverwenden und modifizieren können. Das Streckennetz von SILAB nutzt eine dynamische Datenbasis, die große Vorteile bei der Planung und Durchführung von Simulationsfahrten gegenüber anderer Fahrsimulationssoftware bietet. Im Gegensatz zu einem festen Streckennetz in der Realität kann mit SILAB das Streckennetz und alle zugehörigen Variablen (z.B. andere Verkehrsteilnehmer oder die Landschaft) außerhalb des Sichtbarkeitsbereichs des Fahrers beliebig verändert werden. SILAB ermöglicht das Überwachen diverser Parameter während der Simulation sowie die Aufzeichnung von Messgrößen und Videos. Es können externe Hardware- und Softwarekomponenten über flexible Schnittstellen in die Simulation eingebunden werden.

SILAB ist in drei verschiedenen Editionen erhältlich, welche aufeinander aufbauen und unterschiedliche Nutzeranforderungen bedienen:

Die Edition ESSENTIAL ist für Anwender geeignet, die Untersuchungen oder Trainings mit bereits bestehenden Szenarien und Strecken durchführen möchten. Auf Basis umfangreicher Datenbanken aus Stadt-, Autobahn-, und Landstraßen-Szenarien können diese verschiedenen Szenarien einfach zu individuellen Strecken zusammengesetzt werden. Es bestehen die Möglichkeiten, externe Hard- und Software (Bedienelemente, Blickerfassung, Kommunikationsschnittstellen usw.) einzubinden sowie Messwerte, Audio- und Videodaten aus der Simulation zeitsynchron aufzuzeichnen.

Die Edition PROFESSIONAL richtet sich an Anwender, die eigene Szenarien oder komplexe Straßennetzwerke schnell und effizient gestalten möchten. Hierzu stehen ein grafischer Editor sowie eine einfache Skriptsprache zur Verfügung.

Die Edition ENTERPRISE ist für Anwender konzipiert, die eigene Software für die Simulation entwickeln und integrieren möchten. Dafür stehen Schnittstellen für die Programmiersprachen C/C++, Java und Ruby zur Verfügung. Die Schnittstellen erlauben Zugriff auf die Streckengeometrie, die Beschilderung, die Verkehrsteilnehmer (Egofahrer und simulierte Verkehrsteilnehmer) und deren Eigenschaften sowie nahezu alle Werte, die in SILAB verarbeitet und aufgezeichnet werden können. Hiermit ist es möglich, Assistenzsysteme, Sensorsysteme, Nebenaufgaben, komplexe Human-Machine-Interfaces (HMIs) usw. in SILAB zu integrieren.

2.2 Ziel und Adressaten des Handbuchs

Dieses Handbuch vermittelt Grundlagen zum Arbeiten mit SILAB und richtet sich damit sowohl an Einsteiger als auch an fortgeschrittene Anwender aller SILAB-Editionen. Die ersten sieben Kapitel dienen als zentraler Einstiegspunkt für Personen, die die Anwendung von SILAB von Grund auf erlernen möchten. Darauf aufbauend wird in den Kapiteln 8 bis 12 erweitertes Wissen für fortgeschrittene Nutzer vermittelt, die das Grundlagenwissen aus den vorherigen Kapiteln bereits beherrschen und speziellere Nutzungsanforderungen an das Arbeiten mit SILAB haben.

Wir möchten Ihnen mit diesem Handbuch das benötigte Wissen zum Arbeiten mit SILAB so vermitteln, dass Sie auch ohne Informatikkenntnisse problemlos und mit Freude mit unserer Software arbeiten können. Die Erklärungen dieses Handbuchs sind kurzgehalten und werden oftmals tabellarisch dargestellt, um schnelle Erfolgserlebnisse zu ermöglichen. Sollten Sie daraufhin den Bedarf haben, SILAB zu einem höheren Grad an Ihre eigenen speziellen Fragestellungen anzupassen, empfehlen wir Ihnen, im Anschluss die Kapitel für fortgeschrittene Nutzer zu lesen.

Ausgangspunkt für die Nutzung dieses Handbuchs ist die bereits geschehene Inbetriebnahme Ihres Simulators.

2.3 Inhalte des Handbuchs

Das Handbuch erklärt die grundlegende Funktionsweise und Bedienung der Software. Nutzern wird sowohl ein Überblick über die technischen Komponenten und die Programme von SILAB gegeben als auch die Kompetenz zur Bedienung und Anwendung von SILAB für die Fahrsimulation vermittelt.

Zunächst erhalten Sie einen Überblick über die Programme von SILAB und wie diese organisiert sind (Kapitel 0). Es folgen Erklärungen und Hinweise zur Verwaltung der verschiedenen Simulationsrechner (Kapitel 0). Daraufhin lernen Sie Schritt für Schritt, wie Sie mit dem Programm SILABSessionEdit Simulationsstrecken (sogenannte Sessions) zusammenstellen, speichern und exportieren können (Kapitel 0). Kapitel 0 beinhaltet das benötigte Wissen und die Bedienschritte, um die Simulation mithilfe des Programms SILAB zu bedienen. Somit können Sie Sessions öffnen, starten und stoppen. Zusätzlich lernen Sie, wie Sie Fahrparameter, wie z.B. die aktuelle Geschwindigkeit, sowie die Simulationsrechner während der Simulation überwachen können. In Kapitel 0 erfahren Sie, an wen Sie sich bei auftretenden Problemen wenden können.

Möchten Sie die Simulation spezieller auf Ihre individuellen Anforderungen hin gestalten, vermittelt Ihnen Kapitel 9 das benötigte Wissen zum Erstellen eigener Konfigurationen. Darüber hinaus erfahren Sie, wie Sie mithilfe der Konfigurationsdatei Hardware in SILAB integrieren können (Kapitel 0), Daten aufzeichnen (Kapitel 11), Screenshots und Videos einer Simulationsfahrt generieren (Kapitel 0) und darüber hinaus Softwaremodule mit Ruby erstellen (Kapitel 0) können.

An verschiedenen Stellen des Handbuchs wird auf **WEITERFÜHRENDE DOKUMENTE** verwiesen, mit denen Sie Ihr Wissen je nach Ihren Interessen und Aufgabenstellungen erweitern bzw. vertiefen können. Die entsprechenden Dokumente befinden sich in dem Ordner 'D:\SILAB\doc'.

Das Handbuch enthält diverse Kästen mit Informationen und Warnhinweisen. Diese sehen folgendermaßen aus:

Informationen und Hinweise werden mit Hilfe eines solchen Kastens hervorgehoben.

3 BESTANDTEILE DER SOFTWARE

3.1 Programme von SILAB

Das SILAB-Softwarepaket besteht aus unterschiedlichen Programmen – SILAB ist die Fahrsimulationssoftware selbst und die anderen Programme sind Teil des Pakets. Dieses Kapitel gibt einen Überblick über die Anwendungsbereiche der SILAB-Programme, die Sie als Einsteiger nutzen werden. Die Bedienung dieser Programme werden Sie in den Kapiteln 3.1.1, 3.1.2 und 3.1.3 lernen.

3.1.1 SILABAdmin

An der Simulation mit SILAB in Fahrsimulatoren sind verschiedene Rechner beteiligt. Das Programm SILABAdmin wird benötigt, um diese verschiedenen Rechner zu überwachen und zu steuern sowie Updates auf den Rechnern durchzuführen (Kapitel 0). Mit SILABAdmin können Sie folgende Aktivitäten durchführen:

- Starten einzelner oder aller Rechner
- Neustart einzelner oder aller Rechner
- Herunterfahren einzelner oder aller Rechner
- Überwachen des Status der Rechner
- Aufspielen von Updates zur Aktualisierung von Programmen und Strecken

3.1.2 SILABSessionEdit

Das Programm SILABSessionEdit wird genutzt, um Simulationsstrecken zusammenzustellen (Kapitel 0), die später im Fahrsimulator gefahren werden. Simulationsstrecken werden in SILAB Sessions genannt. Mithilfe von SILABSessionEdit wählen Sie vorgefertigte Szenarien aus, legen die Reihenfolge dieser Szenarien fest und definieren diverse Optionen, wie z.B. Wetter und Tageszeit. Zusammengefasst wird das Programm für folgende Aktivitäten genutzt:

- Auswahl der Szenarien
- Festlegen der Szenarien-Reihenfolge
- Festlegen von zusätzlichen Optionen für die Szenarien, wie z.B. Wetterbedingungen und Tageszeit

3.1.3 Das Programm SILAB

Im Wesentlichen wird das Programm SILAB verwendet, um eine Session in der Simulation zu fahren (Kapitel 0). Mithilfe des Programms können Sie die gewünschte Session öffnen und laden, den Simulationsablauf kontrollieren sowie diverse Parameter überwachen. Das Programm stellt im Hintergrund eine Verbindung zu den an der Simulation beteiligten Rechnern her und stellt sicher, dass diese zeitsynchron ihre spezifischen Aufgaben bearbeiten (z.B. Simulation des physikalischen Fahrzeugmodells, Steuerung des simulierten Verkehrs, Generieren der 3D-Bilder für die Fahrersicht). Das Programm SILAB ermöglicht Ihnen folgende Aktivitäten und Ansichten:

- Öffnen und Laden von Sessions in der Simulation
- Steuerung des Simulationsablaufs (Start, Stop, Pause, Resume)
- Übersicht über Parameter zum Fahrverhalten und Anpassung dieser Ansicht
- Speichern von Arbeitsbereichen
- Ansicht der Softwarekomponenten, die an der Simulation beteiligt sind
- Ansicht der Meldungsfenster mit Informationen, Warnungen und Fehlermeldungen

3.2 Organisation von SILAB auf dem Operatorrechner

3.2.1 Operatorrechner

Der sogenannte Operatorrechner dient der gesamten Bedienung des Simulators und ist in der Regel der einzige Rechner, der mit Tastatur und Maus ausgestattet ist. Auf dem Operatorrechner werden SILAB und die dazugehörigen Programme installiert. Alle Bedieneingaben des Nutzers erfolgen an diesem Rechner. Der Operatorrechner ist der Speicherort für die vom Nutzer erstellten Sessions, Arbeitsbereiche und aufgezeichneten Fahrdaten. Alle anderen Rechner, die an der Simulation beteiligt sind, werden vom Operatorrechner aus gesteuert und verwaltet (Kapitel 0).

3.2.2 Installation

Im Regelfall ist SILAB bereits auf Ihren Rechnern installiert. Sollte die Installation noch ausstehen, folgen Sie bitte der separaten Installationsanleitung für SILAB. Diese Anleitung finden Sie in dem Dokument [INSTALLATIONSANLEITUNG SILAB 7.1](#).

3.2.3 Installationsverzeichnis von SILAB

Wenn Sie auf dem Operatorrechner den SILAB-Installationsordner mit dem Windows-Explorer öffnen (z.B. C:\SILAB\ oder D:\SILAB\), werden Ihnen folgende Ordner angezeigt:

PROJECTS

- In diesem Ordner finden Sie ein Unterverzeichnis für Szenarien-Datenbanken, die Sie erworben haben. Sessions, die Sie mit einer solchen Szenarien-Datenbank erstellt haben, speichern Sie bitte im Ordner der jeweiligen Datenbank.

BIN

- In diesem Ordner befinden sich die Programme SILABAdmin.exe, SILABSessionEdit.exe und SILAB.exe (einen Überblick über die Programme finden Sie im Abschnitt 3.2.3).

CACHE

- In diesem Ordner speichert SILAB interne Dateien zur Optimierung, die in der Simulation häufig verwendet werden.

LIB

- In diesem Ordner sind die System-Dateien von SILAB gespeichert. Hierzu gehören z.B. grafische Modelle und Texturen, Sounddateien, Hardwareeinstellungen usw.

DOC

- In diesem Ordner sind alle Dokumentationen zu SILAB abgelegt. Wenn in diesem Handbuch auf weiterführende Dokumente verwiesen wird, sind sie in diesem Ordner zu finden.

Da die Programme SILABAdmin, SILABSessionEdit und SILAB für jede Arbeit mit dem Fahrsimulator benötigt werden, ist das Einrichten einer Verknüpfung zu den Programmen (Speicherort: ‚bin‘) auf dem Windows-Desktop des Operatorrechners hilfreich.

Bitte nehmen Sie in den Ordnern ‚bin‘, ‚cache‘ und ‚lib‘ keine Veränderungen vor (z.B. Umbenennen, Kopieren, Löschen von Dateien). Wenn Sie nicht wissen, wie sich eine bestimmte Aufgabe ohne Veränderungen in diesen Ordnern erledigen lässt, kontaktieren Sie bitte den Support. In dem Ordner ‚Projects‘ können Sie arbeiten und Änderungen vornehmen. Da die Programme SILABAdmin, SILABSessionEdit und SILAB für jede Arbeit mit dem Fahrsimulator benötigt werden, ist das Einrichten einer Verknüpfung zu den Programmen (Speicherort: ‚bin‘) auf dem Windows-Desktop des Operatorrechners hilfreich.

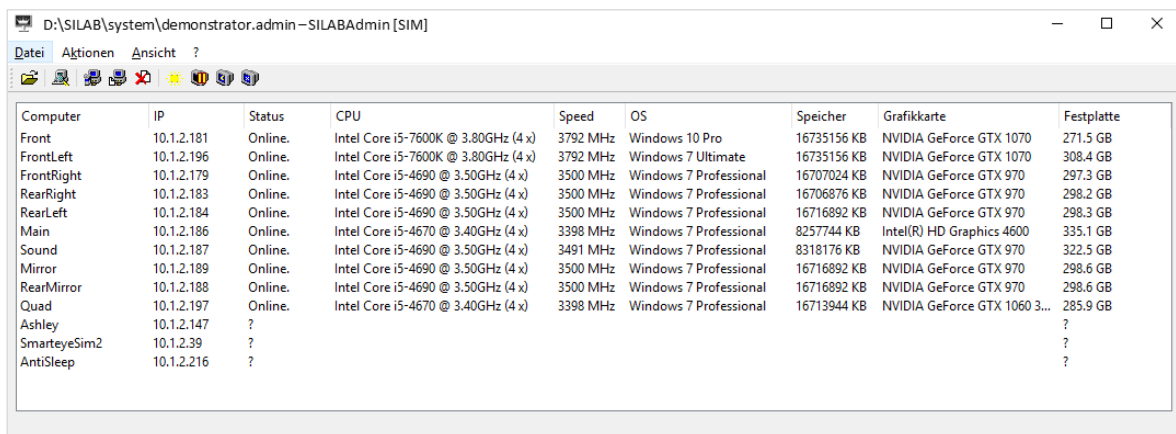
4 VERWALTUNG DER SIMULATIONSRECHNER

In diesem Kapitel lernen Sie, wie Sie die an der Simulation beteiligten Rechner mit dem Programm SILABAdmin überwachen und steuern sowie Updates auf den Rechnern installieren können. Sie erhalten eine Übersicht über die Benutzeroberfläche von SILABAdmin und Anleitungen zur konkreten Anwendung des Programms.

4.1 Benutzeroberfläche von SILABAdmin

Das Programm SILABAdmin finden Sie auf dem Operatorrechner. Sie öffnen das Programm entweder über eine Verknüpfung auf dem Desktop oder aus dem SILAB-Unterverzeichnis ‚bin‘ heraus (s. Abschnitt 3.2.3).

Die Benutzeroberfläche von SILABAdmin sieht folgendermaßen aus:



The screenshot shows the SILABAdmin [SIM] window with a menu bar (Datei, Aktionen, Ansicht, ?) and a toolbar. The main area contains a table with the following data:

Computer	IP	Status	CPU	Speed	OS	Speicher	Grafikkarte	Festplatte
Front	10.1.2.181	Online.	Intel Core i5-7600K @ 3.80GHz (4 x)	3792 MHz	Windows 10 Pro	16735156 KB	NVIDIA GeForce GTX 1070	271.5 GB
FrontLeft	10.1.2.196	Online.	Intel Core i5-7600K @ 3.80GHz (4 x)	3792 MHz	Windows 7 Ultimate	16735156 KB	NVIDIA GeForce GTX 1070	308.4 GB
FrontRight	10.1.2.179	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16707024 KB	NVIDIA GeForce GTX 970	297.3 GB
RearRight	10.1.2.183	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16706876 KB	NVIDIA GeForce GTX 970	298.2 GB
RearLeft	10.1.2.184	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16716892 KB	NVIDIA GeForce GTX 970	298.3 GB
Main	10.1.2.186	Online.	Intel Core i5-4670 @ 3.40GHz (4 x)	3398 MHz	Windows 7 Professional	8257744 KB	Intel(R) HD Graphics 4600	335.1 GB
Sound	10.1.2.187	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3491 MHz	Windows 7 Professional	8318176 KB	NVIDIA GeForce GTX 970	322.5 GB
Mirror	10.1.2.189	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16716892 KB	NVIDIA GeForce GTX 970	298.6 GB
RearMirror	10.1.2.188	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16716892 KB	NVIDIA GeForce GTX 970	298.6 GB
Quad	10.1.2.197	Online.	Intel Core i5-4670 @ 3.40GHz (4 x)	3398 MHz	Windows 7 Professional	16713944 KB	NVIDIA GeForce GTX 1060 3...	285.9 GB
Ashley	10.1.2.147	?						?
SmarteyeSim2	10.1.2.39	?						?
AntiSleep	10.1.2.216	?						?

Abbildung 1. Benutzeroberfläche von SILABAdmin.

Im weiß hinterlegten Hauptbereich des Fensters finden Sie eine Liste aller Rechner, die an der Simulation beteiligt sind. Jeder Rechner wird in der linken Spalte ‚Computer‘ mit seinem Windows-Namen angezeigt. Die Spalte rechts daneben enthält die dazugehörige IP-Adresse. Mittels der Spalte ‚Status‘ können Sie den Status der verschiedenen Rechner überwachen. Hier wird angezeigt, ob ein Rechner gestartet („Online.“) und bereit für einen Simulationslauf ist oder ob SILAB aktuell keine Information über den Status des Rechners hat („?“). Das Fragezeichen-Symbol bedeutet normalerweise, dass dieser Rechner noch nicht hochgefahren bzw. heruntergefahren ist. Bei einem Neustart würde in der Spalte hinter dem Rechnernamen ‚Reboot.‘ stehen. In den restlichen Spalten finden Sie zusätzliche Informationen zur Hardware-Konfiguration der jeweiligen Rechner.

4.2 Steuerung der Simulationsrechner

SILABAdmin wird verwendet, um die für die Simulation benötigten Rechner zu starten, neu zu starten und herunterzufahren. Um eine dieser Aktionen durchzuführen, wählen Sie zunächst die betroffenen Rechner in der Liste mit einem Linksklick aus. Mehrere Rechner gleichzeitig können mit Strg + Linksklick ausgewählt werden. Zur Auswahl der jeweiligen Aktion (Start, Neustart, Herunterfahren) bietet das Programm jeweils zwei alternative Auswahlmöglichkeiten. Jede Aktion lässt sich entweder über einen Linksklick auf den Reiter in der Menüleiste ‚Aktionen → [Auswahl der jeweiligen Aktion]‘ oder das jeweilige Symbol in der Symbolleiste direkt darunter ausführen. Tabelle 1 enthält die Bedienungsanleitungen für die verschiedenen Aktionen mit jeweils zwei Optionen.

Tabelle 1. Steuerung der Simulationsrechner.

Aktion	Option 1: Menüleiste	Option 2: Symbolleiste
Rechner starten	Aktionen → Rechner starten	
Rechner neu starten	Aktionen → Rechner neu starten	
Rechner herunterfahren	Aktionen → Rechner herunterfahren	

Wenn das Rechnernetz zuvor stromlos war (z.B. infolge eines Stromausfalls), kann es sein, dass das Starten der Rechner mit SILABAdmin nicht funktioniert. Bitte schalten Sie in diesem Fall die Rechner manuell ein. Beim nächsten Start sollte die Fernsteuerung über SILABAdmin wieder funktionieren.

Während des Rechnerneustarts verschwinden zunächst die Informationen der Rechner in der Liste. Nach einiger Zeit wird wieder ‚Online.‘ angezeigt und die Rechner sind bereit für die Simulation.

4.3 Installation von Updates auf den Simulationsrechnern

Das Programm SILABAdmin wird zusätzlich benötigt, um Updates von SILAB auf den Simulationsrechnern zu installieren. Der generelle Ablauf wird im Folgenden beschrieben.

- Sichern Sie die Version von SILAB, die aktuell auf Ihrem Simulator installiert ist.
- Installieren Sie die neue Version von SILAB auf dem Operatorrechner.
- Wählen Sie alle Rechner in der Liste von SILABAdmin aus (die Rechner müssen online sein und es darf in diesem Moment keine Simulation laufen).
- Wählen Sie in der Menüleiste über einen Linksklick ‚Aktionen → Verzeichnis übertragen‘. Es erscheint der Standard-Dialog von Windows zur Auswahl von Verzeichnissen. Übertragen Sie schrittweise alle Unterverzeichnisse des Installationsverzeichnisses von SILAB.

Daraufhin kopiert SILABAdmin das gewählte Verzeichnis auf alle Simulationsrechner. Dieser Vorgang kann – je nach Umfang des Updates – einige Zeit dauern.

Die beschriebene Vorgehensweise stellt nur den typischen Ablauf der Installation von Updates dar. Jedem Update von SILAB liegt eine detaillierte Installationsanleitung bei, die Sie unbedingt beachten sollten. Insbesondere bei der Installation auf Simulatoren sollten Sie die exakte Beschreibung befolgen.

Im Folgenden finden Sie weitere Tipps und Hinweise, deren Beachtung einen reibungslosen Rechnerbetrieb für die Simulation mit SILAB ermöglicht:

- Die Betriebsvoraussetzungen des Rechnerherstellers bzgl. der zulässigen Umgebungsparameter (z.B. Temperatur, Luftfeuchtigkeit) sollten erfüllt sein.
- An der Hardware der Rechner, die an der Simulation beteiligt sind, sollten keine Veränderungen vorgenommen werden. Sollte dies aus Wartungsgründen erforderlich sein, sprechen Sie die geplanten Maßnahmen bitte mit unserem Support ab.
- Auf den Simulationsrechnern sollten keine zusätzlichen Programme (z.B. Office-Anwendungen) oder Hintergrundprozesse (z.B. Virens Scanner) installiert werden. Diese Programme benötigen oft Rechenzeit, die während einer laufenden Simulation zu Ruckeln der Grafik oder Abstürzen führen können.
- Treiberaktualisierungen auf den Simulationsrechnern sollten ohne Rücksprache mit dem Support nicht durchgeführt werden.
- Die Simulationsrechner sollten keinen Internetzugriff ermöglichen und nicht in ein bestehendes Netzwerk integriert werden. Diese Maßnahmen schützen die Simulationsrechner vor Schadsoftware und stellen sicher, dass dem Simulator die vollständige Bandbreite des Netzwerks zur Verfügung steht.
- Für den Datenaustausch (z.B. Aufzeichnungsdateien, Konfigurationsdateien, Updates etc.) mit externen Speichermedien und zur Sicherung des Unterordners ‚Projects‘ auf einem externen Speichermedium sollte sichergestellt sein, dass sich keine Schadsoftware auf dem Speichermedium befindet.

5 ZUSAMMENSTELLUNG VON SIMULATIONSSTRECKEN (SESSIONS) MIT SILABSESSIONEDIT

In diesem Kapitel lernen Sie, wie Sie mit dem Programm SILABSessionEdit Simulationsstrecken erstellen, speichern, exportieren und laden können. Da Simulationsstrecken, die mit SILABSessionEdit erstellt werden, Sessions genannt werden, bezeichnen wir diese im weiteren Verlauf nur noch als solche. Die aus Sessions exportierten Simulationsstrecken können daraufhin im Fahrsimulator von Anfang bis Ende durchfahren werden. Sessions bestehen aus einer Abfolge verschiedener Szenarien. Die Auswahl der Szenarien ist ein wesentlicher Bestandteil einer Session-Erstellung. Mit dem Programm SILABSessionEdit können Sie aus bereits bestehenden Szenarien-Datenbanken passende Szenarien auswählen und diese nach einem Baukasten-Prinzip aneinanderreihen. Ein Szenario enthält ein spezifisches Streckenstück mit gegebenenfalls anderen simulierten Verkehrsteilnehmern. Ihnen stehen verschiedene Szenarien-Datenbanken für unterschiedliche Fragestellungen zur Verfügung. Die darin enthaltenen Szenarien nutzen die neuesten Features von SILAB, sind ausgiebig getestet und für die jeweilige Fragestellung hoch sensitiv. Es gibt Szenarien-Datenbanken für folgende Fragestellungen (die Abkürzung ‚SP‘ steht für Szenarienpaket):

SP_BASE

- Diese Datenbank enthält Szenarien in verschiedenen Verkehrsumgebungen (Stadt, Landstraße, Autobahn). Es werden in der Realität häufig auftretende Verkehrsumgebungen ohne die Herstellung spezifischer Verkehrssituationen abgebildet.

SP_DFA

- DFA steht für ‚Driver Fitness and Ability‘. Diese Datenbank enthält Szenarien in verschiedenen Verkehrsumgebungen (Stadt, Landstraße, Autobahn), die sich in bisherigen (z.T. klinischen) Studien gegenüber Müdigkeit und erkrankungs- und substanzbedingter Sedierung als sensitiv erwiesen haben.

SP_EMG

- Diese Datenbank enthält Szenarien in verschiedenen Verkehrsumgebungen (Stadt, Landstraße, Autobahn), die speziell zum Training von Einsatzfahrten entwickelt wurden. Als Zielorte des Einsatzes stehen Unfallorte in allen Verkehrsumgebungen zur Verfügung.

SP_BICYCLE

- Diese Datenbank stellt eine Basis für die Streckengestaltung von Fahrrad-Szenarien zur Verfügung und enthält Landstraßen- und Stadtszenarien.

Die Benennung der Ordner, in denen die Datenbanken gespeichert sind, enthalten noch das entsprechende Kürzel für das jeweilige Land, für das die Szenarien in der Datenbank gebaut wurden (z.B. SPDE_Base für Deutschland). Je nach Land unterscheiden sich z.B. Verkehrszeichen oder Spurmarkierungen.

5.1 Anwendung einer dynamischen Datenbasis

Bevor wir in die konkrete Anleitung zur Zusammenstellung von Sessions einsteigen, möchten wir Ihnen ein Grundverständnis für die Logik vermitteln, mit der in SILAB Szenarien erstellt und verknüpft werden sowie die Vorteile aufzeigen, die sich daraus für die Planung und Durchführung von Simulationsfahrten ergeben. Das Streckennetz von SILAB nutzt eine dynamische Datenbasis (s. Abbildung 2), wodurch das Streckennetz und alle dazugehörigen Variablen (andere Verkehrsteilnehmer, Landschaft usw.) außerhalb des Sichtbereichs des Fahrers beliebig verändert werden können. Ein solches Straßennetzwerk wird als „lokal geometrisch konsistent“ bezeichnet.

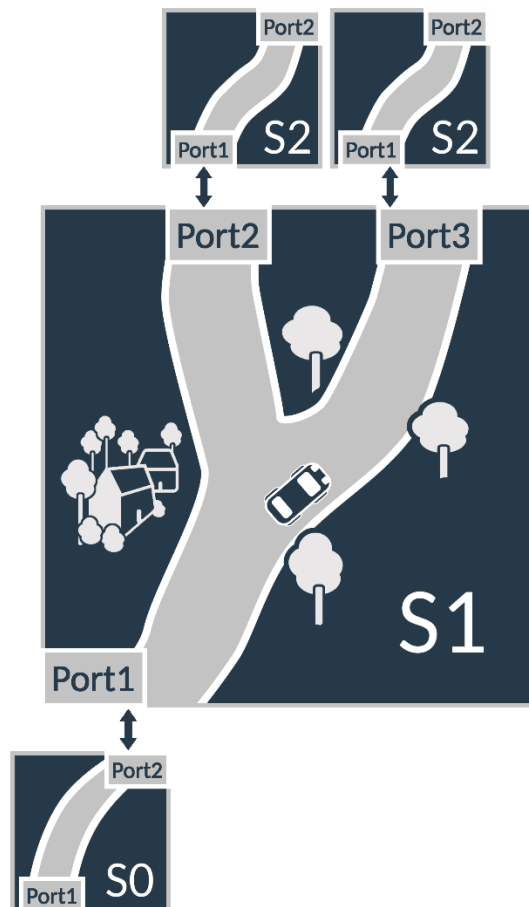


Abbildung 2: Dynamische Datenbasis in SessionEdit. Streckenteile können auf vielfältige Weise aneinandergereiht werden. Ein Verfahren wird verhindert, indem jeder mögliche Ausgang eines Szenarios zum gleichen Folgeszenario führt.

In SILAB besteht eine Fahrt aus Streckenstücken, die nacheinander durchfahren werden. Dabei können die Streckenstücke nahezu beliebig aneinandergereiht werden. Alle Streckenstücke haben in der Regel einen Eingang (siehe z. B. Port1 bei S1 in Abbildung 2) und einen oder mehrere Ausgänge (siehe Port2 und Port3 in Abbildung 2). Unabhängig davon, ob der Fahrer den vorgesehenen Ausgang nimmt oder sich verfährt und einen anderen Ausgang aus dem Szenario nimmt, wird als nächstes das gleiche Szenario angehängt, wie in Abbildung 2 zu sehen ist (S2). Somit ist sichergestellt, dass das nächste Szenario wieder wie beabsichtigt gefahren werden kann. Die Anwendung der dynamischen Datenbasis hat weiterhin den Vorteil, dass Sie beim Festlegen einer Szenarien-Reihen-

folge nicht auf geometrische Einschränkungen achten müssen. Durch die Platzierung entsprechender Sichtverdeckungen (z.B. Häuserzeilen, Mauern) können Sie Szenarien aneinanderreihen, deren Straßengeometrien eigentlich nicht zueinander passen würden. Außerdem kann der Umgebungsverkehr in jedem Szenario neu aufgebaut werden, sodass die Verkehrssituation definiert und reproduzierbar stattfinden kann und nicht vom Verhalten des Fahrers im letzten Szenario abhängt.

In der Realität ist ein Straßennetzwerk (z.B. das einer Stadt oder das Autobahnnetz von Deutschland) „global geometrisch konsistent“ und kann vollständig und maßstabsgerecht auf einer Karte aufgezeichnet werden. Bei Straßennetzwerken für eine Fahrsimulation gilt dies normalerweise ebenfalls. Während der Simulation kann die Position des Fahrers auch kontinuierlich auf dieser Karte angezeigt werden. Global-geometrische Straßennetzwerke sind also starr und eine Fahrstrecke nicht flexibel anpassbar. Somit ist ein Straßennetzwerk in SILAB weitaus flexibler als ein Straßennetzwerk in der Realität.

5.2 SILABSessionEdit-Benutzeroberfläche

Das Programm SILABSessionEdit finden Sie auf dem Operatorrechner. Sie öffnen das Programm entweder über eine Verknüpfung auf dem Desktop oder aus dem SILAB-Unterordner ‚bin‘ heraus (s. Abschnitt 3.2.3). Das Programm gibt es in zwei Versionen: SILABSessionEdit und SILABSessionEdit Light. Die Light-Variante unterscheidet sich hinsichtlich Bedienung und Funktionsumfang von der Standard-Variante. Am Anfang eines jeden relevanten Kapitels wird darauf hingewiesen, welche Unterschiede es bei der Light-Version gibt.

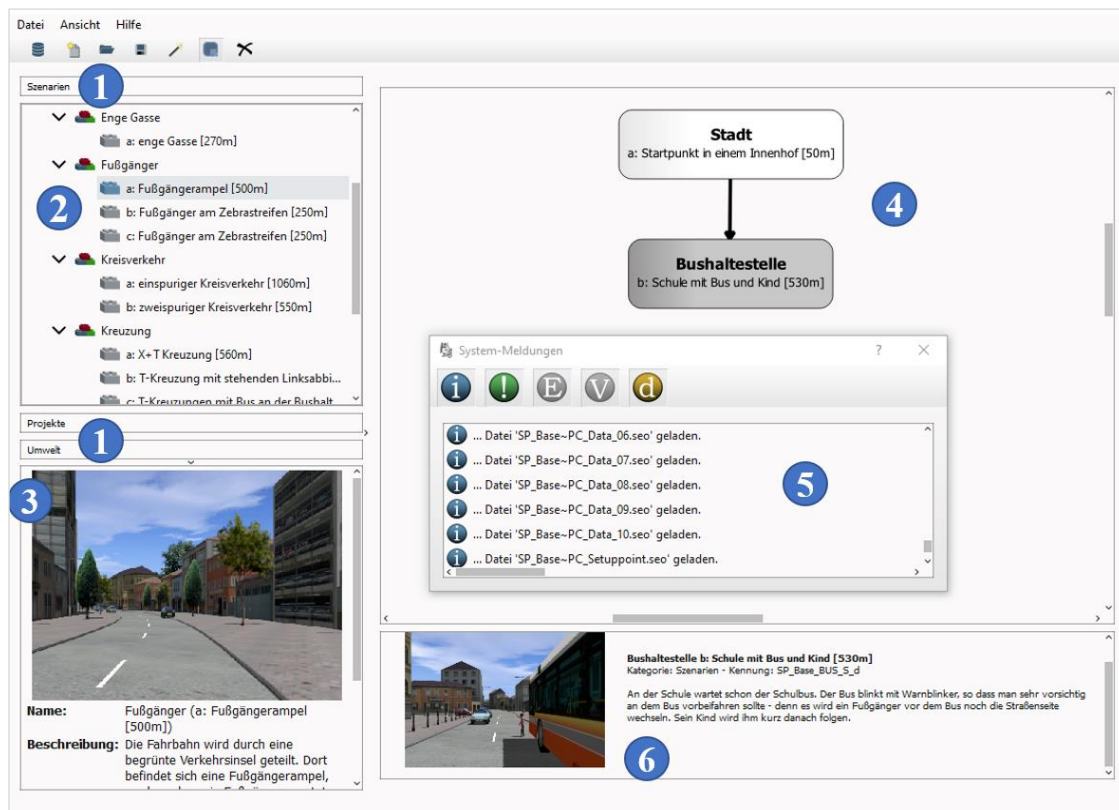


Abbildung 3. Benutzeroberfläche von SILABSessionEdit.

In Tabelle 2 werden die verschiedenen Bereiche der Benutzeroberfläche (s. Abbildung 3) von SILABSessionEdit erklärt.

Tabelle 2. Erklärungen zur Benutzeroberfläche von SILABSessionEdit.

Bereich in Abb. 2	Erklärung
1	Hinter diesen Schaltflächen befinden sich die Kategorien der gewählten Szenarien-Datenbank. In der Abbildung ist beispielsweise die Kategorie ‚Szenarien‘ ausgewählt. Die anderen beiden Kategorien sind ‚Projekte‘ und ‚Umwelt‘ (Erläuterungen in Abschnitt 5.6 und 5.5)
2	In diesem Bereich befindet sich eine hierarchische Ansicht aller Szenarien, die in einer Szenarien-Datenbank enthalten sind und zur Erstellung einer Session verwendet werden können. Zur Verbesserung der Übersicht sind die Szenarien hierarchisch in Kategorien gegliedert (z.B. Oberkategorie: Stadt → Unterkategorie: Fußgänger → Szenario: Fußgängerampel).
3	Wenn Sie ein Szenario in Bereich 2 anklicken, erscheinen in Bereich 3 ein Screenshot und eine Beschreibung mit Informationen zu diesem Szenario.
4	Hier werden die Szenarien dargestellt, die Sie bereits aus Bereich 2 für die Strecke ausgewählt haben, die Sie bauen möchten. Die Pfeilrichtung gibt die Reihenfolge der Szenarien während der Fahrt an.
5	Bei Problemen wird ein Fenster mit Fehlermeldungen eingeblendet, in dem detaillierte Informationen zu dem Problem dargestellt werden.

Die Anzeige unterhalb der Streckenansicht enthält Informationen über das aktuell in Bereich 4 ausgewählte Szenario.

5.3 Öffnen einer Szenarien-Datenbank

Der erste Schritt zur Zusammenstellung einer Session besteht aus dem Öffnen der gewünschten Szenarien-Datenbank. Tabelle 3 erklärt schrittweise mithilfe von Abbildung 3 und Abbildung 4 die nötigen Bedienschritte. Die Szenarien-Datenbank ‚SPDE_Base‘ wird als Beispiel-Datenbank dienen, anhand derer auch die weiteren Schritte zur Zusammenstellung einer Session erklärt werden.

Tabelle 3. Anleitung zum Öffnen einer Szenarien-Datenbank.

Schritt	Ort	Eingabe
1 (Option 1)	Menüleiste (Abbildung 3)	Datei → Datenbank → Datenbank öffnen
1 (Option 2)	Symbolleiste (Abbildung 3)	
2	Dialogfenster (Abbildung 4)	Auswahl der Szenarien-Datenbank, die Sie öffnen möchten (hier: auf SPDE_Base klicken) → Ordner auswählen

SILABSessionEdit öffnet nun die von Ihnen ausgewählte Szenarien-Datenbank. Je nach Umfang der Datenbank kann dieser Vorgang einige Sekunden dauern.

Eine bereits bestehende Session (die Datei hat die Endung *.ses) können Sie im Menü über ‚Datei → Session öffnen‘ öffnen. Die dazugehörige Datenbank wird dabei automatisch geöffnet.

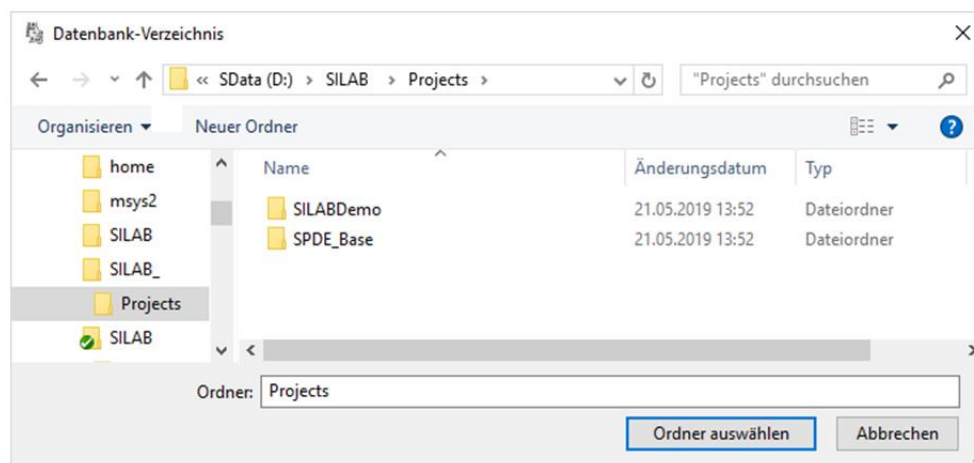


Abbildung 4. Dialogfenster zur Öffnung einer Szenarien-Datenbank.

5.4 Festlegen einer Szenarien-Abfolge

Folgende Features, die in diesem Kapitel beschrieben werden, sind in SessionEdit3 Light nicht enthalten: Einfügen von Streckenstücken aus der Datenbank per Drag and Drop. Das Einfügen per Doppelklick ist möglich.

Nachdem Sie die Szenarien-Datenbank ‚SP_Base‘ (oder eine andere beliebige Datenbank) in SILAB-SessionEdit geöffnet haben, wird Ihnen im Bereich oben links (s. Bereich 1 in Abbildung 5) eine hierarchische Ansicht aller Szenarien angezeigt, die in der ausgewählten Szenarien-Datenbank enthalten sind. Diese können nun zur Erstellung einer Session verwendet werden. Die Anleitung in Tabelle 4 in Kombination mit Abbildung 5 erklärt Ihnen, wie Sie eine Szenarien-Abfolge für eine Session festlegen und verändern können.

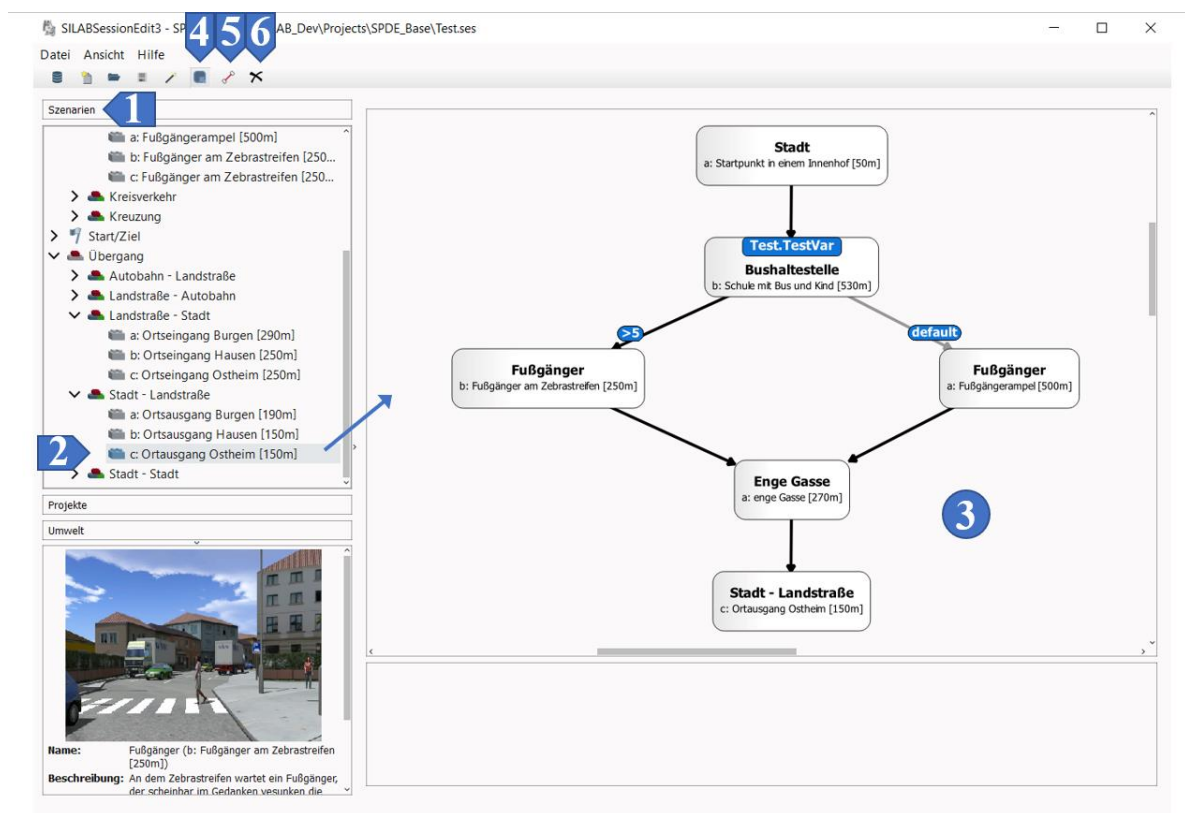


Abbildung 5. Festlegen der Szenarien-Abfolge. Nähere Erklärung zu Nr. 4,5,6 in Kapitel 5.4.1,

Tabelle 4. Anleitung zum Festlegen der Szenarien-Abfolge.

Aktion	Nr. in Abb. 4	Erklärung
Szenario für Session auswählen	1	Wählen Sie die Schaltfläche ‚Szenarien‘.
	2	Klicken Sie auf das Szenario, das Sie in die Session einfügen möchten.
Szenario in Session einfügen		Es gibt zwei alternative Optionen, um ein Szenario in die Session einzufügen:
Option 1	2	Doppelklick auf das ausgewählte Szenario (Nr. 2). Dieses wird unterhalb des zuletzt eingefügten Szenarios in der untersten Reihe platziert. Das neue Szenario wird automatisch mit dem darüberliegenden verknüpft.
Option 2 (<i>nicht verfügbar in Light</i>)	2	Klicken Sie auf das ausgewählte Szenario (Nr. 2), halten Sie die linke Maustaste gedrückt und ziehen Sie das Szenario auf die gewünschte Stelle im Raster. Dieses kann anschließend mit den anderen Szenarien verknüpft werden (s. 5.4.1).
Szenario aus Session entfernen		Es gibt drei alternative Optionen, um ein Szenario aus einer Session zu entfernen:
Option 1	3, 6	Klicken Sie auf das „Löschwerkzeug“ (Nr. 6). Während diese Schaltfläche aktiv ist, können Sie mit der linken Maustaste beliebige Szenarien und Verknüpfungen in der Streckenansicht löschen. Beachten Sie, dass beim Löschen eines Szenarios dessen angrenzende Verknüpfungen ebenfalls gelöscht werden.
Option 2	3	Selektieren Sie zuerst das Szenario (Nr. 3), drücken Sie dann die rechte Maustaste und wählen Sie den entsprechenden Befehl aus dem sich öffnenden Kontextmenü aus.
Option 3	3	Klicken Sie zuerst auf das gewünschte Szenario (Nr. 3) und im Anschluss auf die Tastaturtaste ‚Entf‘.

Sie können nur solche Szenarien in eine Session einfügen, vor denen ein grauer Baustein als Symbol angezeigt wird (s. Abbildung 5, Teilfenster links oben). Die drei bunten Bausteine symbolisieren eine bestimmte Szenarien-Kategorie und noch kein einzelnes Szenario. Die Datenbank SP_Base enthält unter der Kategorie ‚Szenarien‘ mehrere Unterkategorien, in der verschiedene Szenarien zur Auswahl stehen, z.B. Start auf einem Innenhof in der Stadt (s. Pfeil 1 in Abbildung 5).

5.4.1 Szenarien verknüpfen

Dieses Kapitel ist für Nutzer von SILABSessionEdit Light nicht relevant.

Um zwei Szenarien miteinander zu verknüpfen, klicken Sie auf die Schaltfläche „Verknüpfung erstellen“ (s. Pfeil 5 in Abbildung 5). Wählen Sie nun nacheinander zwei Szenarien in der Streckenansicht aus. Damit wird eine Verknüpfung vom ersten zum zweiten gewählten Szenario eingefügt.

Wenn Sie ein Szenario nochmals mit einem vorhergehenden Szenario verknüpfen, werden entstandene Schleifen automatisch erkannt und als grauer, gestrichelter Pfeil dargestellt (s. Abbildung 6). Die Szenarien einer Schleife werden während der Simulation in einer Endlosschleife durchfahren, sofern diese nicht mittels einer bedingten Verknüpfung verlassen wird.

Um eine Verknüpfung zu entfernen, können Sie das „Löschwerkzeug“ (s. Nr. 6 in Abbildung 5) nutzen oder Sie klicken, nachdem Sie die Verknüpfung selektiert haben, mit der rechten Maustaste auf diese und wählen die Option „Verknüpfung löschen“ oder Sie betätigen die „Entf“-Taste, nachdem Sie die entsprechende Verknüpfung ausgewählt haben.

Durch Verknüpfungen bestimmen Sie, in welcher Reihenfolge die Szenarien in der Strecke erscheinen. Als neue Funktionalität können nun in SILAB SessionEdit bedingte Verknüpfungen erstellt werden. Diese werden ausführlich in Kapitel 5.5 beschrieben.

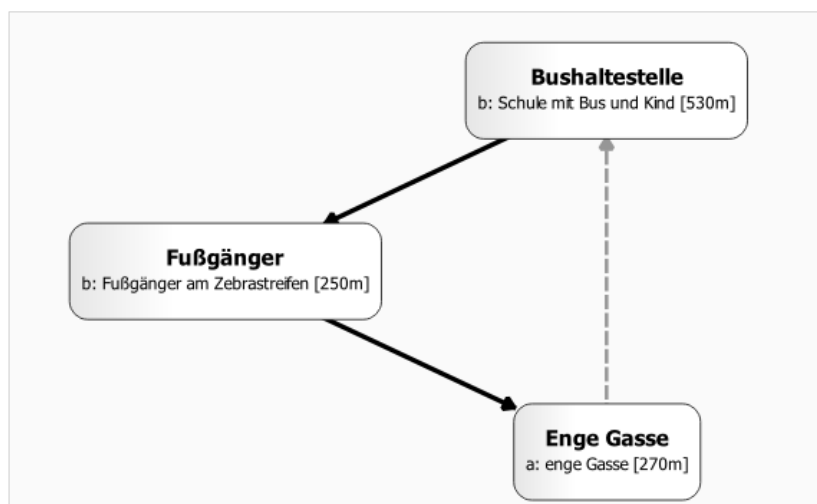


Abbildung 6. Anzeige einer Schleife in der Szenarien-Abfolge. Die drei dargestellten Szenarien werden nun unendlich oft durchfahren.

5.4.2 Szenarienabfolge verändern

Hinweis zu SessionEdit3 Light: Das freie Verändern der Anordnung von Szenarien ist nicht möglich. Das Ändern der Szenarienreihenfolge ist möglich und es können per Drag and Drop Streckenstücke zwischen zwei bereits bestehende Szenarien eingefügt werden. Voraussetzung hierfür ist, dass sich dieses Szenario bereits in der Strecke befindet.

Sie können Szenarien in der Streckenansicht an freie Stellen verschieben, indem Sie das entsprechende Szenario mit der linken Maustaste anklicken und diese gedrückt halten. Wenn Sie ein Szenario verschieben, erscheint ein Raster, das anzeigt, wo das Szenario platziert werden kann. Gültige Rasterzellen sind je nach Designthema in weißer oder schwarzer Farbe gehalten. Andersfarbige Zellen können nicht besetzt werden! Szenarien können auf zwei Arten miteinander vertauscht werden:

- Reihenfolge der Szenarien in einer Session: Hierfür müssen Sie die Verknüpfungen zweier Szenarien umdrehen. Ziehen Sie dazu ein Szenario auf eine Rasterzelle, in der sich bereits ein anderes Szenario befindet.
- Anordnung der Szenarien im SessionEdit: Drücken und halten Sie die SHIFT-Taste, bevor Sie ein Szenario auf ein anderes verschieben. Die beiden Szenarien tauschen Plätze, wobei vorab festgelegte Verknüpfungen unverändert bleiben. An der Reihenfolge, wie die Szenarien in der Strecke durchfahren werden, ändert sich nichts. Dies ist nicht in SILABSessionEdit-Light verfügbar.

Mit der R-Taste können sie alle Szenarien in der Streckenansicht per Zufall umsortieren.

5.4.3 Fehlerbehandlung

Nicht alle Szenarien können an beliebiger Position bzw. in einer beliebigen Reihenfolge im Ablauf stehen. Sie erkennen dies durch rote Fehlermeldungen unterhalb der Szenario-Beschreibung (s. Nr. 2 in Abbildung 7), sowie durch das Erscheinen eines Warnsymbols (s. Nr. 1 in Abbildung 7) in der oberen linken Ecke der Streckenansicht. Solange Sie die linke Maustaste auf diesem Warnsymbol gedrückt halten, werden alle Szenarien rot markiert (s. Nr. 3 in Abbildung 7), bei denen eine Inkonsistenz vorliegt. Auf diese Weise können Sie schnell einen Überblick über Probleme innerhalb Ihrer Strecke gewinnen. Um die Warnmeldung zu lesen, müssen Sie nacheinander die Szenarien anklicken, die eine Warnmeldung enthalten.

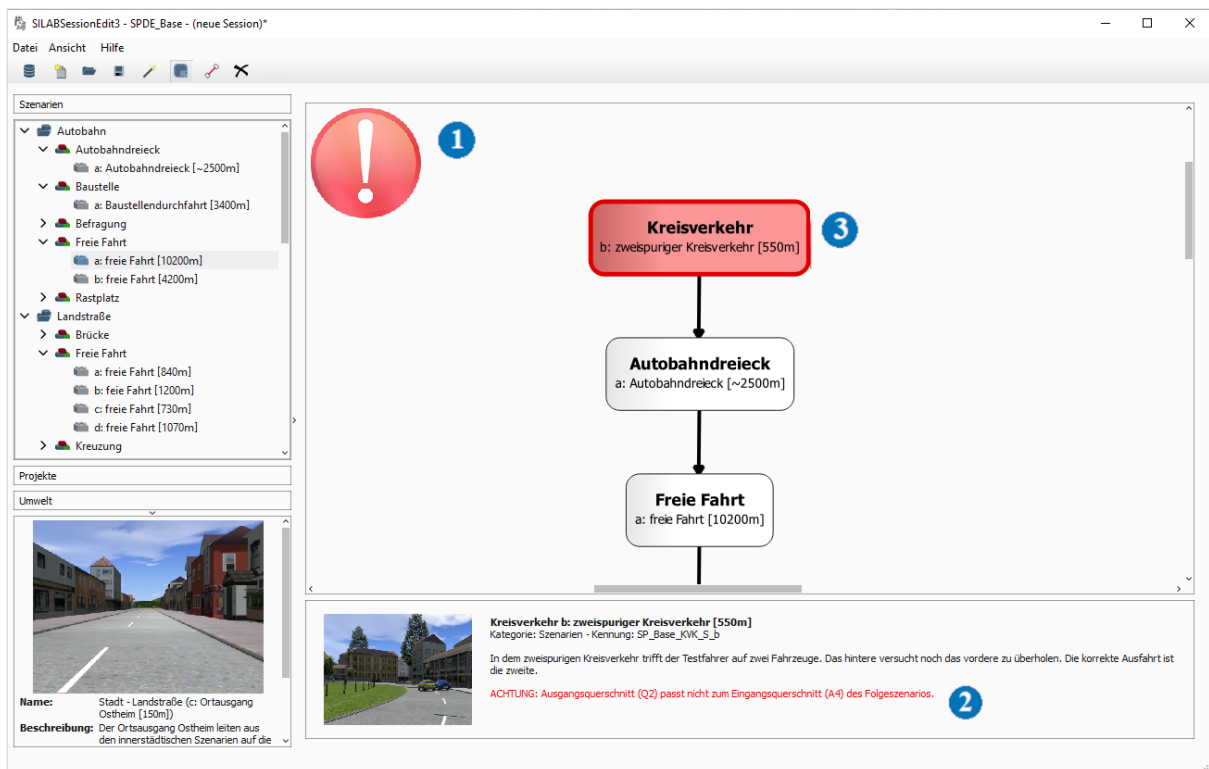


Abbildung 7. Warnsymbol in der Szenarien-Übersicht.

Eine Warnung des Typs „ACHTUNG: Ausgangsquerschnitt (X) passt nicht zum Eingangsquerschnitt (Y) des Folgeszenarios.“ kann folgende Ursachen haben:

- Das Folgeszenario kann nicht angeschlossen werden, weil sein Straßenquerschnitt an der Einfahrt nicht zu dem Querschnitt an der Ausfahrt des vorhergehenden Szenarios passt (in Abbildung 7 sind diese Querschnitte mit Q2 und A4 benannt). Informationen über den Eingangs- und Ausgangsquerschnitt erhalten Sie in der Beschreibung des Szenarios (Pfeil 3 in Abbildung 8). Sie können den Fehler beheben, indem Sie ein entsprechendes Übergangsstück zwischen den beiden Szenarien einfügen. Übergangsstücke finden Sie beispielsweise in der Datenbank SP_Base als Unterkategorie ‚Übergang‘ von ‚Szenarien‘ (Pfeil 2 in Abbildung 8).
- Das Szenario hat keine Einfahrt und kann dementsprechend nur an erster Stelle in der Session erscheinen. Diese Szenarien finden Sie in der Unterkategorie ‚Start/Ziel‘ (Pfeil 1 in Abbildung 8). Der Eingangsquerschnitt solcher Streckenstücke heißt „Start“.
- Das Szenario hat keine Ausfahrt und kann dementsprechend nur an letzter Stelle in der Session erscheinen. Diese Szenarien finden Sie in der Unterkategorie ‚Start/Ziel‘ (Pfeil 1 in Abbildung 8). Der Ausgangsquerschnitt solcher Streckenstücke heißt „Ziel“.

Alle möglichen Warnmeldungen sind in Tabelle 5 aufgelistet.

Tabelle 5: Warnmeldungen in SessionEdit3

Warnmeldung	Erklärung
"ACHTUNG: Ausgangsquerschnitt (X) passt nicht zu Eingangsquerschnitt (Y) des Folgeszenarios!"	Siehe oben.
"ACHTUNG: Dieses Szenario ist mit keinem anderen Szenario verbunden!"	Das Szenario ist noch nicht mit anderen Szenarien verbunden und wird daher nie befahren bzw. wird bei einem StartszENARIO als einziges befahren.
"ACHTUNG: Zwei oder mehr ausgehende Verknüpfungen haben die gleiche Bedingung!"	Ein Szenario hat zwei oder mehr Verknüpfungen, die unter der gleichen Bedingung befahren werden. Das bedeutet, dass mindestens eine dieser Verknüpfungen niemals genutzt wird.
"ACHTUNG: Zwei oder mehr ausgehende Verknüpfungen haben eine Default-Verknüpfung!"	Spezialfall von „Zwei oder mehr ausgehende Verknüpfungen haben die gleiche Bedingung“. Zu dieser Meldung kann es auch kommen, wenn einer Verknüpfung (oder mehreren) gar keine Bedingung zugewiesen wurde, da dies eine Default-Bedingung impliziert. Näheres zu bedingten Verknüpfungen finden Sie in Kapitel 5.5.

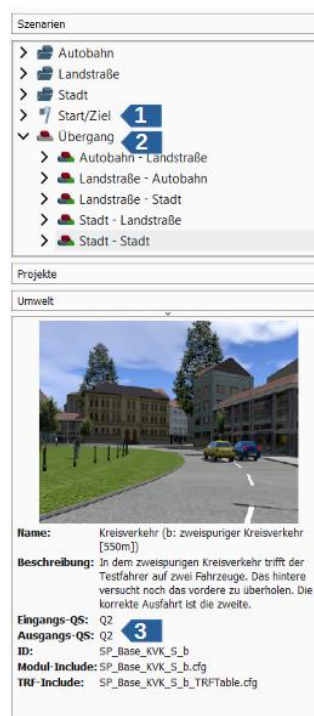


Abbildung 8. Übergangsstücke und Informationen über den Eingangs- und Ausgangsquerschnitt eines Szenarios.

5.5 Konfiguration bedingter Verknüpfungen

In *SessionEdit3-Light* ist es nicht möglich, Bedingungen zu erstellen, oder zu bearbeiten. Daher muss dieses Kapitel von Nutzern von *SILABSessionEdit Light* nicht gelesen werden.

In *SILABSessionEdit3* haben Sie die Möglichkeit, bedingte Verknüpfungen zwischen Szenarien einzufügen. Das bedeutet, dass SILAB während der Simulation anhand von Variablenwerten entscheiden kann, welcher Streckenabschnitt als nächstes befahren wird.

5.5.1 Bedingte Verknüpfungen erstellen

Die Streckenansicht ist so gestaltet, dass Sie Szenarien sehr einfach mit mehreren Verknüpfungen ausstatten können, ohne dabei den Überblick zu verlieren. Nutzen Sie dabei nach Belieben das Werkzeug „Verknüpfung erstellen“ in der Toolbar (s. Pfeil 5 in Abbildung 5).

Damit SILAB weiß, welches Szenario der Fahrer als nächstes erreichen soll, müssen Sie in *SILABSessionEdit3* zuvor spezifizieren, welche Bedingung für welche Verknüpfung geprüft werden soll. Es gibt drei Möglichkeiten, die Bedingung einer Verknüpfung zu editieren.

- Option 1: Mit der Schaltfläche „Auswahl“ können Sie neben Szenarien auch Verknüpfungen markieren. Betätigen Sie die rechte Maustaste, nachdem Sie eine Verknüpfung ausgewählt haben und wählen Sie im Kontext Menü „Bedingung bearbeiten“. Es öffnet sich ein Dialog, in dem Sie der Verknüpfung eine neue Bedingung hinzufügen können.

- Option 2: Klicken Sie in der Toolbar auf „Verknüpfung erstellen“. Wählen Sie zwei Szenarien aus, zwischen denen bereits eine Verknüpfung in die angedeutete Richtung existiert. Anstatt eine neue Verknüpfung zu erstellen, öffnet sich der Dialog aus Option 1.
- Option 3: Im Dialog „Bedingungen anordnen“. Dieser wird im Kapitel „Strecken exportieren“ näher erläutert.

5.5.2 Bedingte Verknüpfungen bearbeiten

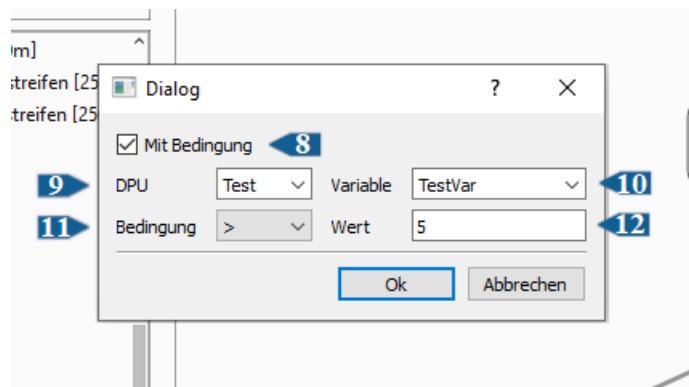


Abbildung 9: Bearbeiten von Bedingungen

Wollen Sie eine Verknüpfung mit einer Bedingung versehen, setzen Sie zuerst das Häkchen bei „Mit Bedingung“ (s. Pfeil 8 in Abbildung 9). Im Dialogfeld „DPU“ (s. Pfeil 9 in Abbildung 9) und „Variable“ (s. Pfeil 10 in Abbildung 9) geben Sie an, welche Variable bei einer Abzweigung geprüft werden soll. Mit dem Pfeil öffnen Sie ein Drop-Down Menü, das Ihnen einige mögliche Werte vorschlägt. Alternativ können Sie auch eigene Namen eintippen. Im Drop-Down-Menü „Bedingung“ (s. Pfeil 11 in Abbildung 9) bestimmen Sie die gewünschte Vergleichsoperation. Sie haben dabei die Wahl zwischen den Operationen in Tabelle 6.

Tabelle 6: Mögliche Operationen zum Prüfen einer Bedingung.

Operation	Bedeutung
<	Kleiner-als „Wert“
>	Größer-als
<=	Kleiner-gleich
>=	Größer-gleich
=	Ist-gleich
!=	Ungleich

default	Alternativer Weg, wenn alle anderen geprüften Bedingungen einer Abzweigung nicht erfüllt sind.
---------	--

Schließlich geben Sie bei „Wert“ (s. Pfeil 12 in Abbildung 9) an, auf welche Zahl genau geprüft werden soll. Wenn Sie den Haken bei „Mit Bedingung“ entfernen, wird eine default-Bedingung angenommen, d.h., das folgende Szenario wird immer geladen, sofern die Bedingung keiner anderen Verknüpfungen erfüllt ist.

5.5.3 Reihenfolge der Bedingungen festlegen

Wenn Sie mit der rechten Maustaste das Kontextmenü eines Szenarios aufrufen, sehen Sie dort die Option „Bedingungen anordnen...“. Diese ist anwählbar, sobald das betreffende Szenario ein Ausgangspunkt für bedingte Verknüpfungen ist. Klicken Sie auf diese Option, öffnet sich ein Dialog, der eine Tabelle aller bedingten Verknüpfungen anzeigt, ausgehend vom gewählten Szenario. Mit den Pfeilen (s. Pfeil 1 in Abbildung 10) können Einträge nach oben bzw. unten verschoben werden. Die hier festgelegte Ordnung bestimmt die Auswertungsreihenfolge der Bedingungen während der Fahrt. Die am weitesten oben stehende Bedingung wird zuerst geprüft und sollte diese erfüllt sein, wird der Strecke das passende Szenario angehängt, ohne alle weiteren Bedingungen abzufragen.

Sie haben in diesem Dialog ebenfalls die Möglichkeit, bestehende Bedingung des gewählten Szenarios zu bearbeiten. Es gibt zwei Textfelder, um DPU- (s. Pfeil 2 in Abbildung 10) und Variablennamen (s. Pfeil 3 in Abbildung 10) der Bedingung zu verändern. Einträge in den Tabellenspalten „Operation“ (s. Pfeil 4 in Abbildung 10) und „Wert“ (s. Pfeil 5 in Abbildung 10) können Sie hier ebenso editieren, indem Sie auf der entsprechenden Zelle einen Doppelklick ausführen. Die Spalte „Strecke“ (s. Pfeil 6 in Abbildung 10) ist nicht editierbar. Sie gibt an, mit welchem anderen Streckenabschnitt das Szenario verknüpft ist.

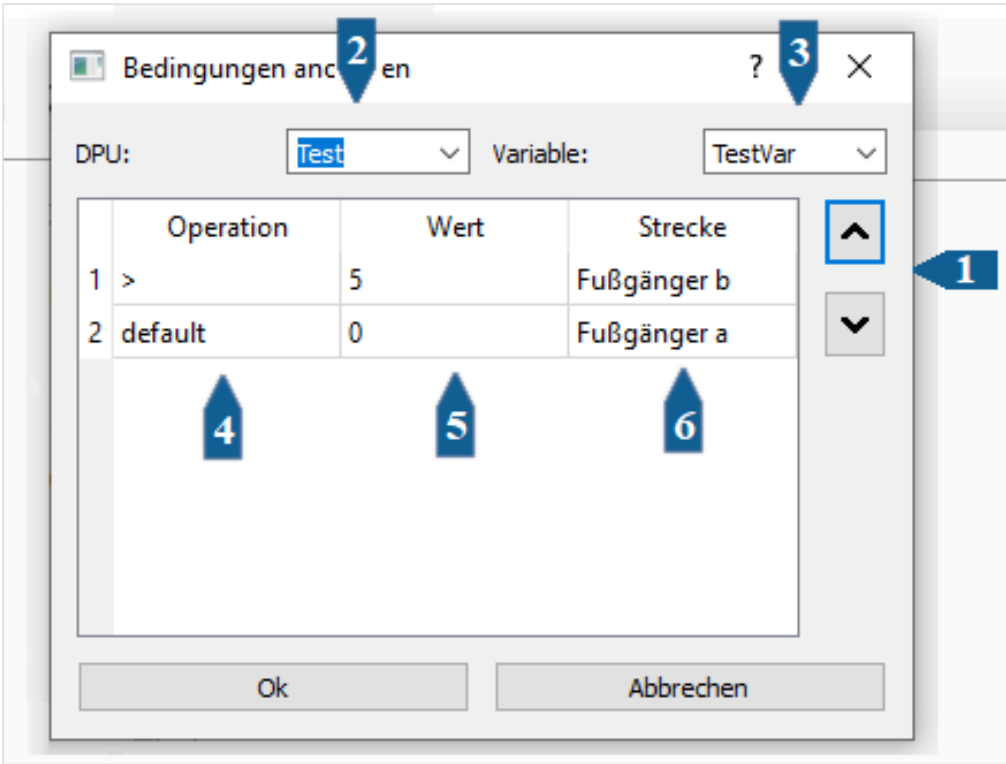


Abbildung 10. Festlegen der Bedingungsreihenfolge.

5.6 Definition der Projekteinstellungen

Für Ihre erstellte Session können Sie verschiedene Einstellungen zur Versuchssteuerung vornehmen. Diese sogenannten Projekteinstellungen können aus Einstellungen zur Datenaufzeichnung, Konfiguration und Launch-Daten bestehen.

Ob und welche Projekteinstellungen Ihnen zur Verfügung stehen, hängt von der jeweiligen Szenarien-Datenbank ab, mit der Sie arbeiten. Nicht in jeder Szenarien-Datenbank stehen die gleichen Optionen für Projekteinstellungen zur Verfügung.

Die nachfolgende Anleitung (Tabelle 7) erklärt Ihnen in Kombination mit Abbildung 11, wie Sie Projekteinstellungen auf Sessions anwenden und wieder entfernen können.

Tabelle 7. Anleitung zur Definition der Projekteinstellungen.

Schritt	Aktion	Nr. in Abb.Ab-bildung 11	Erklärung
1		1	Wählen Sie die Schaltfläche mit der Kategorie ‚Projekte‘ aus.

	Kategorie 'Projekte' auswählen	2	Wählen Sie in der Baumansicht die gewünschte Option (z.B. 'Standard Datenaufzeichnung').
2	Option auf Session anwenden	2	Klicken Sie auf die einzufügende Option (Nr. 2), halten Sie die linke Maustaste gedrückt und ziehen Sie die Option in die Liste im rechten Teil des Hauptfensters.
3	Option entfernen	3, 4	Klicken Sie auf die ausgewählte Option (Nr. 3) und im Anschluss auf das Symbol Nr. 4 in der Symbolleiste.

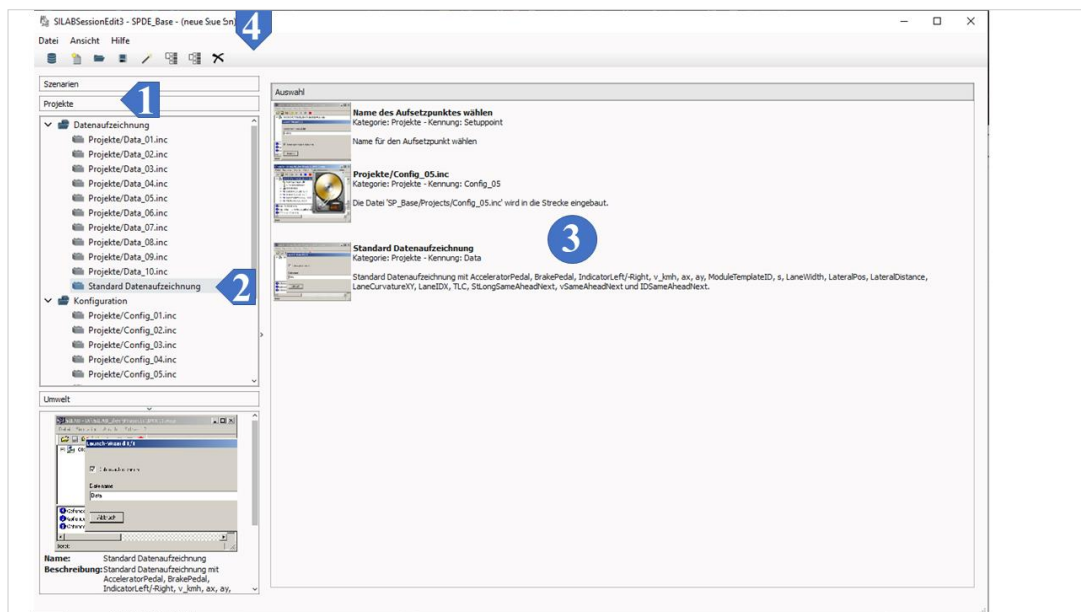


Abbildung 11. Festlegen der Projekteinstellungen.

5.6.1 Datenaufzeichnung

Bei Bedarf können Sie während der Simulation Messdaten zum Fahrverhalten aufzeichnen und diese Daten im Nachgang in ein Statistikprogramm importieren und analysieren. Mit SILABSessionEdit können Sie unter ‚Projekte → Datenaufzeichnung‘ die Option ‚Standard Datenaufzeichnung‘ auswählen. In der Beschreibung dieser Option (im Fenster unten links in Abbildung 11) finden Sie eine Aufzählung der Fahrparameter, die mit dieser Einstellung standardmäßig aufgezeichnet werden. Weitere Beschreibungen zu diesen Parametern finden Sie im Abschnitt 11. Wenn Sie die Option ‚Standard Datenaufzeichnung‘ auf Ihre Session anwenden, werden Sie vor jedem Start dieser Session in der Simulation mit einem Dialogfenster aufgefordert, den Namen der Aufzeichnungsdatei einzugeben. Außerdem können Sie vor jedem Start auswählen, ob Sie im aktuellen Simulationsdurchlauf Daten aufzeichnen möchten oder nicht. Das Dialog-Fenster, das zu diesem Zwecke vor dem Simulationsstart erscheint, wird ‚Launch-Wizard‘ genannt und sieht folgendermaßen aus:

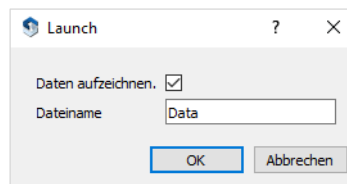


Abbildung 12. Launch-Wizard zur Datenaufzeichnung in SILAB.

Für die Datenbank SP_Base bestehen hier weiterhin die Optionen Data_01.inc bis Data_10.inc. In diesen Dateien können Datenaufzeichnungen für bestimmte Projekte definiert werden. Dies setzt allerdings Kenntnisse zum Bearbeiten von Konfigurationsdateien voraus. Dieses Wissen wird in Kapitel 9 und 10 vermittelt.

5.6.2 Konfiguration

Für die Datenbank SP_Base besteht die Option, unterschiedliche Konfigurationen einzubinden. In den Dateien Config_01.inc bis Config_10 sind in neu installierten Szenarien-Datenbanken zunächst keine Konfigurationen enthalten (die Dateien sind leer). Die Erstellung von eigenen Konfigurationen erfordert Kenntnisse zum Bearbeiten von Konfigurationsdateien und wird in Kapitel 9 thematisiert.

5.6.3 Launch-Daten

Mit den Einstellungen zu den Launch-Daten können Sie festlegen, welche Einstellungen Sie unmittelbar vor dem Start der Session in dem Programm SILAB treffen möchten.

Mit der Option ‚Aufsetzpunkt auswählen‘ haben Sie die Möglichkeit, die Session an unterschiedlichen Punkten starten zu lassen. Als Aufsetzpunkt wird in SILAB eine Stelle in der Session bezeichnet, an der die Simulation gestartet werden kann. Wenn diese Option nicht ausgewählt wird, wird der Fahrer immer am Startpunkt der Session aufgesetzt. Wenn Sie diese Option auswählen, haben Sie die Freiheit, vor dem Start jedes Simulationsdurchlaufs festzulegen, bei welchem Szenario die Simulationsfahrt starten soll. Die Simulation wird dann am Eingang des ausgewählten Szenarios gestartet. Abbildung 13 zeigt den Launch-Wizard, mit dem Sie in SILAB vor dem Start der Simulation den Aufsetzpunkt bestimmen können.

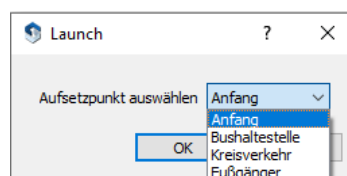


Abbildung 13. Launch-Wizard zur Auswahl des Aufsetzpunktes in SILAB.

Mit der Option ‚Start ohne Launchdatendialog‘ wird die Simulationsumgebung direkt nach dem Ladevorgang gestartet, ohne dass der Launch- und Start-Button ausgewählt werden muss (s. Abschnitt 6.2.2).

5.7 Definition der Umweltbedingungen

Sie haben die Möglichkeit, Umweltbedingungen für eine Session zu definieren. Eine Übersicht aller verfügbaren Optionen finden Sie in der Dokumentation der jeweiligen Szenarien-Datenbank.

Welche Optionen Ihnen zur Verfügung stehen, hängt von der jeweiligen Szenarien-Datenbank ab, mit der Sie arbeiten. Nicht in jeder Szenarien-Datenbank stehen die gleichen Umwelt-Optionen zur Verfügung.

Die nachfolgende Anleitung (Tabelle 8) erklärt Ihnen in Kombination mit Abbildung 14, wie Sie Optionen auf Sessions anwenden und wieder entfernen können. Die gewählte Umweltbedingung wird über die gesamte Session hinweg verwendet.

Tabelle 8. Anleitung zur Anwendung von Optionen auf Sessions.

Schritt	Aktion	Nr. in Abb. 13	Erklärung
1	Kategorie ‚Umwelt‘ auswählen	1	Wählen Sie die Schaltfläche mit der Kategorie ‚Umwelt‘ aus.
		2	Wählen Sie in der Baumansicht die gewünschte Option (z.B. mittlerer Regen).
2	Option auf Session anwenden	2	Klicken Sie auf die einzufügende Option (Nr. 2), halten Sie die linke Maustaste gedrückt und ziehen Sie die Option in die Liste im rechten Teil des Hauptfensters.
3	Option entfernen	3, 4	Klicken Sie auf die ausgewählte Option (Nr. 3) und im Anschluss auf das Symbol Nr. 4 in der Symbolleiste.

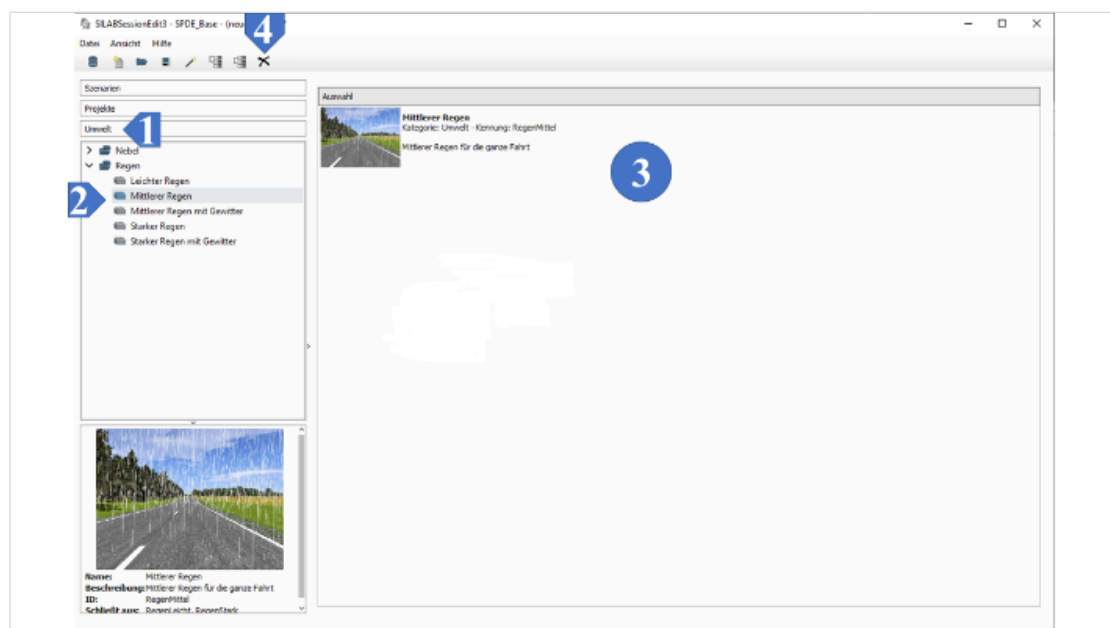


Abbildung 14. Festlegen von Umweltbedingungen.

5.8 Speichern einer Session

In diesem Abschnitt erklären wir Ihnen, wie Sie Ihre erstellten Sessions erstmalig speichern, während der Bearbeitung zwischenspeichern und unter einem neuen Namen speichern können.

Tabelle 9. Anleitung zum Speichern einer Session.

Aktion	Schritt	Ort	Erklärung
Erstmaliges Speichern einer neuen Session	1 (Option 1)	Menüleiste	Datei → Speichern
	1 (Option 2)	Symbolleiste	
	2	Dialogfeld	Wählen Sie im geöffneten Dialogfeld das Verzeichnis der Szenarien-Datenbank, mit der Sie aktuell arbeiten, oder ein von Ihnen erstelltes Unterverzeichnis aus.
	3	Dialogfeld	Geben Sie einen Dateinamen für die Session ein.
	4	Dialogfeld	Klicken Sie auf ‚Speichern‘.
Session unter neuem Namen speichern	1	Menüleiste	Datei → Speichern unter...
	2	Dialogfeld	Wählen Sie im geöffneten Dialogfeld das Verzeichnis der Szenarien-Datenbank, mit der Sie aktuell arbeiten, oder ein von Ihnen erstelltes Unterverzeichnis aus.
	3	Dialogfeld	Geben Sie einen Dateinamen für die Session ein.
	4	Dialogfeld	Klicken Sie auf ‚Speichern‘.

Wenn Sie eine Session zwischenspeichern möchten, folgen Sie ebenfalls dem Pfad ‚Datei → Speichern‘ in der Menüleiste oder klicken Sie auf das Disketten-Symbol in der Symbolleiste. Dieser Menübefehl steht nur zur Verfügung, wenn Sie seit dem letzten Speichern Änderungen an der Session vorgenommen haben.

5.9 Öffnen einer Session

Die nachfolgende Anleitung erklärt Ihnen, wie Sie eine bereits erstellte oder schon bestehende Session in SILABSessionEdit öffnen, um sie daraufhin zu bearbeiten.

Tabelle 10. Anleitung zum Öffnen einer Session.

Aktion	Schritt	Ort	Erklärung
Öffnen einer Session	1 (Option 1)	Menüleiste	Datei → Öffnen
	1 (Option 2)	Symbolleiste	

2	Dialogfeld	Wählen Sie im geöffneten Dialogfeld die entsprechende Session-Datei mit der Dateiendung *.ses aus.
3	Dialogfeld	Klicken Sie auf ‚Öffnen‘.

5.10 Exportieren einer Session für die Simulation

Bei der Erstellung einer Session haben Sie stets das Ziel, diese Session nach ihrer Fertigstellung in der Simulation zu fahren. Damit die Session mit dem Programm SILAB geöffnet und im Simulator gefahren werden kann, muss die Session-Datei zunächst in eine Konfigurationsdatei mit der Dateiendung *.cfg umgewandelt werden. Eine exportierte Session wird in SILAB als ‚Parcours‘ bezeichnet. Um eine Session zu exportieren, d.h. um sie als Konfigurationsdatei abzuspeichern, folgen Sie bitte der Anleitung in Tabelle 11.

Tabelle 11. Anleitung zum Exportieren einer Session.

Aktion	Schritt	Ort	Erklärung
Exportieren einer Session	1 (Option 1)	Menüleiste	Datei → Parcours exportieren
	1 (Option 2)	Symbolleiste 	
	2	Dialogfeld	Wählen Sie im geöffneten Dialogfeld das Verzeichnis der Szenarien-Datenbank, mit der Sie aktuell arbeiten, oder ein von Ihnen erstelltes Unterverzeichnis aus.
	3	Dialogfeld	Geben Sie einen Dateinamen für die zu exportierende Session ein.
	4	Dialogfeld	Klicken Sie auf ‚Speichern‘.

6 BEDIENUNG DER SIMULATION

In diesem Kapitel lernen Sie, wie Sie das Programm SILAB nutzen, um einen Parcours in der Simulation zu fahren und den Simulationsablauf zu kontrollieren. Sie können nun Sessions, die Sie mithilfe von SILABSessionEdit selbst aus Szenarien-Datenbanken erstellt haben und als Parcours für die Simulation exportiert haben (Kapitel 0), in der Simulation fahren. Darüber hinaus sind im Lieferumfang von SILAB Demo-Strecken enthalten. Dabei handelt es sich um vorgefertigte Strecken in verschiedenen Verkehrsumgebungen und mit unterschiedlichen Umweltbedingungen, die Sie auch ohne vorherige Nutzung von SILABSessionEdit zur Simulatorfahrt nutzen können.

Sie werden lernen, wie sie mit der grafischen Benutzeroberfläche von SILAB Sessions öffnen, laden, an unterschiedlichen Stellen starten, pausieren und stoppen können.

6.1 Benutzeroberfläche von SILAB

Das Programm SILAB finden Sie auf dem Operatorrechner. Sie öffnen das Programm entweder über eine Verknüpfung auf dem Desktop oder aus dem SILAB-Unterverzeichnis 'bin' heraus (s. Abschnitt 3.2.3).

Die Benutzeroberfläche von SILAB sieht folgendermaßen aus:

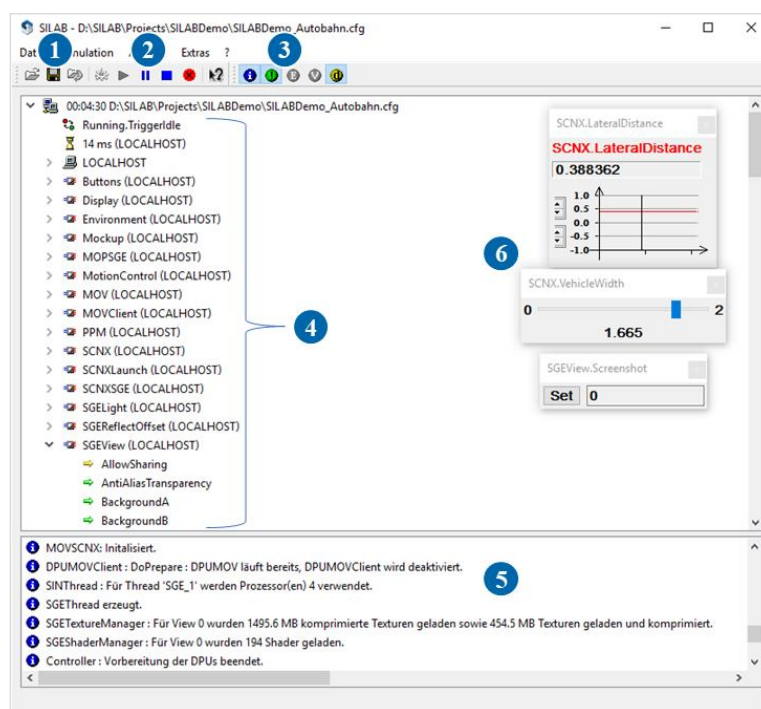


Abbildung 15. Benutzeroberfläche von SILAB.

Die nummerierten Bereiche in Abbildung 15 werden in Tabelle 12 näher erläutert.

Tabelle 12. Erklärung zur Benutzeroberfläche von SILAB.

Bereich in Abb. 11	Erklärung
1	Im linken Bereich der Symbolleiste können Sessions und Arbeitsbereiche geöffnet  und gespeichert  werden.
2	Der mittlere Bereich der Symbolleiste wird zur Steuerung des Simulationsablaufs genutzt:
	 Laden
	 Starten
	 Pausieren
	 Stoppen
	 Notausschalter
3	 Erklärungen für die verschiedenen Bereiche der Benutzeroberfläche
	Mit dem rechten Bereich der Symbolleiste kann die Anzeige des Meldungsfensters konfiguriert werden. Sie können wählen und abwählen, welche Meldungen angezeigt werden sollen:
	 Allgemeine Informationen
	 Warnungen
	 Fehlermeldungen
	 Versionsfehler
4	 Debug-Informationen
	In diesem Bereich befindet sich eine Baumansicht mit allen an der Simulation beteiligten Rechnern, laufenden Softwarekomponenten und deren Parametern.
	1. Zeile Aktuelle Simulationszeit (gemessen vom Start eines Simulationslaufs), Pfad und Dateiname der geladenen Session
	 Information über den Gesamtstatus der Simulation
	 Information über Verzögerungen bei der Durchführung der Simulation (in ms) (nähere Informationen in Abschnitt 6.5)
	 An der Simulation beteiligte Rechner mit ihren Namen. Detailliertere Informationen finden sich, wenn eine Zeile aufgeblättert wird.
5	 An der Simulation beteiligte Softwarekomponenten (nähere Informationen in Abschnitt 6.3.1)
	In diesem Meldungsfenster werden Informationen, Warnungen und Fehler zur laufenden Simulation angezeigt.
6	In diesem Bereich können Sie die Anzeigen zur Überwachung von Simulationsparametern platzieren.

6.2 Steuerung des Simulationsablaufs

Bevor wir Ihnen in diesem Kapitel Schritt für Schritt erklären, wie Sie den Simulationsablauf mit SILAB steuern können, erhalten Sie zunächst Informationen über die mitgelieferten Demo-Stre-

cken. Diese Informationen sind vorab interessant, da Sie anhand der Demo-Strecken die Anwendung von SILAB in diesem Kapitel lernen können, auch wenn Sie selber noch keine eigenen Sessions mit dem Programm SILABSessionEdit erstellt haben.

6.2.1 Exkurs: Demo-Strecken

Demo-Strecken sind vorgefertigte Konfigurationen, mit denen die verschiedenen Features von SILAB umfassend bei einer Simulatorfahrt demonstriert werden können. Sie sind im Ordner ‚SILAB-Demo‘ gespeichert, welcher ein Unterordner von ‚Projects‘ ist (siehe Screenshot in Abbildung 16). Im Lieferumfang von SILAB sind folgende Demo-Strecken enthalten:

SILABDEMO

- besteht aus Fahrabschnitten auf der Autobahn, in der Innenstadt und auf der Landstraße. Die Session startet auf einem Parkplatz. Durch Folgen der Markierungspfeile erreicht der Fahrer nach kurzer Fahrt einen Kreisverkehr. Die verschiedenen Ausfahrten des Kreisverkehrs führen entweder auf die Autobahn, in die Innenstadt oder auf die Landstraße. Die Beschilderung des Kreisverkehrs ist in Abbildung 17 dargestellt. Je nachdem, an welcher Ausfahrt der Kreisverkehr verlassen wird, kann der Fahrer sich einen anderen Abschnitt der Demostrecke anschauen.

SILABDEMO_AUTBAHN

- Enthält die Autobahnfahrt aus der übergreifenden Demo-Strecke ‚SILABDemo‘

SILABDEMO_CITY

- Enthält die Innenstadt-Fahrt aus der übergreifenden Demo-Strecke ‚SILABDemo‘.

SILABDEMO_COUNTRY

- Enthält die Fahrt auf der Landstraße aus der übergreifenden Demo-Strecke ‚SILABDemo‘.

SILABDEMO_COUNTRY_WINTER

- Enthält eine ähnliche Fahrt auf der Landstraße wie in der Demo-Strecke ‚SILABDemo‘ in winterlicher Landschaft.

SILABDEMO_HIGHWAY

- Enthält eine Fahrt auf der Autobahn, die hauptsächlich der Demonstration der Möglichkeiten zur Verkehrssteuerung mit SILAB dient, insbesondere der Bildung und Auflösung von Stauwellen.

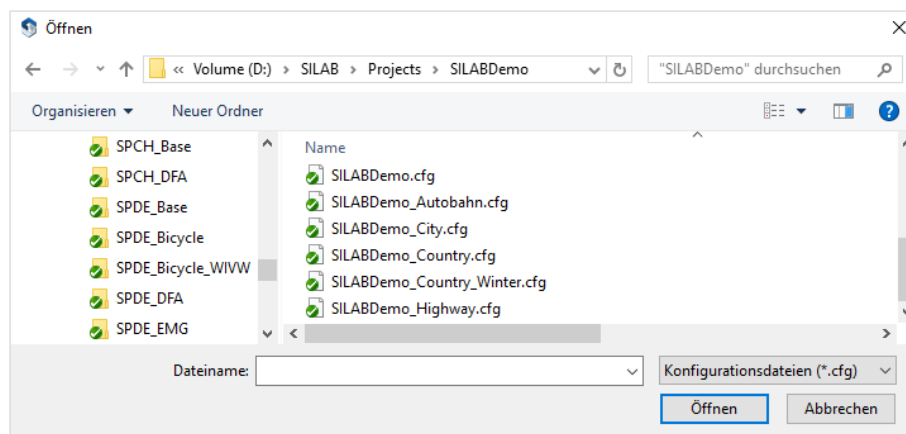


Abbildung 16. Screenshot des Speicherorts der Demo-Strecken.



Abbildung 17. Screenshot aus ‚SILABDemo‘.

Die Demo-Strecken enthalten viele aktuelle Features von SILAB und sind daher grafisch sehr aufwändig. Abhängig von der Leistungsfähigkeit der Hardware Ihres Simulators kann es deswegen vorkommen, dass die Darstellung an einigen Stellen der Strecke nicht ganz flüssig ist.

Wenn Sie die Demo-Strecken verwenden möchten, um Ihren Fahrsimulator Personen zu zeigen, die noch nicht an Fahrsimulation gewöhnt sind, empfiehlt es sich, zunächst die Landstraßen- oder Autobahnfahrt zu wählen. Die visuelle Komplexität der Stadt-Szenarien kann leicht zu Unverträglichkeiten („simulator sickness“) führen. Bitte lesen Sie zu diesem Thema auch das Dokument ‚Gesundheits- und Sicherheitshinweise‘.

Wir behalten uns vor, mit jeder neuen Version von SILAB die Demo-Strecken zu überarbeiten oder in großen Teilen anders zu gestalten.

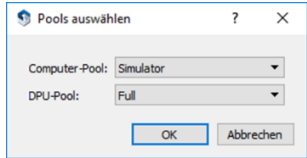
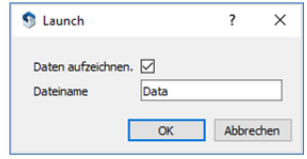
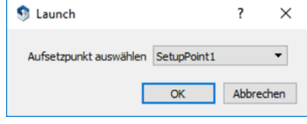
6.2.2 Anleitung zur Steuerung des Simulationsablaufs

In diesem Abschnitt erfahren Sie, wie Sie die Simulation mithilfe von SILAB steuern können. Die Steuerung der Simulation besteht im Wesentlichen aus den Schritten: Simulationsdatei öffnen, Simulationslauf vorbereiten (Launch), Simulation starten und Simulation beenden. Sie haben außerdem die Möglichkeit, die Simulation zwischendurch zu pausieren und wieder zu starten sowie die Simulation bei Problemen mit dem Notausknopf so schnell wie möglich zu beenden.

Bevor Sie die schrittweise Anleitung (Tabelle 13) zur Steuerung von SILAB anwenden, sind vorab diese Informationen nützlich für Sie:

- Für die meisten Aktionen zur Simulationssteuerung haben Sie zwei alternative Eingabeoptionen: Entweder Sie nutzen die Menüleiste oder die Symbolleiste.
- SILAB kann zwei verschiedene Dateitypen öffnen: Konfigurationsdateien mit der Dateierweiterung *.cfg und Arbeitsbereich-Dateien mit der Endung *.wsp. Im Dialogfeld zur Öffnung der Datei sollten Sie darauf achten, dass Sie eine dieser Dateitypen eingestellt haben. Der Unterschied zwischen diesen beiden Dateitypen wird in Abschnitt 6.3.3 erklärt.
- Welche Dialogfelder von SILAB beim Öffnen und Launchen einer Datei angezeigt werden, hängt davon ab, ob es sich um eine Demo-Strecke oder eine selbst erstellte Konfiguration handelt und welche Optionen im Vorfeld mit SILABSessionEdit ausgewählt wurden (z.B. Datenaufzeichnung oder unterschiedliche Aufsetzpunkte). Deshalb sind einige Schritte in der nachfolgenden Anleitung mit ‚optional‘ gekennzeichnet.
- Mit dem Dialogfeld zur Auswahl des Aufsetzpunktes können Sie entscheiden, ob der Parcours ganz am Anfang oder zu einem anderen vordefinierten Startpunkt beginnen soll. Der/die Fahrer/in wird infolge der Auswahl an unterschiedlichen Stellen im Parcours aufgesetzt.
- Das Laden einer Konfigurationsdatei kann je nach ihrer Komplexität einige Sekunden oder Minuten dauern.

Tabelle 13. Anleitung zur Steuerung des Simulationsablaufs.

Aktion	Schritt	Ort der Eingabe			Reaktion von SILAB
		Menü-leiste	Symbol-leiste	Dialogfeld	
Datei öffnen	1	Datei → Datei öff- nen			Dialogfeld öffnet sich
	2			Im Feld rechts neben 'Dateiname' den Datei- typ *.cfg auswählen	
	3			Datei auswählen und 'Öffnen' klicken	Beim Öffnen von Demo- Strecken: Dialogfeld 'Pools auswählen' öffnet sich
	4 (optional)			Computer-Pool: 'Simula- tor', DPU-Pool: 'Full', 'OK' klicken	
					Konfigurationsdatei lädt
Vorbereiten des Simulati- ons-laufs (Lau- nch)					Wenn Konfigurationsdatei geladen: Launch-Button wird aktiviert 
	1	Simula- tion → Launch			SILAB bereitet die Simula- tion vor, z.B. Initialisierung der Mockup-Hardware
					Optional öffnet sich ein Lau- nch-Dialogfenster
	2 (optional)			Dateinamen für Auf- zeich-nungsdatei einge- ben und 'OK' klicken	
	3 (optional)			Aufsetzpunkt auswählen und 'OK' klicken	
Simulation starten					Wenn Simulation vorberei- tet ist: ► wird blau ►
	1	Simula- tion → Start			
Simulation pausieren	1	Simula- tion → Pause			

Simulation stoppen	1	Simula- tion → Stop	
Simulation be- enden	1	Datei → Beenden	Kreuz-Symbol oben rechts
Notaus: Simu- lation so schnell wie möglich been- den	1	Simula- tion → Shut- down	

In seltenen Fällen kann es vorkommen, dass das Laden einer Konfigurationsdatei fehlschlägt. Dies erkennen Sie daran, dass der Launch-Knopf nicht aktiviert wird und im Meldungsfenster Fehler gemeldet werden. Klicken Sie in diesem Fall auf das Shut-Down-Symbol (●) und verfahren Sie, wie in Kapitel 0 beschrieben.

Der Befehl Shutdown sollte nicht zum regulären Beenden der Simulation verwendet werden. Dieser Befehl beendet zwar auch die Simulation, hat aber die Funktionalität eines Notaus-Knopfs. Seine Aufgabe ist, die Simulation so schnell wie möglich anzuhalten. Unter Umständen werden nicht alle nötigen Schritte zum Beenden ausgeführt, was einen Neustart der Simulationsrechner und weiterer Hardware des Simulators erforderlich machen kann.

An jeder Stelle eines Simulationslaufs können Sie das Programm SILAB einfach schließen (über das Menü mit ‚Datei → Beenden‘ oder das Kreuz-Icon rechts oben im Hauptfenster). SILAB führt dann automatisch alle Schritte zum Beenden der Simulation aus (z.B. Hardware deaktivieren, Bewegungssystem in den Ruhezustand fahren, Aufzeichnungsdateien schließen).

6.3 Überwachung von Parametern

Mit SILAB können Sie während der Simulationsfahrt diverse Parameter zum Verhalten des Fahrers beobachten und überwachen, bspw. die aktuell gefahrene Geschwindigkeit oder der Abstand zum Vorderfahrzeug. In diesem Kapitel erhalten Sie einen Überblick über die verschiedenen Parameter, deren Überwachung im Rahmen Ihrer Simulation relevant sein könnte. Des Weiteren lernen Sie, wie die ausgewählten Parameter in übersichtlicher und anschaulicher Form während der Fahrt überwacht werden können und die gewählte Ansicht für zukünftige Simulationsläufe speichern können.

Zur Auswahl der interessierenden Parameter ist ein Zugriff auf Ein- und Ausgänge der Softwarekomponenten, die an der Simulation beteiligt sind, notwendig. Um ein Verständnis über die Bedeu-

tung, Aufgaben und Organisation der Softwarekomponenten zu erhalten, geht der folgende Abschnitt zunächst auf die Softwarekomponenten und deren Zusammenhang zu den entsprechenden Parametern ein.

6.3.1 Data Processing Units (DPUs)

An der Simulation sind diverse Softwarekomponenten beteiligt, die in SILAB als DPUs (Data Processing Units) bezeichnet werden. Diese DPUs sind für verschiedene Bereiche der Simulation zuständig, d.h. sie erfüllen verschiedene Aufgaben. Es gibt beispielsweise DPUs für die Simulation des physikalischen Fahrzeugmodells, für die Bedieneingaben des Fahrers, für die Simulation des Umgebungsverkehrs und für die Visualisierung der Szenerie aus Sicht des Fahrers.

Sobald SILAB eine Konfigurationsdatei geladen hat, wird im Hauptfenster eine Baumansicht mit allen an der Simulation beteiligten DPUs angezeigt. Der Name einer DPU wird jeweils hinter dem Stecker-Symbol angezeigt (s. Abbildung 18). Die DPUs, die in Abbildung 18 angezeigt werden, sind beispielsweise für die Visualisierung zuständig. Das bedeutet, dass diese DPUs zusammen ein 3D-Bild der Szenerie aus Sicht des Fahrers berechnen. Der Ordner ‚Visualisierung‘ oberhalb der DPUs zeigt an, für welche Aufgabenkategorie die DPUs zuständig sind. Die anderen Ordner in Abbildung 18, wie ‚Datenbasis‘ und ‚Fahrdynamik‘ lassen sich mit einem Mausklick ebenfalls aufklappen, so dass die dazugehörigen DPUs aufgelistet werden.

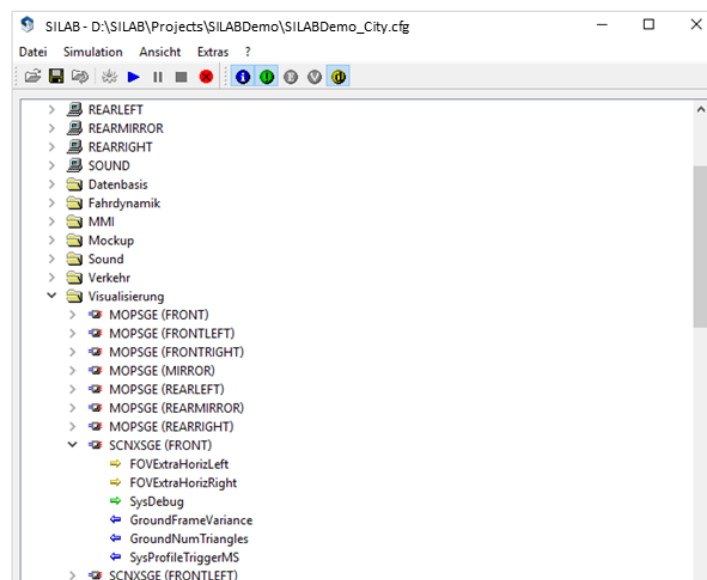


Abbildung 18. Anzeige der DPUs in SILAB.

Die verschiedenen DPUs tauschen während der Simulation untereinander Daten aus. Der Datenaustausch findet über die Ein- und Ausgänge der DPUs statt. In jedem Simulationsschritt berechnet eine DPU aus den aktuell an den Eingängen anliegenden Werten neue Ausgangswerte. Die Ausgänge einer DPU sind mit Eingängen anderer DPUs verknüpft. Im nächsten Simulationsschritt sorgt SILAB dafür, dass die Werte bei solchen Verknüpfungen im System aktualisiert werden.

Wenn Sie die Zeile einer bestimmten DPU aufblättern, können Sie die Ein- und Ausgänge dieser DPU sehen (s. Abbildung 18 unterhalb von DPU „SCNXSGE (FRONT)“). Im Folgenden erhalten Sie Erklärungen zu den Bedeutungen der Pfeile unterschiedlicher Typen und Farben:

-  Eingang, der nicht mit einem Ausgang einer anderen DPU verknüpft ist
-  Eingang, der seinen Wert von einem Ausgang einer anderen DPU bezieht
-  Ausgang einer DPU
-  Parameter einer DPU, die während des Simulationsablaufs nicht geändert werden können
-  Signal, das sowohl Eingang als auch Ausgang ist
-  ein Signal vom Typ  wird als Eingang verwendet, der mit einem Ausgang einer anderen DPU verschaltet ist

Grüne Pfeile signalisieren also dynamische Eingänge, die während der Simulation in jedem Schritt verändert werden. Rote Pfeile sind verknüpfte Eingänge, die in jedem Schritt durch den verknüpften Ausgang einer anderen DPU definiert werden. Ein gelber Pfeil signalisiert einen statischen Eingang, der bereits beim Start der Simulation definiert wird. Blaue Pfeile sind Ausgänge von DPUs, die in jedem Simulationsschritt durch die DPU neu gesetzt werden.

Durch die beschriebene modulare Struktur ist SILAB sehr transparent und ermöglicht Ihnen den Zugriff auf alle Ein- und Ausgänge aller DPUs während der Simulation. Ein typischer Anwendungsfall ist das Beobachten und Überwachen von Parametern. Die Parameter stellen Ausgänge von DPUs dar, die zu jedem Zeitpunkt bestimmte Werte annehmen.

Übersicht über DPUs und zugehörige Parameter

In den folgenden Tabellen finden Sie eine Übersicht von DPUs und den dazugehörigen Parametern, die bei den meisten Studien mit Fahrsimulatoren interessant sind:

Tabelle 14. Parameter der DPU Mockup (Bedieneingaben des Fahrers).

Parametername	Bedeutung	Kodierung	Einheit
SteeringWheel	Lenkwinkel	0 = Mittelstellung	rad
AcceleratorPedal	Gaspedalstellung	0 = kein Gas, bis 1 = Vollgas	
BrakePedal	Bremsdruck	0 = keine Bremsung, bis 10 = Vollbremsung	
IndicatorLeft	Stellung Blinker links	0 = aus, 1 = an	
IndicatorRight	Stellung Blinker rechts	0 = aus, 1 = an	
SteeringMoment	Vom Fahrer aufgebrachtes Lenkmoment		Nm
Horn	Hupe	0 = aus, 1 = an	
UpperBeam	Fernlicht/Lichthupe	0 = aus, 1 = an	

Tabelle 15. Parameter der DPU VDyn (physikalisches Modell des Ego-Fahrzeugs).

Parametername	Bedeutung	Einheit
v	Längsgeschwindigkeit	m/s
v_kmh	Längsgeschwindigkeit	km/h
ax	Längsbeschleunigung	m/s ²
ay	Querbeschleunigung	m/s ²
RPM	Motordrehzahl	1/min

Tabelle 16. Parameter der DPU SCNX (Informationen über die aktuelle Position des Fahrers im Straßennetzwerk).

Parametername	Bedeutung (und Kodierung)	Einheit
s	Streckenmeter im aktuellen Streckenabschnitt. Im Bereich eines Verkehrsknotens (Kreuzung, Autobahnauffahrt usw.) ist dieser Wert 0.	m
LateralDistance	Seitlicher orientierter Abstand des Fahrzeugschwerpunkts zur Idealspur.	m
LateralPos	Seitlicher Abstand des Fahrzeugschwerpunkts zum rechten Straßenrand.	m
NLanes	Anzahl der Spuren der Straße	
LaneIdx	Nummer der aktuellen Spur von 0 bis NLanes-1, gezählt von links nach rechts.	
Orientation	Fahrtrichtung auf der aktuellen Spur. 0 = Fahrer fährt in Richtung der Spur 1 = Fahrer fährt gegen die Richtung der Spur	
ModuleTemplateID	Typ-ID eines Szenarios. Eine Auflistung der Typ-IDs finden Sie in der Dokumentation eines Szenarien-Pakets.	

Tabelle 17. Parameter der DPU TRFDData (Zustand der Simulation anderer Verkehrsteilnehmer).

Parametername	Bedeutung	Einheit
StLongAheadNext	Längsabstand zum nächsten, vorderen Fahrzeug auf der Spur des Fahrers, von Stoßstange zu Stoßstange gemessen.	m
StLatAheadNext	Seitlicher Abstand zum nächsten, vorderen Fahrzeug auf der Spur des Fahrers, von Stoßstange zu Stoßstange gemessen.	m
vSameAheadNext	Geschwindigkeit des nächsten, vorderen Fahrzeugs auf der Spur des Fahrers.	m/s
SpLongAheadNext	Längsabstand zum nächsten, vorderen Fahrzeug auf der Spur des Fahrers, von Schwerpunkt zu Schwerpunkt gemessen.	m
SpLatAheadNext	Seitlicher Abstand zum nächsten, vorderen Fahrzeug auf der Spur des Fahrers, von Schwerpunkt zu Schwerpunkt gemessen.	m
StLongAheadRight	Abstände und Geschwindigkeit zum nächsten, vorderen Fahrzeug auf der Spur rechts neben dem Fahrer, von Stoßstange zu Stoßstange gemessen.	m
StLatAheadRight		m
vSameAheadRight		m/s
StLongAheadLeft		m
StLatAheadLeft		m

vSameAheadLeft	Abstände und Geschwindigkeit zum nächsten, vorderen Fahrzeug auf der Spur links neben dem Fahrer, von Stoßstange zu Stoßstange gemessen.	m/s
StLongBehindNext	Abstände und Geschwindigkeit zum nächsten, hinteren Fahrzeug auf der Spur des Fahrers, von Stoßstange zu Stoßstange gemessen.	m
StLatBehindNext		m
vSameBehindNext		m/s
StLongBehindRight	Abstände und Geschwindigkeit zum nächsten, hinteren Fahrzeug auf der Spur rechts neben dem Fahrer, von Stoßstange zu Stoßstange gemessen.	m
StLatBehindRight		m
vSameBehindRight		m/s
StLongBehindLeft	Abstände und Geschwindigkeit zum nächsten, hinteren Fahrzeug auf der Spur links neben dem Fahrer, von Stoßstange zu Stoßstange gemessen.	m
StLatBehindLeft		m
vSameBehindLeft		m/s

6.3.2 Auswahl und Anzeigeformen der Parameter

In diesem Abschnitt erfahren Sie, wie Sie einen oder mehrere Parameter auswählen und während der Simulation beobachten und überwachen können. Es gibt unterschiedliche Darstellungsformen für die Parameter – als Textfeld, Slider oder Oszilloskop. Sie können je nach persönlichen Präferenzen, dem gewählten Parametern und dem jeweiligen Anwendungsfall entscheiden, welche Darstellungsform für Sie am übersichtlichsten und praktikabelsten ist.

Zur Auswahl eines Parameters klicken Sie mit der rechten Maustaste auf den Parameter im Bereich der jeweiligen DPU (s. Abbildung 18). Es erscheint das Kontextmenü in Abbildung 19. Aus diesem Menü können Sie wählen, auf welche Weise der Parameter angezeigt werden soll. Im Folgenden werden die verschiedenen Möglichkeiten beschrieben.

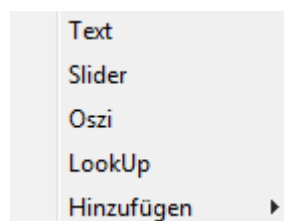


Abbildung 19. Kontextmenü zur Auswahl eines Parameters.

Wenn Sie die Anzeigeeoption Text wählen, erhalten Sie eine Anzeige wie in der linken Spalte von Abbildung 20 dargestellt. In der Titelleiste des Fensters wird der Name des Parameters (hier: die Längsbeschleunigung a_x aus dem physikalischen Modell des Fahrzeugs VDyn) angezeigt. Im Feld darunter finden Sie den aktuellen Wert. Sie können das Fenster beliebig auf dem Windows-Desktop verschieben und seine Größe anpassen. Wenn Sie mit der rechten Maustaste auf das Fenster klicken, erscheint ein Kontextmenü, mit dem Sie

- den Titel des Fensters festlegen können.

- die Farben aller Komponenten (Rahmen, Hintergrund, Schrift usw.) anpassen können.
- einen Wertebereich definieren können, in dem der Fensterrahmen seine Farbe wechselt (Hervorhebung). Der Rahmen könnte bei der Anzeige der aktuellen Geschwindigkeit z.B. seine Farbe von grün auf rot wechseln, wenn der/die Fahrer/in eine maximal gewünschte Geschwindigkeit überschreitet.



Abbildung 20. Parameter-Anzeigeoptionen in SILAB am Beispiel des Parameters Längsbeschleunigung (ax).

Die Darstellung eines Parameters mit einem Slider ist in der Mitte von Abbildung 20 zu sehen. Auch bei diesem Element können Sie Größe und Position der Anzeige verändern. Im Kontextmenü (rechte Maustaste auf Anzeige) finden Sie die gleichen Einträge wie bei der Text-Anzeige. Mit dem zusätzlichen Eintrag ‚Min/Max‘ können Sie den Wertebereich des Sliders einstellen (in obigem Beispiel von -10 m/s^2 bis 10 m/s^2 für die Längsbeschleunigung).

In der rechten Spalte von Abbildung 20 ist die Anzeige eines Parameters als Oszilloskop dargestellt. In dieser Anzeige wird der ausgewählte Parameter als Wert entlang der Zeitachse (x-Achse) geplottet. Der aktuelle Wert eines Parameters wird als Zahlenwert in dem Feld oberhalb des Oszilloskops angezeigt. Sie können den Wertebereich der y-Achse mit den Pfeil-Buttons links neben der y-Achse anpassen. Der senkrechte schwarze Strahl innerhalb der Grafik repräsentiert den aktuellen Zeitpunkt.

Durch einen Rechtsklick auf das Oszilloskop wird ein Kontextmenü angezeigt, in dem Sie wie bei den anderen Anzeigen Titel und Farben festlegen können. Folgende Menüpunkte sind bei Oszilloskopen zusätzlich vorhanden:

ABTASTFREQUENZ:

- Legt fest, wie schnell der Parameter im Oszilloskop dargestellt werden soll.

SAMPLE-MODUS:

- Legt fest, wie Parameter zusammengefasst werden sollen, wenn Sie eine langsamere Abtastfrequenz als die Aktualisierungsfrequenz des Parameters gewählt haben.

SYNC:

- Synchronisiert die Anzeige aller Oszilloskope.

T-BESCHRIFTUNG:

- Schaltet die Beschriftung der Zeitachse ein bzw. aus (in Millisekunden).

HILFSLINIEN:

- Schaltet horizontale Hilfslinien ein bzw. aus.

Y-ACHSEN VEREINHEITLICHEN:

- Vereinheitlicht die Wertebereiche der y-Achsen für alle dargestellten Parameter.

ERWEITERT:

- Zeigt einen Dialog an, mit dem Sie die Farbe der Darstellung eines Parameters wählen können. Durch die Auswahl des Menüpunktes ‚Automatische Skalierung‘ in diesem Dialog wird der Wertebereich der y-Achse automatisch angepasst. Alternativ kann der Wertebereich der y-Achse manuell eingegeben werden.

Sie haben darüber hinaus die Möglichkeit, einen weiteren Parameter zum Oszilloskop hinzuzufügen. Wählen Sie dafür im Kontextmenü des Parameters ‚Hinzufügen‘ (Abbildung 19). Es erscheint eine Liste mit allen bereits vorhandenen Oszilloskopen. Wählen Sie in dieser Liste das Oszilloskop, in dem der Parameter zusätzlich angezeigt werden soll. Abbildung 21 zeigt ein Oszilloskop mit zwei Parametern.

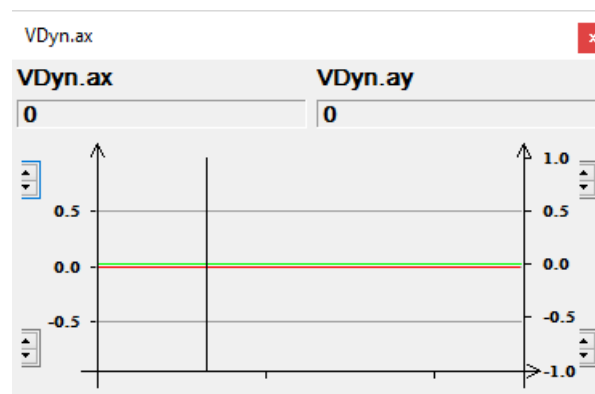


Abbildung 21. Oszilloskop mit zwei Parametern.

- Für die Darstellung der Parameter gibt es zwei y-Achsen (rechts und links). Welcher Parameter auf welcher y-Achse dargestellt wird, können Sie im Dialogfenster festlegen, das im Kontextmenü unter dem Punkt ‚Erweitert‘ erscheint. Wählen Sie im Dialog den Kanal aus, der dem Parameter entspricht. Legen Sie dann für diesen Kanal die y-Achse und die Farbe der Darstellung fest. Auch die Festlegung der automatischen Skalierung bezieht sich immer auf den aktuell gewählten Kanal.

Weiterführende Informationen zu den angesprochenen Möglichkeiten Werte zu beobachten, finden Sie in Kapitel 5.4 im Dokument [REFERENZ SILAB-KONFIGURATION UND BEDIENUNG](#).

6.3.3 Abspeichern der ausgewählten Parameter als Arbeitsbereich

In der Anleitung zum Öffnen von Dateien in SILAB (s. Abschnitt 6.2.2) haben Sie bereits erfahren, dass SILAB sowohl Konfigurationsdateien mit der Dateiendung *.cfg, als auch Arbeitsbereich-Dateien mit der Endung *.wsp (,wsp' steht für die englische Bezeichnung ,Workspace') öffnen kann. Ein Parcours, den Sie mit SILABSessionEdit erstellt und für die Simulation exportiert haben, hat zunächst den Typ einer Konfigurationsdatei. Eine Konfiguration enthält noch keine Informationen über das Layout des SILAB-Fensters und die ausgewählten Parameteranzeigen. Wenn Sie eine Konfigurationsdatei in SILAB geöffnet, das Hauptfenster von SILAB nach Ihren Wünschen platziert und verschiedene Anzeigen von Parametern eingerichtet haben, können Sie diese Einstellungen als sogenannten Arbeitsbereich abspeichern. Wählen Sie hierzu in der Menüleiste ,Datei → Speichern'. Wenn Sie beim nächsten Mal nicht die Konfigurations- (*.cfg), sondern die neu abgespeicherte Arbeitsbereich-Datei (*.wsp) laden, öffnet SILAB die Konfiguration und positioniert das Hauptfenster sowie alle Parameteranzeigen, wie zuletzt von Ihnen angeordnet. Sie können für eine Konfigurationsdatei auch mehrere Arbeitsbereiche anlegen. Wählen Sie hierzu in der Menüleiste ,Datei → Speichern unter' und geben Sie einen Namen für den alternativen Arbeitsbereich an. Um zu verhindern, dass speziell zur Versuchsdurchführung erstellte Workspaces überschrieben werden, können geöffnete Workspaces über ,Extras → Workspace sperren' schreibgeschützt werden.

6.4 Veränderung von Parameterwerten während der Simulation

SILAB bietet die Möglichkeit, die Werte verschiedener Parameter während der Simulation beliebig zu verändern. Diese Option besteht für alle DPU-Eingänge, die nicht mit dem Ausgang einer anderen DPU verknüpft sind. Dies lässt sich in der Baumansicht unter der entsprechenden DPU an einem grünen Pfeil erkennen (s. Abschnitt 6.3.1). Das manuelle Ändern von Parameterwerten während der Simulation wird üblicherweise vor allem zum Ausprobieren bei der Versuchsvorbereitung genutzt. Während der Versuchsdurchführung werden solche Einstellungen gewöhnlich vordefiniert, um das Fehlerpotential bei der Versuchsdurchführung zu minimieren.

Bitte prüfen Sie zunächst anhand der zugehörigen DPU-Anleitung, ob der Eingang während der Simulation verändert werden darf. Bitte beachten Sie auch, dass Parameter von DPUs, die in der jeweiligen Dokumentation als „statisch“ gekennzeichnet sind, nur einmal beim Start der Simulation eingelesen werden.

Zum Ändern eines Parameterwerts klicken Sie zunächst mit der rechten Maustaste auf den Parameter im Bereich der jeweiligen DPU (s. Abbildung 18). Es öffnet sich ein Kontextmenü, in dem Sie zwischen den Displays ,Schalter', ,ValueEdit' und ,Slider' wählen können. Screenshots der drei Display-Varianten sind in Abbildung 22 dargestellt.

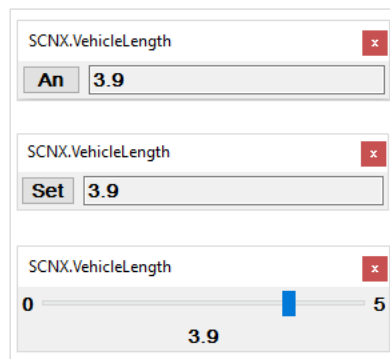


Abbildung 22. Display-Varianten zur Änderung von Parameterwerten.

In allen Displays wird der aktuelle Wert der Variablen als Text angezeigt. Außerdem kann er folgendermaßen verändert werden:

- Bei Schalter-Displays kann durch Klick auf den An-/Aus-Button der Wert auf 1 bzw. 0 gesetzt werden. Diese eignen sich am besten für binäre Eingänge wie z.B. die Aktivierung und Deaktivierung von Fahrerassistenzsystemen.
- Bei ValueEdit-Displays kann in das Textfeld ein neuer Parameterwert eingegeben werden und dann über einen Klick auf ‚Set‘ umgesetzt werden.
- Bei Slider-Displays kann der Parameterwert durch Bewegung des Sliders geändert werden.

Bei allen Displays können noch weitere Einstellungen über das Kontextmenü vorgenommen werden. Weitere Informationen zu den angesprochenen Möglichkeiten, Parameter zu verändern, finden Sie in Kapitel 5.4 im Dokument [REFERENZ SILAB-KONFIGURATION UND BEDIENUNG](#).

6.5 Überwachung der Simulationsrechner während der Simulation

Es wird empfohlen, während der Simulation regelmäßig sicherzustellen, dass keine Verzögerungen an den Simulatorrechnern auftreten, da dadurch die Simulation beeinträchtigt werden kann. Insbesondere wenn Änderungen an der DPU-Konfiguration vorgenommen werden oder eine neue, grafisch aufwändige Strecke entworfen wurde, sollten die Simulationsrechner überwacht werden.

Verzögerungen können mithilfe der Sanduhr-Symbole in der Baumansicht im SILAB-Hauptfenster untersucht werden (s. Abbildung 23). Die angegebenen Zahlen in Millisekunden sollten im Optimalfall 0 betragen. Konstante einstellige Zahlen sind noch akzeptabel, ebenso kurze Perioden mit Werten größer als 0, die jedoch nicht regelmäßig auftreten. Das oberste Sanduhr-Symbol bezieht sich auf den maximalen Zeitverzug aller an der Simulation beteiligten Rechner. Der Rechner, auf dem der maximale Zeitverzug vorliegt, wird stets in Klammern erwähnt (hier: ‚SOUND‘). Um die Verzögerungen der einzelnen Rechner zu sehen, klicken Sie den Rechnernamen an. Es wird ein Bereich ausgeklappt, in dem der Status des Rechners und seine Verzögerung (neben dem Sanduhr-Symbol) angezeigt wird.



Abbildung 23. Baumansicht des SILAB-Hauptfensters mit Informationen über Simulationsrechner.

Noch genauer kann der Zeitverzug mit der Simulations-Status-Anzeige (s. Abbildung 24) untersucht werden, die im Menü über die Optionen ‚Ansicht → Simulations-Status‘ geöffnet werden kann. Die Prozentangaben zeigen an, zu wie viel Prozent die verschiedenen Rechner jeweils ausgelastet sind. Ist ein Rechner zu mehr als 100% ausgelastet, resultiert daraus ein zeitlicher Verzug.

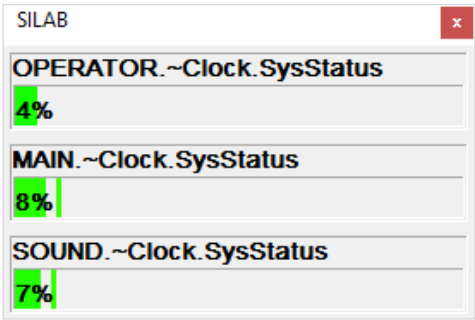


Abbildung 24. Simulations-Status-Anzeige.

6.6 Auto-Start der Simulation

Wenn Sie eine Versuchskonfiguration häufig mit demselben Arbeitsbereich (*.wsp) starten möchten, kann eine Verknüpfung auf dem Desktop zu diesem Arbeitsbereich nützlich sein (Erklärung Arbeitsbereich s. Abschnitt 6.3.3). Wenn Sie diese Verknüpfung anklicken, öffnet sich SILAB mit dem entsprechenden Arbeitsbereich automatisch. Folgen Sie bitte der Anleitung in Tabelle 18 zur Erstellung der Auto-Start-Verknüpfung auf dem Desktop.

Tabelle 18. Anleitung zur Erstellung der Auto-Start-Verknüpfung auf dem Desktop.

Schritt	Aktion	Erklärung
1	Neue SILAB-Verknüpfung auf Desktop	Erstellen Sie eine neue Verknüpfung des Programms SILAB auf dem Desktop (Speicherort von SILAB ist im SILAB-Unterverzeichnis ‚bin‘).
2	Öffnen der Eigenschaften	Klicken Sie mit der rechten Maustaste auf die Verknüpfung und wählen Sie ‚Eigenschaften‘ aus.

3 Bearbeiten der Verknüpfung
(Abbildung 25)

Wählen Sie nun den Reiter ‚Verknüpfung‘ aus. Fügen Sie im Feld ‚Ziel‘ den vollständigen Pfad- und Dateinamen des Arbeitsbereichs hinter die Verknüpfung zu SILAB (D:\SILAB\bin\SILAB.exe) ein. Die Eingabe könnte zusammen z.B. folgendermaßen aussehen:

D:\SILAB\bin\SILAB.exe

D:\SILAB\Projects\Musterprojekt\Musterstrecke.wsp

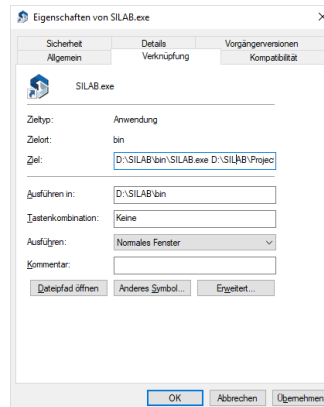


Abbildung 25. Bearbeiten der Verknüpfung zu SILAB und dem gewünschten Arbeitsbereich.

6.7 Fernsteuerung der Simulation von anderen Rechnern

Sie haben die Möglichkeit, das Programm SILAB über das Netzwerk von einem anderen Rechner fernzusteuern. Ein typisches Anwendungsszenario für diese Fernsteuerung könnte sein, dass Sie einen Laptop mit in den Simulator nehmen möchten, von dem aus Sie während der Fahrt Parameter beobachten, verändern oder unterschiedliche Konfigurationen testen möchten. Diese Fernsteuerung erfolgt mit dem Programm SILABExplorer. Die Funktionen und die Benutzeroberfläche des Programms werden im folgenden Abschnitt erklärt. Abschnitt 0 beinhaltet eine Anleitung zur Anwendung von SILABExplorer zur Fernsteuerung von SILAB.

6.7.1 Funktionen und Benutzeroberfläche von SILABExplorer

Das Programm SILABExplorer finden Sie auf dem Operatorrechner in dem SILAB-Unterverzeichnis ‚bin‘.

Das Programm bietet folgende Funktionen, die Sie von einem anderen Rechner, der mit dem Rechnernetz der Simulation verbunden ist, ausführen können:

- Herstellen einer Verbindung zum Operatorrechner einer SILAB-Simulation über das Netzwerk (TCP/IP)
- Laden einer Konfigurations- oder Arbeitsbereichsdatei auf dem Operatorrechner
- Steuerung der Simulation (Start/Stop/Pause/Resume)
- Beobachtung und Modifikation von Parametern (DPU-Variablen): Dabei kann die Frequenz, mit der ein Parameter beobachtet wird, gewählt werden

- Beenden von SILAB
- Es ist außerdem möglich, dass sich mehrere Rechner mit SILABExplorer gleichzeitig oder nacheinander mit dem Operatorrechner verbinden.

Die Benutzeroberfläche von SILABExplorer ähnelt der von SILAB und ist auf Abbildung 26 zu sehen. Die einzelnen Elemente der Benutzeroberfläche werden in Tabelle 19 erklärt.

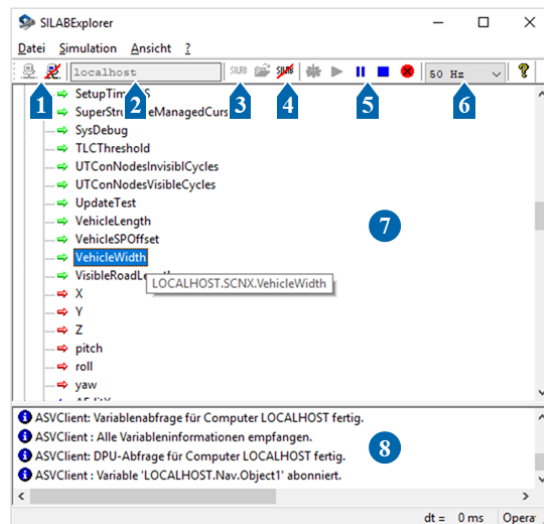


Abbildung 26. Benutzeroberfläche von SILABExplorer.

Tabelle 19. Erläuterungen zur Benutzeroberfläche von SILABExplorer.

Bereich in Abb. 22	Erklärung
1	Schaltflächen zum Herstellen (links) und Trennen (rechts) einer Verbindung zum Operatorrechner
2	IP-Adresse oder Rechnername des Operatorrechners, zu dem Verbindung aufgenommen werden soll
3	Schaltfläche zum Starten von SILAB. Bei bestehender Verbindung zu SILAB: Öffnet eine Konfigurationsdatei oder einen Workspace
4	Bei bestehender Verbindung zu SILAB: Beendet SILAB
5	Steuerung des Simulationsablaufs: Launch / Start / Pause / Stop / Shutdown der Simulation (s. Abschnitt 6.1)
6	Frequenz, mit der Parameter in SILABExplorer aktualisiert werden (z.B. in einem Oszilloskop)
7	Sobald die Verbindung zu SILAB hergestellt ist und ein Arbeitsbereich in SILAB geöffnet ist, erscheint hier eine Baumansicht mit allen Rechnern, DPUs und Parametern der Simulation (wie in SILAB)
8	In diesem Meldungsfenster werden Informationen, Warnungen und Fehlermeldungen angezeigt

6.7.2 Einrichten einer Fernsteuerung mit SILABExplorer

In diesem Abschnitt erfahren Sie, wie Sie die Fernsteuerung von SILAB mithilfe von SILABExplorer einrichten, steuern und beenden können. Dafür stellen Sie eine Verbindung von einem Laptop zum Operatorrechner her.

Als erster Schritt muss SILAB auf dem entsprechenden Laptop installiert werden. Befolgen Sie dafür die Installationsanleitung in dem Dokument **INSTALLATIONSANLEITUNG SILAB 7.1** (Kapitel zur Erstinstallation auf einem Einzelplatzsystem). Im Anschluss sollte der Laptop mit dem Rechnernetz der Simulation verbunden werden.

Die Voraussetzung für die Anleitung zur Fernsteuerung mit SILABExplorer (Tabelle 20) ist, dass SILAB auf dem Operatorrechner bereits gestartet ist.

Tabelle 20. Anleitung zum Einrichten einer Fernsteuerung mit SILABExplorer.

Schritt	Aktion	Ort der Eingabe (Abb. 22)	Erklärung	Reaktion von SILABExplorer
1	IP-Adresse eintragen	2	IP-Adresse des Operatorrechners eintragen	
2	Verbindung mit Operatorrechner herstellen	1	Auswahl der linken Schaltfläche	SILABExplorer stellt Verbindung mit Operatorrechner und SILAB her
3	Auswahl einer Datei (cfg/wsp)	3	Auswahl von Schaltfläche 3	SILABExplorer öffnet ein Dialogfenster (Abbildung 27)
4	Öffnen einer Datei (cfg/wsp)	Dialogfeld	Eingabe des Pfads zu der gewünschten Datei → Auswahl von ‚OK‘ Bei Auswahl einer *.cfg-Datei: Angabe von Computer-Pool und DPU-Pool	SILABExplorer öffnet die Datei auf dem Operatorrechner, holt sich alle Rechner-, DPU- und Parameter-Informationen und stellt sie in der Baumansicht dar (Bereich 7 in Abbildung 26)
5	Steuerung der Simulation	5	Steuerung der Simulation über die von SILAB bekannten Schaltflächen (Abschnitt 6.1)	
6	Beenden der Simulation	4 oder 5	Auswahl von Schaltfläche 4 der Stop-Schaltfläche in Bereich 5	
7	Trennen der Verbindung zum Operatorrechner	1	Auswahl der rechten Schaltfläche	

Wenn auf dem Operatorrechner bereits eine Konfigurations- oder Arbeitsbereich-Datei in SILAB geladen ist oder die Simulation aktuell bereits läuft, können Sie Schritt 3 und 4 überspringen. Nachdem Sie in Schritt 2 die Verbindung zum Operatorrechner hergestellt haben, füllt sich die Bauman-sicht von SILABExplorer direkt mit allen Rechner-, DPU- und Parameter-Informationen.

Sie erhalten mit SILABExplorer dieselben Steuerungsmöglichkeiten, wie bei lokaler Bedienung der Simulation über SILAB. Sie können nun auch hier Parameter mit Displays (Text, ValueEdit, Oszis und Schalter) beobachten und verändern.

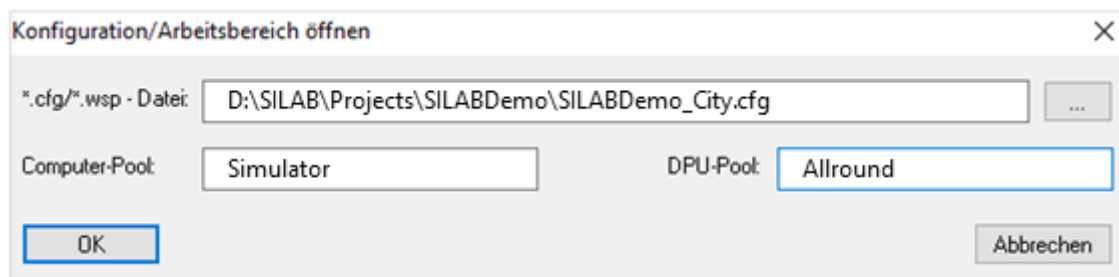


Abbildung 27. Dialogfenster von SILABExplorer zum Öffnen einer Konfigurations- oder Arbeitsbereichdatei.

7 DURCHFÜHRUNG VON UNTERSUCHUNGEN MIT DEM FAHRSIMULATOR

Die folgenden ‚Best Practices‘ sind durch die langjährige Erfahrung bei der Planung und Durchführung von Untersuchungsreihen an der WIVW GmbH entstanden. Wir empfehlen Ihnen, diese auch bei Untersuchungen an Ihrem Fahrsimulator anzuwenden. Planen Sie eine Voruntersuchung (Pre-Test) ein, bei der Sie den Untersuchungsablauf einschließlich der Simulatorfahrten mit einigen wenigen Probanden unter möglichst realistischen Versuchsbedingungen testen. Prüfen Sie insbesondere den reibungslosen Ablauf der Versuchsstrecke (Session). Analysieren Sie bereits bei der Voruntersuchung die Aufzeichnungsdateien mit den Programmen und Verfahren, die später auch bei der Untersuchung zum Einsatz kommen.

Leeren Sie regelmäßig die Ordner, in denen SILAB Aufzeichnungsdateien, digitale Videos etc. abspeichert. Diese Dateien können viel Speicherplatz einnehmen. Wenn während einer Simulatorfahrt die Festplatte voll wird, kommt es zu Datenverlust.

Installieren Sie keine Updates von SILAB während einer laufenden Untersuchungsreihe oder kurz davor. Erstellen Sie gemäß der beiliegenden Installationsanleitung SILAB 7.1 eine Sicherungskopie der vorhandenen SILAB Installation, bevor Sie die neue Version aufspielen.

Fahren Sie selbst regelmäßig mit dem Fahrsimulator. Achten Sie dabei auf evtl. vorhandene Defekte, Verschleiß oder geänderte Einstellungen.

8 HILFE BEI PROBLEMEN

Bei Problemen stehen wir Ihnen im Rahmen des Supports gerne zur Verfügung. Bitte wenden Sie sich an:

- Mail: silab@wivw.de
- Tel.: 0931-7809-400

Um die Ursache für ein aufgetretenes Problem möglichst genau identifizieren zu können, sind folgende Punkte hilfreich:

- Versuchen Sie, den Fehler möglichst reproduzierbar nachzustellen. Bitte schildern Sie Ihrem Ansprechpartner für den Support die Abläufe, die zum Fehler führen, möglichst genau.
- Im Unterordner ‚bin‘ des Installationsverzeichnis von SILAB (z.B. C:\SILAB\bin oder D:\SILAB\bin) finden Sie folgende Dateien:
 - SILAB.crash.log
 - SILAB.debug.log
 - SILAB.error.log
 - SILAB.mem.log
 - SILAB.sys.log
 - SILABSessionEdit.debug.log
 - SILABSessionEdit.error.log
 - SILABSessionEdit.mem.log
 - SILABSessionEdit.sys.log

Bitte schicken Sie diese Dateien Ihrem Ansprechpartner für den Support auf dessen Anfrage. Bitte beachten Sie, dass diese Dateien bei jedem Programmstart von SILAB überschrieben werden. Stellen Sie also sicher, dass Sie die Dateien verschicken, die bei dem Simulationslauf entstanden sind, bei dem der Fehler aufgetreten ist.

- Im Installationsverzeichnis von SILAB (z.B. C:\SILAB\bin oder D:\SILAB\bin) finden Sie eine Datei mit dem Namen default.cfg. Bitte schicken Sie diese Datei Ihrem Ansprechpartner für den Support auf dessen Anfrage. Bitte beachten Sie, dass diese Datei bei jedem Laden einer Session oder eines Arbeitsbereichs überschrieben wird. Stellen Sie also sicher, dass Sie die Dateien verschicken, die bei dem Simulationslauf entstanden ist, bei dem der Fehler aufgetreten ist.

Erweitertes Wissen für Fortgeschrittene

9 ARBEITEN MIT EIGENEN KONFIGURATIONEN

Bei Bedarf können Sie SILAB auf Ihre speziellen Fragestellungen anpassen, indem Sie Ihre eigenen Konfigurationsdateien erstellen. Konfigurieren bedeutet „gestalten“ oder „ausgestalten“. Eine Konfigurationsdatei stellt eine Art Bauplan für die individuelle Gestaltung der Simulation dar, welcher Informationen über verschiedene Komponenten enthält. Diese Informationen beinhalten, welche Rechner an der Simulation beteiligt sind, welche Softwarekomponenten auf welchen Rechnern laufen, wie die Komponenten Daten austauschen, in welchem Streckennetz sich die Simulation abspielt und wie sich die anderen Verkehrsteilnehmer verhalten sollen. Diese verschiedenen Komponenten werden letztlich in einer Versuchs-Konfigurationsdatei zusammengefasst. Konfigurationsdateien sind reine Textdateien, die mit Texteditoren erstellt und bearbeitet werden.

In SILAB werden Softwarekomponenten, die an der Simulation beteiligt sind und während der Simulation verschiedene Aufgaben ausführen, DPUs (Data Processing Units) genannt.

Die folgende Erläuterung dient dem vertieften Verständnis über die Informationen, die in Konfigurationsdateien enthalten sind: Wenn Sie mit dem Programm SILAB eine Simulation auf dem Operatorrechner starten, bezieht SILAB aus der Konfigurationsdatei die Informationen bezüglich der Rechner, die an der Simulation beteiligt sind. Danach nimmt SILAB Kontakt zu diesen Rechnern auf und schickt auch ihnen den Bauplan, der in der Konfigurationsdatei enthalten ist. Jeder Rechner entnimmt daraufhin der Konfigurationsdatei, welche DPUs er berechnen soll. Die Rechner laden die für sie bestimmten DPUs und initialisieren sie. Danach entnehmen die Rechner der Konfigurationsdatei, mit welchen anderen Rechnern sie Daten austauschen müssen und bauen Netzwerkverbindungen zu diesen Rechnern auf. Erst jetzt kann die Simulation gestartet werden.

In den bisherigen Kapiteln haben Sie unter anderem gelernt, wie Sie mit SILABSessionEdit Sessions aus Szenariendatenbanken erstellen und diese in SILAB zur Simulation einsetzen können. Für solche Arbeiten verwendet SILAB die Standardkonfiguration. Diese befindet sich im Lieferumfang von SILAB und beinhaltet ein vollständiges Grundgerüst für die Konfiguration Ihres Simulators. Bei der Installation von SILAB auf Ihrem Simulator wird dieses Grundgerüst von uns an die Besonderheiten Ihres Simulators angepasst (z.B. Anzahl der Sichtkanäle, Anbindung von Bedienelementen, Soundsystem usw.). So wird Ihnen das Arbeiten mit SILAB erleichtert und Sie benötigen für das Erstellen

von Sessions mit `SILABSessionEdit` vorerst kein Wissen über Konfigurationsdateien und wie diese Dateien bearbeitet werden können. Wenn Sie nun speziellere Anforderungen an die Simulation haben, für die das Arbeiten mit der Standardkonfiguration nicht mehr ausreicht, bietet SILAB einfache Mechanismen, um diese Standardkonfiguration zu erweitern und zu modifizieren, um somit die Simulation auf Ihre speziellen Bedürfnisse anzupassen. Das bedeutet, dass Sie die Konfigurationsdatei je nach Fragestellung nicht komplett neu entwerfen müssen, sondern ausgewählte Teile der Konfiguration anpassen können. Natürlich können Sie eine Konfigurationsdatei bei Bedarf auch von Grund auf neu schreiben.

Der Inhalt dieses Kapitels soll Ihnen ermöglichen, bei der Arbeit mit SILAB mehr Freiheiten zu erhalten und Simulationen auf Ihre individuellen Anforderungen anzupassen. Das Kapitel wird Ihnen ein Grundverständnis zum Sinn und Zweck von Konfigurationsdateien in SILAB geben, den Aufbau und die Bestandteile einer Konfigurationsdatei erklären, sowie Grundlagenwissen zum Bearbeiten der Konfigurationsdatei vermitteln. Zur detaillierten Bearbeitung und Erstellung einer Konfigurationsdatei ist es darüber hinaus notwendig, weitere SILAB-Dokumente zu lesen, in denen noch detailliertere Informationen enthalten sind, z.B. über einzelne DPUs und ihre Ein- und Ausgänge. Auf diese Dokumente wird an den entsprechenden Stellen verwiesen.

9.1 Vorteile von Konfigurationsdateien

Das Arbeiten mit Konfigurationsdateien hat folgende Vorteile:

- SILAB ist „schlank“: Da SILAB nur solche DPUs lädt, die in der Konfigurationsdatei angegeben sind, können Sie sicher sein, dass Ihre Simulation nicht von DPUs beeinflusst wird, die für Ihre Simulation irrelevant sind.
- SILAB kann sehr leicht erweitert werden: Eine DPU, die gemäß einer Konfigurationsdatei von SILAB nachgeladen wird, besteht aus einer (oder mehreren) Dateien. Wenn Sie also eine neue DPU, die Sie entweder von uns bekommen oder selber programmiert haben, installieren möchten, dann müssen Sie einfach diese Dateien in ein bestimmtes Verzeichnis von SILAB kopieren. Wenn laut Konfigurationsdatei eine DPU geladen werden soll, dann sucht SILAB automatisch in diesem Verzeichnis nach den entsprechenden Dateien. Es handelt sich dabei um eine Art Plugin-Konzept, das Sie vielleicht von anderen Programmen kennen.
- SILAB ist sehr flexibel: Je nach Konfigurationsdatei kann SILAB völlig unterschiedliche Aufgaben ausführen: Es kann Simulatoren in verschiedenen Ausbaustufen betreiben (z.B. mit unterschiedlich vielen Sichtkanälen, mit oder ohne Bewegungssystem, von einem Joystick oder einem richtigen Fahrzeug gesteuert), es kann zur Messwertaufzeichnung und -verarbeitung in einem realen Versuchsfahrzeug (z.B. auf einem Laptop oder Car-PC) eingesetzt werden oder es kann Signale erzeugen und diese z.B. auf einem CAN-Bus ausgeben. Für alle diese Einsatzbereiche müssen Sie lediglich eine entsprechende Konfigurationsdatei erstellen.

9.2 Bestandteile einer Konfigurationsdatei

- Im Folgenden erhalten Sie eine Übersicht über die Bestandteile einer Konfiguration, auf die Sie mithilfe der Konfigurationsdatei Einfluss nehmen können:
- Systemkonfiguration: Konfiguration grundlegender Einstellungen des SILAB-Systems
- Rechnerkonfiguration: Definition der Rechner, die an der Simulation beteiligt sind
- DPU-Konfiguration: Definition der Softwaremodule, die an der Simulation beteiligt sind
- Allgemeine Angaben zur Verkehrssimulation
- Allgemeine Angaben zu weiteren Komponenten der Simulation
- Szenariodefinition: Definition von Strecke, Umgebung, Verhalten von Verkehrsteilnehmern und Fußgängern für verschiedene Szenarien
- In der Regel müssen für unterschiedliche Simulationaufgaben jedoch nur einzelne Teile der Konfiguration angepasst werden. Daher sollte unbedingt darauf geachtet werden, dass die festen Teile der Konfiguration leicht wieder verwendbar sind. Diese festen Teile stellen insbesondere die Rechnerkonfiguration, der allgemeine Teil der DPU-Konfiguration und die allgemeinen Angaben zu weiteren Komponenten der Simulation dar. Sie müssen normalerweise für jeden Fahrsimulator nur einmalig erstellt werden und können dann wiederverwendet werden. Bei der Rechnerkonfiguration ändern sich die Anzahl der Rechner eines Simulators, deren Name und IP-Adresse im Rechnernetz normalerweise sehr selten. In der Regel erledigen wir diese grundlegende Anpassung an Ihre Simulator-Hardware bei der Installation von SILAB für Sie. Bei dem allgemeinen Teil der DPU-Konfiguration ist ein Großteil bei jeder Simulation gleich. Dies betrifft insbesondere die DPUs, die für den grundlegenden Betrieb der Fahrsimulation und deren Visualisierung benötigt werden. Diese DPUs werden universelle DPUs genannt. Auch die allgemeinen Angaben zu weiteren Komponenten der Simulation, die Einstellungen zur Sound- und Fußgängersimulation sowie zur grafischen Darstellung der Umgebung enthalten, bleiben in der Basis für alle Konfigurationen konstant und können bei Bedarf modifiziert werden.
- Da einige Konfigurationen wiederverwendet werden und dem häufig großen Umfang kompletter Konfigurationsdateien, wird diese meist in mehrere Dateien aufgeteilt. Diese werden durch einen sogenannten ‚include‘-Mechanismus automatisch zu einer kompletten SILAB-Konfigurationsdatei zusammengefügt. Das bedeutet, dass der Teil der Konfiguration, der für mehrere Simulationen verwendet werden kann, in eine include-Datei ausgelagert wird. Eine solche Datei wird als Teil Ihrer Konfigurationsdatei mittels include-Anweisungen an entsprechender Stelle in die Hauptkonfigurationsdatei eingebunden.

Es empfiehlt sich, include-Dateien die Endung *.inc zu geben. Damit ist erkennbar, dass es sich nur um einen Teil eines Skripts handelt, der für sich allein nicht in SILAB geladen werden kann.

Die Szenario-Definition inklusive der Verkehrssimulation sind oftmals sehr spezifisch und müssen daher für jede Fragestellung, die mit der Simulation untersucht werden soll, angepasst werden. Wenn Sie SILAB nicht für die Fahrsimulation verwenden, sondern z.B. um Messwerte in einem Realfahrzeug aufzuzeichnen, dann entfallen dementsprechend die simulationsspezifischen Abschnitte der Konfiguration.

Die System-, Rechner- und DPU-Konfigurationen können mit der SILAB-Edition ESSENTIAL bearbeitet werden. Die Bearbeitung der Angaben zur Verkehrssimulation, zur Datenbasis-/Szenariodefinition und zu den weiteren Komponenten der Simulation ist ab der SILAB-Edition PROFESSIONAL möglich. Daher wird in diesem Handbuch nicht über die Standardkonfiguration (s. Abschnitt 9.5) hinaus auf die diese Bestandteile eingegangen.

9.3 Struktur einer Konfigurationsdatei

Die Konfiguration von SILAB erfolgt mittels einer Textdatei. Jede Konfigurationsdatei hat folgende grundlegende Struktur:

```
SILAB System
{
    # Systemkonfiguration
};

SILAB Configuration
{
    Computerconfiguration Computer
    {
        # Rechner-Konfiguration
    };

    DPUConfiguration DPUs
    {
        # DPU-Konfiguration
    };
};

SILAB TRF
{
    # Allgemeine Angaben zur Verkehrssimulation
};
# Allgemeine Angaben zu weiteren Komponenten der Simulation

SILAB SCN
{
    # Szenario-Definition
};
```

Code 1. Struktur einer Konfigurationsdatei.

Im Folgenden wird der Aufbau der Konfigurationsdatei erläutert:

- Zwischen den geschwungenen Klammern werden die Informationen für die verschiedenen Konfigurationsbereiche (Systemkonfiguration, Rechnerkonfiguration, DPU-Konfiguration, allgemeine Angaben zur Verkehrssimulation, Szenariodefinitionen) angegeben.
- `Computer` und `DPUs` sind von Ihnen frei wählbare Namen, die außer der eindeutigen Zuweisung keine weitere Funktion erfüllen.
- SILAB interpretiert das Rautezeichen `#` als Beginn eines Kommentars und ignoriert alles, was in der Zeile nach diesem Zeichen folgt. In roter Schrift sind Kommentare geschrieben, die Ihnen zur Strukturierung der Konfigurationsdatei dienen, während sie von SILAB nicht beachtet werden. Eine zusätzliche Möglichkeit zur optischen Strukturierung der Konfigura-

tionsdatei besteht in dem Einrücken von Abschnitten nach Gliederung in die unterschiedlichen Bestandteile der Konfiguration. Unterabschnitte können eine weitere Stufe eingerückt werden.

- Anders als bei den anderen Konfigurationsbereichen stehen die allgemeinen Angaben zu weiteren Komponenten der Simulation nicht in Klammern. An dieser Stelle werden verschiedene Standardkonfigurationen als include-Dateien eingefügt (s. Abschnitt 9.5). Dabei handelt es sich um Standardkonfigurationen der Sound- und Fußgänger-Simulation, der Visualisierung und der grafischen Repräsentation der Datenbasis.
- TRF steht für ‚Traffic‘ und ist ein Verbund aus verschiedenen DPUs, die in SILAB für die Simulation anderer Verkehrsteilnehmer zuständig sind. Auf diese DPUs wird zur Konfiguration der allgemeinen Angaben zur Verkehrssimulation zurückgegriffen. Mit TRF können andere Verkehrsteilnehmer (PKW, LKW, Motorräder und Fahrräder) in ein Streckennetz eingefügt werden und einzeln oder in Gruppen in Abhängigkeit vom Verhalten des Testfahrers oder vom Verhalten anderer Verkehrsteilnehmer gesteuert werden. Die Messwerte (Position, Geschwindigkeit, Beschleunigung usw.) jedes einzelnen Verkehrsteilnehmers können aufgezeichnet oder zur weiteren Verwendung mit anderen DPUs verknüpft werden. Nähere Informationen zu TRF finden Sie in dem Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#).
- SCNX ist die Datenbasis für die SILAB-Fahrsimulation. SCN steht für ‚Scenery‘ und X für die weiterentwickelte Version ‚Extended‘. Die Datenbasis enthält sowohl Informationen über den Verlauf, die Gestaltung und den Aufbau des befahrbaren Streckennetzes, als auch über Aussehen und Höhenprofil der Umgebung. Nähere Informationen zu SCNX finden Sie in dem Dokument [REFERENZ DATENBASIS SCNX](#).

9.4 Bearbeitung der Konfigurationsdatei

SILAB-Konfigurationsdateien und include-Dateien sind unformatierte Textdateien (sogenannte Plain-Text-Dateien), die ähnlich wie Quelltexte einer Programmiersprache bearbeitet werden. Das Programm SILAB durchsucht diese Datei nach den benötigten Informationen.

Es kommen Plain-Text-Editoren wie Geany, TextPad oder Crimson Editor zum Einsatz. Die meisten Plain-Text-Editoren bieten Funktionen wie Syntaxhervorhebung oder das Einbinden von externen Programmen, wodurch das Schreiben von Konfigurationen erheblich erleichtert wird. Im Lieferumfang von SILAB ist beispielsweise ein Programm enthalten, mit dem Sie eine Konfigurationsdatei auf syntaktische Korrektheit überprüfen können. Dieses Programm lässt sich in Ihren Text-Editor einbinden und mit einer Tastenkombination aufrufen (je nach Texteditor unterschiedlich, in Geany z.B. mit F8). Wir empfehlen die Verwendung des Texteditors Geany, da dieser frei verwendbar ist, eine gute Unterstützung für die Syntaxhervorhebung und die Einbindung externer Programme bietet sowie für alle aktuellen Betriebssysteme verfügbar ist. Im nächsten Abschnitt erfahren Sie, wie Sie den Texteditor Geany installieren und einrichten.

9.4.1 Installation und Einrichten des Texteditors Geany

Zur Installation und Einrichtung des Texteditors Geany gehen Sie bitte wie folgt vor.

Tabelle 21. Anleitung zur Installation und Einrichtung des Texteditors Geany.

Schritt	Aktion	Erklärung
1	Geany herunterladen	Link zum Download: https://www.geany.org/Download/Releases
2	Geany installieren	Führen Sie die Installation mit den Standardeinstellungen durch.
3	Geany starten und beenden	Starten Sie Geany einmal und beenden Sie das Programm wieder.
4	Syntaxdefinitionsdateien von SILAB in das Geany-Verzeichnis entpacken	Entpacken Sie die Syntaxdefinitionsdateien aus D:\SILAB\System\Syntax\SILAB_Syntax_Geany.zip in das Verzeichnis C:\Users\<IhrBenutzername>\AppData\Roaming\geany Setzen Sie Ihren Windows-Benutzernamen für <IhrBenutzername> ein. Das Verzeichnis AppData ist versteckt; ggf. müssen Sie daher im Windows Explorer die Anzeige ausgeblendeter Elemente aktivieren. Überschreiben Sie beim Entpacken ggf. bereits vorhandene Dateien.
5	Geany erneut starten	Starten Sie Geany erneut
6	Verzeichnis von SILAB in Geany eintragen (Teil 1)	Wählen Sie im Menü die Optionen: Werkzeuge → Konfigurationsdateien → Dateityp-Einstellungen → filetypes.silabcfg.conf Gehen Sie zum Ende der Datei (Abschnitt [build-menu]). Ersetzen Sie hier die Bezeichnung ‚Dev‘ (z.B. in ‚Syntax-check Dev‘) durch die aktuelle SILAB-Version ‚6.5‘. Ersetzen Sie im gesamten Abschnitt den Pfad D:\SILAB_Dev\ durch das Verzeichnis, in dem Sie SILAB installiert haben Anmerkung: Nehmen Sie im unteren Abschnitt an der Bezeichnung ‚SILAB Stable‘ keine Änderungen vor Speichern und schließen Sie die Datei
7	Verzeichnis von SILAB in Geany eintragen (Teil 2, optional)	Falls Sie auch grafische Objektdaten der Visualisierung SGE bearbeiten möchten: Wiederholen Sie Schritt 6 mit der Datei filetypes.silabsig.conf

9.4.2 Regeln zur Bearbeitung der Konfigurationsdatei

Zur Bearbeitung einer Konfigurationsdatei ist es wichtig, dass die Textdateien syntaktisch korrekt formuliert werden. Dieser Abschnitt enthält grundlegende Regeln zur Bearbeitung von Konfigurationsdateien. Es gibt darüber hinaus für die verschiedenen Bestandteile der Konfiguration viele Anweisungen, die nicht im Rahmen dieses Handbuchs abgedeckt werden können. Das Dokument **REFERENZ SILAB-KONFIGURATION UND BEDIENUNG** enthält ein Kapitel (2.2.2 Notation) mit detaillierten Erklärungen zu den formalen grammatikalischen Regeln der Konfigurationssprache von SILAB. Weiterhin ist es hilfreich, sich bei der Anpassung einer Konfigurationsdatei an der Form der Anweisungen in der Standardkonfigurationsdatei zu orientieren. Wenn Sie mit Geany eine Konfigurationsdatei neu erstellen oder eine bereits bestehende Datei bearbeiten, sollten Sie darauf achten, dass unter dem Pfad ‚Dokument → Dateityp festlegen‘ der Dateityp ‚silabcfg-Datei‘ ausgewählt ist. Im Folgenden werden die Grundregeln zur Erstellung und Bearbeitung von Konfigurationsdateien anhand eines Ausschnitts einer Beispiel-Konfiguration erläutert.

```
SILAB System
{
    # Systemkonfiguration
    LaunchData1 = ("Aufsetzpunkt auswählen", SCN_SetupPoint,
        LD_COMBO_DEFAULT, "SetupPoint1");
    WatchdogPeriod = 10000;
};

SILAB Configuration
{
    Computerconfiguration IrgendeinName
    {
        # Rechner-Konfiguration
        Executable = true;

        Computer LOCALHOST
        {
            IP = "10.1.2.180";
            Frequency = 60;
            Alias = {ALL, ASS, TRF, MOV, VDYN, OBS, MOTION, SCN, ODB};
            Operator = true;
        };

        DPUConfiguration NochEinName
        {
            # DPU-Konfiguration
            include system\DPUBase_SGE.inc
        };
    };
};
```

Code 2. Beispiel für System-, Rechner- und DPU-Konfiguration.

Eine Konfigurationsdatei besteht aus verschiedenen Anweisungen. Diese werden mit blauer, fettgedruckter Schrift kenntlich gemacht. Ziffern werden mit grüner, Kommentare mit roter und Strings

mit gelber Schrift gekennzeichnet. Tabelle 22 enthält einen Überblick über die Regeln zu Bearbeitung einer Konfigurationsdatei.

Tabelle 22. Regeln zur Bearbeitung der Konfigurationsdatei.

Regel	Erläuterung
Groß- und Kleinschreibung	Es wird grundsätzlich Groß- und Kleinschreibung unterschieden. Dies gilt insbesondere für die Namen von DPUs und Instanzen (Erklärung zu DPU-Instanzen s. Abschnitt 9.8) und alle feststehenden Bezeichner.
Kommentare	Das Zeichen # sowie die Zeichenfolge // leiten einen Kommentar ein. Alles, was hinter diesen Zeichen steht, wird von SILAB ignoriert. Geany kennzeichnet Kommentare in roter Farbe. Kommentare dienen der Dokumentation und besseren Lesbarkeit der Konfigurationsdatei.
Abschließen einer Anweisung mit Strichpunkt ;	Jede Anweisung wird mit einem Strichpunkt abgeschlossen.
Gruppierung zusammengehöriger Bestandteile	Zusammengehörige Blöcke werden mit geschwungenen Klammern umschlossen {...}. Zur Übersichtlichkeit sollten zusammengehörige Blöcke auf einer Höhe eingerückt werden. Ein Block kann aus weiteren Subblöcken bestehen.
Zuweisungsoperator =	Wertzuweisungen erfolgen mit dem Gleichzeichen. Die Variable k erhält den Wert auf der rechten Seite des Gleichzeichens. Auf der rechten Seite kann auch ein Term stehen. k = Wert;
include-Funktion	Mit der Anweisung ,include' gefolgt vom Dateinamen einer anderen Konfigurationsdatei, die im gleichen Projektverzeichnis gespeichert ist, wird diese Datei an dieser Stelle in die Haupt-Konfigurationsdatei eingebunden.
String in Anführungszeichen	Ein String (Kette von beliebigen Zeichen) wird von doppelten Anführungsstrichen umschlossen. Geany kennzeichnet einen String in gelber Farbe.

Beim Bearbeiten von Skriptdateien können versehentlich Fehler auftreten. Damit Sie nicht erst beim Laden der Datei in SILAB auf Fehler hingewiesen werden, enthält der Texteditor ein Tool, das die Überprüfung des Skripts schon im Texteditor vornimmt. Wenn Sie die Syntax im Geany-Texteditor prüfen möchten, dann wählen Sie im Menü ,Erstellen → Syntaxcheck'. Unter dem Reiter ,Meldungen' im Output-Fenster von Geany (unten) erhalten Sie nun ggf. eine Fehlermeldung.

Wenn man bei Code 2 z.B. hinter ,WatchdogPeriod = 10000' vergessen hätte, den Strichpunkt zum Beenden der Anweisung einzufügen, würde der Syntax-Check folgende Fehlermeldung ausgeben:

```
Syntax-Check: Datei D:\SILAB\Projects\Konfiguration, Zeile 10 : syntax
error, unexpected TK_Identifizier, expecting TK_Semicolon
```

Code 3. Beispiel für eine Fehlermeldung des Syntax-Checks.

Diese Fehlermeldung enthält folgende Angaben:

- Der Name der Skript-Datei, in der sich der Fehler befindet, lautet: D:\SILAB\Projects\Konfiguration
- Die Zeile, in der SILAB bemerkt hat, dass die Skript-Datei einen Fehler enthält. Bitte beachten Sie, dass diese Angabe nicht genau der Zeile mit dem Fehler entsprechen muss. Meistens werden eine oder ein paar Zeilen nach der Stelle gemeldet: Zeile 10
- Die Art des Fehlers und einige nähere Angaben dazu: Es handelt sich in diesem Fall um einen Syntaxfehler und SILAB hätte einen Strichpunkt erwartet: `syntax error, unexpected TK_Identifier, expecting TK_Semicolon`

9.5 Arbeiten mit der Standardkonfiguration

Die Standardkonfigurationsarchitektur besteht aus zahlreichen kleinen include-Dateien, aus denen automatisch Basiskonfigurationen zusammengefügt werden. Üblicherweise passen wir bei der Installation von SILAB diese Basiskonfigurationen auf die Hardware Ihres Simulators an. Tabelle 23 enthält eine Übersicht über die Inhalte der include-Dateien, die die Standardkonfiguration beinhaltet. Mit dem Mechanismus der Ableitung können Sie daraus Konfigurationen für Ihre speziellen Fragestellungen und Aufgaben erstellen. Ein Gerüst für eine Konfigurationsdatei, die auf der Standardarchitektur beruht, finden Sie in Code 4.

Um einen reibungslosen Betrieb der Simulationskonfiguration zu gewährleisten, sollten die include-Dateien der Standardkonfiguration in jeder Konfigurationsdatei enthalten sein (s. Code 4). Um die Standardeinstellungen an Ihre Anforderungen anzupassen, sollten niemals Änderungen innerhalb der include-Dateien der Standardkonfiguration vorgenommen werden. Stattdessen können die Anpassungen in der Konfigurationsdatei unterhalb der include-Zeile erfolgen, wodurch die entsprechenden Werte der include-Dateien überschrieben werden.

Wenn Sie die Standardkonfiguration der DPUs auf Ihre speziellen Anforderungen anpassen möchten, arbeiten Sie in dem Bereich, in dem sich Code 4 der Kommentar ‚# ab hier folgen Ihre DPUs und Verknüpfungen‘ befindet. Das Vorgehen zum Ableiten eigener DPU-Pools von bereits bestehenden DPU-Pools wird in Abschnitt 9.8.2 näher beschrieben. Der Pool `MeinDPUPool` (oder irgendein anderer Name) sollte eine Ableitung vom Basispool mit dem Namen `Basispool` sein. Dieser Basispool enthält bereits eine vollständige Simulationsumgebung für Ihren Fahr Simulator (ebenso wie die Rechnerkonfiguration). Im abgeleiteten Pool `MeinDPUPool` können Sie jetzt DPUs instanziiieren (Erklärung s. Abschnitt 9.8) und verknüpfen, die Sie speziell für Ihre Aufgabenstellung brauchen (z.B. Assistenzsysteme, Datenaufzeichnung, Ansteuerung zusätzlicher Hardware usw.). Sie können auch mehrere Pools vom Basispool oder von einem bereits abgeleiteten Pool ableiten.

Beim Laden der Konfiguration in SILAB erscheint ein Dialog, mit dem Sie dann denjenigen Pool auswählen, den Sie für diesen Simulationsdurchgang verwenden möchten.

Mehr Details zum Arbeiten mit der Standardkonfiguration erhalten Sie in Kapitel 3 im Dokument **REFERENZ SILAB-KONFIGURATION UND BEDIENUNG**.

```
# Systemkonfiguration
include system/SYSBase.inc

SILAB Configuration
{
    Computerconfiguration Rechner
    {
        # Rechner-Konfiguration
        include system/CompBase.inc
    };

    DPUConfiguration DPUs
    {
        # Basis DPU-Konfiguration
        include system/DPUBase_SGE.inc

        # Ihre eigene abgeleitete DPU-Konfiguration
        Pool MeinDPUPool : Basispool
        {
            Executable = true;
            # ab hier folgen Ihre DPUs und Verknüpfungen
        };
    };
};

SILAB TRF
{
    # Grundkonfiguration von TRFX
    include SILAB_TRFX_600.cfg
};

# Allgemeine Angaben zu weiteren Komponenten der Simulation
include SNDX/SNDX.inc
include SGE/SGE_600.inc
include MOV/MOP_600.inc
include SCNX/SCNXSGE_600.cfg

# Szenario-Definition
SILAB SCN
{
    # Grundkonfiguration von SCNX
    include SCNX/SCNX_600.cfg
    # ab hier folgt Ihre Definition von Streckenstücken, Szenario-Modu-
    len
    # und Modul-Abläufen.
};
```

Code 4. Konfigurationsdatei mit Standard-Architektur.

Tabelle 23. Übersicht über include-Dateien der Standardkonfiguration.

include-Datei	Beschreibung
system/SYSBase.inc	System-Standardkonfiguration
system/CompBase.inc	Rechnerstandardkonfiguration mit Informationen über die an der Simulation beteiligten Rechner
system/DPUBase_SGE.inc	DPU-Standardkonfiguration
SILAB_TRFX_600.cfg	Standardkonfiguration mit den allgemeinen Einstellungen für die Verkehrssimulation, die u.a. grundlegende Verhaltensmuster und Fahrzeuggruppen beinhaltet
SNDX/SNDX.inc	Standardkonfiguration der Sound-Simulation
SGE/SGE_600.inc	Standardkonfiguration der Visualisierung SGE, die Voreinstellungen des grafischen Subsystems beinhaltet
MOV/MOP_600.inc	Standardkonfiguration der Fußgängersimulation, die Fußgängermodelle und -animationen beinhaltet
SCNX/SCNXSGE_600.cfg	Standardkonfiguration der grafischen Repräsentation der Datenbasis SCNX, mit der für eine realistische Darstellung des simulierten Szenarios auf den Anzeigegeräten des Fahrsimulators (Monitore oder Projektoren) gesorgt wird
SCNX/SCNX_600.cfg	Standardkonfiguration der SILAB-Datenbasis SCNX, die sowohl Informationen über den Verlauf, die Gestaltung und den Aufbau des befahrbaren Streckennetzes, als auch über Aussehen und Höhenprofil der Umgebung enthält

9.6 Systemkonfiguration

In diesem Kapitel lernen Sie, welche Informationen in der Systemkonfiguration enthalten sind und welche Möglichkeiten Sie haben, um diesen Konfigurationsabschnitt zu bearbeiten.

In der Systemkonfiguration werden Parameter definiert, die global das Verhalten und die Ablaufsteuerung von SILAB beeinflussen. Sie besteht aus den folgenden Bestandteilen:

LAUNCH-PROZESS:

- Ablaufverhalten der Simulation

LAUNCH-DATEN:

- Informationen, die vor jedem Start der Simulation an DPU-Instanzen übermittelt werden

WATCHDOG-PARAMETER:

- Überwachung der Remoterechner vom Operatorrechner aus

SYSTEMSTEUERUNGS-PARAMETER:

- Parameter zur Beeinflussung weiterer Eigenschaften von SILAB

In den folgenden Abschnitten wird auf die einzelnen Bestandteile der System-Konfiguration genauer eingegangen und Möglichkeiten zur Modifizierung aufgezeigt.

9.6.1 Launch-Prozess

Mit dem Eintrag `LaunchProcess` können Sie das Ablaufverhalten der Simulation verändern. Die dazugehörige Anweisung in der Konfigurationsdatei lautet:

```
LaunchProcess = {<Launch Token>;
```

Code 5. Definition des Launch-Prozesses.

Die Bezeichner in der Menge `LaunchProcess = { . . . }` stehen jeweils für einen bestimmten Übergang zwischen den Zuständen der Applikation. Anstatt `<Launch Token>` tragen Sie eine oder mehrere der folgenden Bezeichnungen in die geschwungenen Klammern ein.

Tabelle 24. Übersicht über Launch Token.

<Launch Token>	Beschreibung
LC_SWITCHON_ON_LOAD	SILAB kontaktiert automatisch die Remoterechner, wenn die Konfiguration geladen wird (Standardeinstellung; kann nicht deaktiviert werden).
LC_LAUNCH_ON_SWITCHON	SILAB startet automatisch den Launch-Schritt, wenn die Applikation die Remoterechner erfolgreich kontaktiert hat. SILAB fragt hier keine Launch-Daten ab (s. Abschnitt 9.6.2).
LC_START_ON_LAUNCH	SILAB startet automatisch nach dem Launch-Schritt die Simulation.
LC_SWITCHOFF_ON_STOP	SILAB schaltet die Remoteapplikationen automatisch aus, wenn die Simulation gestoppt wurde.
LC_CLOSE_ON_SWITCHOFF	SILAB beendet sich automatisch, wenn die Simulation ausgeschaltet wurde.

9.6.2 Launch-Daten

Bei Launch-Daten handelt es sich um Informationen, die Sie vor jedem Start einer Simulation den DPU-Instanzen (Erklärung für DPU Instanzen, s. Abschnitt 9.8) mitteilen können. Das bedeutet, dass Sie vor jedem Start einer Simulation in SILAB im Launch-Vorgang bestimmte Einstellungen für diesen spezifischen Simulationsdurchlauf treffen können. SILAB zeigt Ihnen hierzu vor dem Start der Simulation einen ‚Launch-Wizard‘ an. Wie bereits in Abschnitt 5.6.1 erklärt, können Sie beim Launch für jeden Simulationslauf erneut festlegen, ob eine Aufzeichnungsdatei erstellt werden soll und den gewünschten Dateinamen festlegen. Eine weitere Möglichkeit besteht darin, beim Launch einer Versuchskonfiguration den Aufsetzpunkt des Fahrers anzugeben (s. Abschnitt 5.6.3). Während Sie in Abschnitt 5.5 gelernt haben, wie Sie Launch-Daten mithilfe des Programms `SILABSessionEdit` definieren, wird in diesem Abschnitt die direkte Bearbeitungsmöglichkeit innerhalb der Konfigurationsdatei beschrieben. Bevor auf die konkrete Erstellung der Launch-Wizards für die Datenaufzeichnung und das Festlegen des Aufsetzpunktes eingegangen wird, erhalten Sie zunächst eine Erläuterung zur Erstellung von Launch-Daten in der Systemkonfiguration.

Die abzufragenden Daten werden in der Konfigurationsdatei folgendermaßen definiert:

```
LaunchData<Nummer> =
  ("<Titeltext>",<Variablenname>,<Variablentyp>) |
  ("<Titeltext>",<Variablenname>,<Variablentyp>,"<Defaultwerttext>") |
  ("<Titeltext>",<Variablenname>, LD_COMBO, "<Textauswahlelemente>") |
  ("<Titeltext>",<Variablenname>, LD_COMBO_DEFAULT,
   "<Defaultwerttext>", "<Textauswahlelemente>")
```

Code 6. Definition der Launch-Daten.

Der Eintrag `LaunchData1` beschreibt dabei den ersten Eintrag im Launch-Wizard (z.B. ‚Daten aufzeichnen‘ in Abbildung 28) und `LaunchData2` den zweiten Eintrag (z.B. ‚Dateiname‘ in Abbildung 28). Um einen dritten und vierten Eintrag hinzuzufügen, nutzen Sie die Benennungen `LaunchData3` und `LaunchData4`. Die eingegebenen Werte werden an alle im Computerpool befindlichen Rechner versendet und von deren DPUs ausgewertet. Tabelle 25 enthält Erklärungen zu den einzelnen Elementen der Launch-Daten-Definition:

Tabelle 25. Übersicht über Elemente der Launch-Daten-Konfiguration

Element	Beschreibung	
<Titeltext>	Derjenige Text, der im Launch-Wizard angezeigt wird (‚Daten aufzeichnen‘ und ‚Dateiname‘ in Abbildung 28)	
<Variablenname>	Der Variablenname hängt von den gewünschten Launch-Daten ab und wird von den benötigten DPUs festgelegt. Diese Variablenamen können relevant sein:	
	VAR_SAVEDATA	Mit dieser Variablen wird der DPUDataFile vor jedem Start mitgeteilt, ob für diesen Simulationslauf eine Datenaufzeichnung erfolgen soll
	VAR_DATAFILENAME	Über diese Variable wird der Name der Aufzeichnungsdatei festgelegt
<Variablentyp>	SCN_SetupPoint	Über diese Variable wird der Aufsetzpunkt ausgewählt
	Hier wird der gewünschte Variablentyp aus der folgenden Liste eingetragen. Je nach Variablentyp müssen hinter der Bezeichnung des Variablentyps noch ein Defaultwert-Text und Textauswahlelemente angegeben werden.	
	LD_EDIT	Leeres Eingabefeld für Texteingabe
	LD_EDIT_DEFAULT	Eingabefeld für Texteingabe mit dem Defaulttext als Inhalt
	LD_BOOL	Checkbox zum Ankreuzen
	LD_BOOL_DEFAULT	Checkbox mit Defaultwert belegt ,1‘ für angekreuzt und ,0‘ für nicht angekreuzt
	LD_COMBO	Drop-down Menü mit den vorgegebenen Textelementen zur Auswahl. Hier müssen zusätzlich die Textauswahlelemente in Anführungszeichen und durch Kommas abgetrennt angegeben werden.
	LD_COMBO_DEFAULT	Drop-down Menü mit Defaulttext und den vorgegebenen Textelementen zur Auswahl. Hier müssen zusätzlich der Defaultwerttext und die Textauswahlelemente in

Anführungszeichen und durch Kommas abgetrennt angegeben werden.

9.6.2.1 Launch der Datenaufzeichnung

Um den Launch-Wizard zur Datenaufzeichnung so wie in Abbildung 28 zu generieren, werden folgende Angaben in der Systemkonfiguration gemacht:

```
SILAB System
{
    LaunchData1 = ("Daten aufzeichnen", VAR_SAVEDATA, LD_BOOL_DEFAULT,
        "1");
    LaunchData2 = ("Dateiname", VAR_DATAFILENAME, LD_EDIT_DEFAULT,
        "Data");
};
```

Code 7. Konfiguration der Launch-Daten zur Datenaufzeichnung.

Weitere Informationen finden Sie im Dokument [DPUDATAFILE](#).

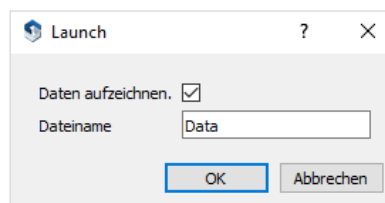


Abbildung 28. Launch-Wizard zur Datenaufzeichnung in SILAB.

9.6.2.2 Launch des Aufsetzpunktes

Um den Aufsetzpunkt bei jeder Simulationsvorbereitung über die Launch-Daten vom Versuchsleiter auswählen zu lassen (Abbildung 29), machen Sie folgende Angaben in der Systemkonfiguration:

```
SILAB System
{
    LaunchData1 = ("Aufsetzpunkt auswählen", SCN_SetupPoint,
        LD_COMBO_DEFAULT, "Anfang")
};
```

Code 8. Konfiguration der Launch-Daten zur Auswahl der Aufsetzpunkte.

Wenn Sie Ihre Session mit `SILABSessionEdit` erstellt haben, sind die unterschiedlichen Aufsetzpunkte (engl. ‚Setup points‘) bereits in der Szenariodefinition vorkonfiguriert. In der Konfiguration der Launch-Daten können Sie nun definieren, ob der Launch-Wizard zum Einstellen des Aufsetzpunktes vor jeder Simulation angezeigt werden soll, welche Beschriftung dafür angezeigt wird (z.B. „Aufsetzpunkt auswählen“) und welcher Aufsetzpunkt als Default-Wert erscheinen soll (z.B. „Anfang“).

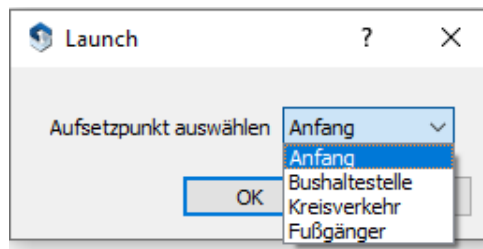


Abbildung 29. Launch-Wizard zur Auswahl des Aufsetzpunktes.

9.6.3 Watchdog-Parameter

„Watchdogs“ sind Funktionen, die Zeitüberschreitungen von Rechnerabläufen überwachen. Die Überwachung der Remoterechner bei SILAB erfolgt periodisch vom Operatorrechner aus. Sobald die Remoterechner für eine definierte Zeitdauer die Anfragen des Operatorrechners nicht mehr beantworten, wird die Simulation angehalten.

Die dazugehörige Anweisung in der System-Konfiguration lautet:

```
<Watchdog-Parameter> = <Zeit in ms | Anzahl der Versuche>;
```

Code 9. Definition der Watchdog-Parameter.

Das Symbol | bedeutet, dass Sie je nach Watchdog-Parameter entweder die Zeit in ms oder die Anzahl der Versuche rechts vom Operatorzeichen angeben. Es gibt folgende Watchdog-Parameter, die Sie in die Anweisung eintragen können.

Tabelle 26. Übersicht über Watchdog-Parameter.

<Watchdog-Parameter>	Beschreibung
WatchdogPeriod	Periode des Watchdogs in ms
WatchdogRemote_Lost	Anzahl der Versuche einen Remoterechner zu erreichen
WatchdogDispatchConfigs	Timeout (ms) für den Versuch, die geöffnete Konfigurationsdatei an die Remoterechner zu übertragen.
WatchdogExecApps	Timeout (ms): Startvorgang der Konfiguration
WatchdogControllerInit	Timeout (ms): Vorbereitungsphase
WatchdogShutDownControllers	Timeout (ms): Shutdown-Vorgang
WatchdogTerminateApps	Timeout (ms): Schließen der Remote-Applikation
WatchdogKillApps	Timeout (ms): Hartes Beenden der Remote-Applikation

Normalerweise müssen Sie diese Parameter nicht verändern. Ein möglicher Anwendungsfall kann sein, dass ein (langsamer) Rechner nach dem Laden einer Konfiguration zur Vorbereitung der Simulation mehr Zeit benötigt, als von den Watchdog-Parametern erlaubt wird. Bevor der Rechner die Vorbereitung abschließen konnte, wird die Simulation aufgrund eines Watchdog-Timeouts beendet. Um dies zu verhindern, sollten Sie den Parameter WatchdogPeriod schrittweise erhöhen, bis das Problem nicht mehr auftritt.

9.6.4 Systemsteuerungs-Parameter

Mit den folgenden Systemsteuerungs-Parametern können Sie weitere Eigenschaften von SILAB beeinflussen.

Tabelle 27. Übersicht über Systemsteuerungs-Parameter.

Parameter	Beschreibung	Default-Wert
ShutDownOnDPUErrror	Simulation abbrechen, wenn eine DPU einen Fehler auslöst (z.B. Speicherzugriffsfehler). Auch ohne diese Einstellung wird eine Meldung in der Log-Datei ‚SILAB.error.log‘ eingetragen.	false
MinSleepIdleTime	Wenn nach Abarbeiten eines Simulationsschrittes noch mindestens <code>MinSleepIdleTime</code> Millisekunden übrig sind, wird der Prozessor in den idle-Zustand versetzt, Abhängig von der Hintergrundbelastung des Systems kann die Einstellung eines höheren Wertes die Anzahl auftretender Verzögerungen (die als Ausgang <code>TimeError</code> der <code>DPUclock</code> ausgegeben werden) reduzieren.	2.0
TransferProject	Zum einfacheren Test von Projekten kann das Projektverzeichnis bei jedem Start der Simulation automatisch auf die Simulatorrechner kopiert werden, indem dieser Wert auf <code>true</code> gesetzt wird. Hinweis: Dies verzögert den Start der Simulation, auch wenn keine Dateien geändert wurden. Es wird daher empfohlen, vor dem Start einer Versuchsreihe den Eintrag wieder zu entfernen.	false

9.7 Rechner-Konfiguration

In diesem Kapitel erfahren Sie, welche Informationen die Rechnerkonfiguration enthält und wie diese bearbeitet werden können.

Der Abschnitt in der Konfigurationsdatei zur Festlegung der Rechner- und DPU-Konfiguration wird mit ‚SILAB Configuration‘ eingeleitet. Dieser Abschnitt enthält sowohl die Informationen über die für die Simulation verwendeten Rechner, als auch die Verteilung der DPUs auf die verschiedenen Rechner.

Nachdem Sie eine Konfigurationsdatei auf dem Operatorrechner eines Simulators geladen haben, verschickt SILAB diese an alle Simulationsrechner. SILAB muss Informationen darüber erhalten, welche Rechner an einer Simulation beteiligt sind. Die Rechnerkonfiguration in der Konfigurationsdatei enthält eine Liste aller an der Simulation beteiligten Rechner. Für jeden Rechner enthält die Konfigurationsdatei folgende Informationen:

RECHNERNAME

- Unter welchem Namen ist der Rechner im Windows-Netzwerk registriert? Der Rechnername muss dem im Betriebssystem vorgegebenen Namen entsprechen. In SILAB besteht dieser Name immer ausschließlich aus Großbuchstaben.

IP-ADRESSE

- Über welche IP-Adresse kann mit dem Rechner via TCP/IP kommuniziert werden?

FREQUENCY

- Taktrate. Wie oft pro Sekunde soll ein Berechnungsschritt aller DPUs, die auf diesem Rechner laufen, durchgeführt werden (Angabe mit Fließkommazahlen möglich, z.B. 20.52). Sind die DPUs eines Rechners sehr rechenintensiv, kann der Rechner diese Zeitschritte ggf. nicht einhalten. In der Folge gerät der Rechner im Laufe der Simulation mit der Aktualisierung seiner DPUs in Verzug. Die entstehenden Zeitverzögerungen werden für jeden Rechner in der Baumansicht von SILAB angezeigt (s. Abschnitt 6.5). Grundsätzlich sollten in einer Simulation keine Zeitverzögerungen auftreten. Oder:
- Period MS (Periode des Computers). Zeitschritt in Millisekunden, zu denen Berechnungsschritte durchgeführt werden (PeriodMS = 10 bedeutet, dass die Berechnungsschritte alle 10 ms durchgeführt werden, also 100 Mal pro Sekunde)

OPERATOR

- Wird die Simulation von diesem Rechner aus bedient? Falls es sich bei dem entsprechenden Rechner um den Operatorrechner handelt, dann wird `Operator = true` gesetzt.

ALIAS-NAMEN

- Optionale Info: Zusätzlich zu seinem eigentlichen (Betriebssystem-)Namen kann ein Rechner beliebig viele sogenannte Aliasnamen haben. Hat ein Rechner X einen Aliasnamen Y dann bedeutet dies: Rechner X ist SILAB auch unter dem Namen Y bekannt. Im Abschnitt der DPU-Konfiguration können Sie den Rechner entweder über seinen Betriebssystemnamen oder über einen seiner Aliasnamen identifizieren.

GROUP

- Optionale Info: Die Zuweisung eines Rechners zu einer Gruppe (Group) beeinflusst die Verknüpfung von DPUs. Variablen, die auf mehreren Remote-Computern zur Verfügung stehen, werden bevorzugt von einem Computer bezogen, der in derselben Gruppe wie der lokale Computer ist.

Rechner, die häufig zusammen für die Simulation eingesetzt werden, können in sogenannten Rechner-Pools zusammengefasst werden. Die Rechnerkonfiguration enthält eine beliebige Anzahl von Rechner-Pools. Durch einen sogenannten Vererbungsmechanismus kann ein Pool von einem anderen Pool abgeleitet werden. Das bedeutet, dass der abgeleitete Pool dabei alle im Vorgänger enthaltenen Rechner erbt und diesen Pool um zusätzliche Rechner erweitern kann. Die verschiedenen Rechner-Pools werden Ihnen beim Öffnen der Konfiguration in SILAB als Auswahl präsentiert. In der

Liste der auswählbaren Pools werden nur solche aufgenommen, bei denen der Eintrag `Executable = true` gesetzt worden ist.

Im Folgenden finden Sie ein Beispiel für eine Rechnerkonfiguration eines Simulators, der aus drei Rechnern (namens OPERATORRECHNER, HAUPTRECHNER, GRAFIKRECHNER) besteht. Diese drei Rechner werden zu einem Rechner-Pool zusammengefasst, der den Namen ‚Sim1‘ erhält.

```
Computerconfiguration Rechner
{
  Pool Sim1
  {
    Executable = true;           # Name der Rechnerkonfiguration
                                # Name des Rechner-Pools
                                # Auswahl des Rechner-Pools
  }
  Computer OPERATORRECHNER
  {
    IP = "10.1.2.1";           # Rechnername
    Frequency = 50;
    Operator = true;           # IP-Adresse
  };
  Computer HAUPTRECHNER
  {
    IP = "10.1.2.2";           # Taktrate
    PeriodMS = 10;
  };
  Computer GRAFIKRECHNER
  {
    IP = "10.1.2.3";           # alternativ zur Taktrate: Periode
    Frequency = 60;            # des Computers
    Alias = {COMP_A};          # Vergabe eines Aliasnamens
  };
};
};
};
```

Code 10. Beispiel für eine Rechnerkonfiguration.

Normalerweise gibt es keinen Bedarf, die Standardkonfiguration der Rechner zu ändern. Das bedeutet, dass Sie an der Stelle der Rechnerkonfiguration eine `include`-Anweisung zu den gewünschten Rechner-Pools einfügen (s. Code 4).

Weitergehende Informationen zur Rechnerkonfiguration finden Sie in den Kapiteln 2.1, 2.2, 2.5 und 2.6 im Dokument [REFERENZ SILAB-KONFIGURATION UND BEDIENUNG](#).

9.8 DPU-Konfiguration

Die DPU-Konfiguration definiert, welche DPUs für die Simulation benötigt werden, auf welchen Rechnern sie laufen und wie sie miteinander verschaltet sind. DPUs sind Softwarekomponenten, die für verschiedene Aufgaben und Bereiche der Simulation zuständig sind. Es gibt DPUs, die speziell auf die Fahrsimulation zugeschnitten sind, z.B. für die Fahrdynamik, Soundsimulation, Datenbasis, Verkehr, Visualisierung und Datenaufzeichnung. Daneben gibt es zahlreiche weitere DPUs, die sehr

universell eingesetzt werden können. Ihre Aufgaben sind z.B. das Generieren, Vergleichen, Mischen und Filtern von Signalen, die Berechnung von Verknüpfungen und das Verstärken, Abschwächen und Begrenzen von Werten. Diese DPUs werden als ‚universelle DPUs‘ bezeichnet.

Zu Beginn dieses Abschnitts erhalten Sie zunächst eine Übersicht über wichtige DPUs, die im Lieferumfang von SILAB enthalten sind.

Tabelle 28. Übersicht über wichtige DPUs und ihre Aufgaben.

DPU	Aufgabe	Univer-sell?
DPUAmplifier	Verstärkung/Abschwächung/Umrechnung von Signalen	✓
DPUBooleanGate	Berechnung logischer Operationen (AND, OR, XOR)	✓
DPUCANBUS	Lesen von/Schreiben auf CAN-Bus	✓
DPUComparator	Vergleich von Signalen	✓
DPUCounter	Einfacher Zähler	✓
DPUCreateProcess	Start anderer Windows-Programme	✓
DPUDelayLine	Verzögerung von Signalen	✓
DPUDemultiplexer	Demultiplexer für Signale	✓
DPUDX8Joystick	Abfrage eines DirectX kompatiblen Joysticks	✓
DPUFader	Ein-/Ausblenden von Signalen	✓
DPUJoystickConsole	Umrechnung von Joystickwerten in Gas/Bremse/Lenkung	
DPUKeyboard	Verarbeiten von Tastatureingaben	✓
DPU Limiter	Begrenzung von Signalen	✓
DPU LookUpTable	Zweidimensionale Look Up Tabelle	✓
DPU MixAmplifier	Kombination von Signalen	✓
DPUMOPSGE	Visualisierung von Fußgängern	
DPUMOV	Teil der Fußgängersimulation	
DPUMOVClient	Teil der Fußgängersimulation	
DPUmstVeda	Fahrdynamiksimulation mstVeda	
DPU Multiplexer	Multiplexer für Signale	✓
DPU Oscillator	Signalgenerator	✓
DPU Parallelport	Lesen von/Schreiben auf Parallelport	✓
DPU Random	Zufallszahlengenerator	✓
DPU SampleFilePlayer	Schrittweises Auslesen von Werten aus einer Datei	✓
DPUSCNXSGE	Visualisierung des Streckennetzes und der Landschaft	
DPUSCNX	Datenbasis	
DPU Script	Programmierung einfacher DPUs mit einer Scriptsprache	✓
DPUSGEView	Teil der Grafik-Engine von SILAB	
DPUSGEWorld	Teil der Grafik-Engine von SILAB	

Ähnlich wie die Rechnerkonfiguration, wird die DPU-Konfiguration aus verschiedenen DPU-Pools zusammengesetzt. Jeder DPU-Pool enthält verschiedene DPUs und deren Verschaltung.

Die meisten DPUs, insbesondere universelle DPUs, können in der Simulation mehrfach eingesetzt werden. Die gleichen DPUs unterscheiden sich im Fall einer mehrfachen Verwendung hinsichtlich ihrer spezifischen Parametereinstellungen. Diese unterschiedlichen Varianten derselben DPU werden DPU-Instanzen genannt. Unter Instanziierung versteht man in der Informatik das Erzeugen eines Objekts (DPU-Instanz) aus einer bestimmten Klasse (DPU). Die DPUs dienen als Baupläne für die DPU-Instanzen, da sie vorgeben, welche Parametereinstellungen die dazugehörigen DPU-Instanzen enthalten. Zum besseren Verständnis wird die Instanziierung einer DPU mit der eines Hauses verglichen: Die Klasse ‚Haus‘ legt fest, durch welche Parameter ein Haus definiert wird, z.B. durch die Maße Länge x Breite x Höhe, die Anzahl der Fenster, die Eingangstür und das Dach. Diese Parameter können sich in ihrer genauen Ausführung und Anzahl von Objekt (Haus 1) zu Objekt (Haus 2) der Klasse ‚Haus‘ unterscheiden. Analog dazu unterscheiden sich die Instanzen einer DPU ebenso durch ihre spezifischen Parametereinstellungen.

Die Instanziierung von DPUs erfolgt in der DPU-Konfiguration. Um eine Instanz zu erzeugen, müssen folgende Informationen angegeben werden:

- Von welchem DPU-Typ soll eine Instanz erzeugt werden?
- Wie soll die Instanz heißen? Jede Instanz bekommt einen eindeutigen Namen
- Von welchem Simulationsrechner soll die Instanz berechnet werden?
- Welche Parametereinstellungen legen das Verhalten der Instanz fest?

Grundsätzlich kann jeder Ausgang einer DPU-Instanz mit den Eingängen einer oder mehrerer anderer DPU-Instanzen verschaltet werden (s. Abschnitt 6.3.1 für detaillierte Erklärung). Diese Verschaltungen werden in Abbildung 30 visualisiert. Die Menge aller DPU-Instanzen und deren Verschaltung zusammen wird DPU-Konfiguration genannt.

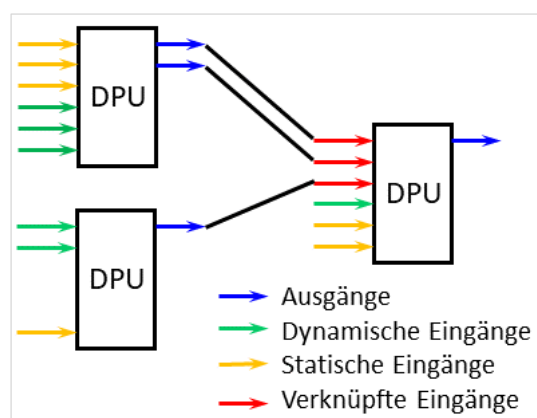


Abbildung 30. Visualisierung von DPU-Verschaltungen.

9.8.1 Aufbau und Inhalt einer DPU-Konfiguration

Als Beispiel für den Aufbau einer solchen DPU-Konfiguration finden Sie in Code 11 eine kommentierte, sehr vereinfachte DPU-Konfiguration eines Simulators. Die dazugehörigen DPUs und ihre Verschaltungen werden in Abbildung 31 dargestellt.

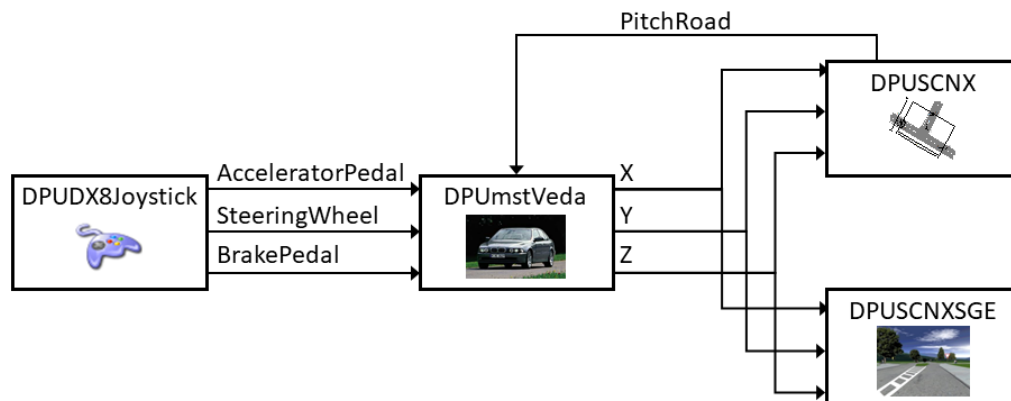


Abbildung 31. Visualisierung einer DPU-Konfiguration.

In der beispielhaften DPU-Konfiguration (s. Code 11) sind folgende Informationen enthalten. Mit der Konfiguration werden vier DPU-Instanzen mit den Namen `joy`, `vdyn`, `scn` und `vis` erzeugt. Diese DPU-Instanzen werden zu einem DPU-Pool zusammengefasst, der den Namen `Basisspool` erhält und durch die Anweisung `Executable = true` in die Liste der ausgewählten DPU-Pools für die vorliegende Konfiguration ausgewählt wird. Unterhalb jeder DPU-Instanz werden Informationen zu der Instanz angegeben. Dazu gehört die Angabe des Rechnernamens, von dem die DPU-Instanz geladen und ausgeführt werden soll. Parametereinstellungen zu der jeweiligen Instanz werden ebenfalls in diesem Abschnitt definiert. Die Parametereinstellung `Index` definiert die Reihenfolge, in der die DPUs auf den Simulationsrechnern während eines Simulationsschritts berechnet werden. Bei DPU-Verschaltungen sollte darauf geachtet werden, dass DPUs, die voneinander abhängen, nacheinander berechnet werden. Dies wird über eine entsprechende Index-Vergabe sichergestellt. Ansonsten dauert es mehrere Simulationstakte, bis die Daten die DPU Verschaltung einmal komplett durchlaufen haben. Der Parameter `map` verweist auf die darzustellende Karte. In dem Abschnitt `Connections` werden die Verschaltungen zwischen den DPU-Instanzen definiert. Die ersten drei Zeilen des Abschnitts sorgen dafür, dass in jedem Simulationsschritt die Ausgänge `SteeringWheel` (Lenkwinkel), `AcceleratorPedal` (Fahrpedal) und `BrakePedal` (Bremsen) der Instanz `joy` an die entsprechenden Eingänge der Instanz `vdyn` übertragen werden. Anders formuliert wird so definiert, dass die Eingaben am Joystick (umgerechnet in Lenkwinkel und Brems- und Fahrpedalstellung) an die Fahrdynamik-DPU (`DPUMstVeda`) weitergegeben werden, welche ein physikalisches Modell eines Fahrzeugs enthält. Die Fahrdynamik-DPU berechnet aus diesen Eingängen das Verhalten eines Fahrzeugs, also Parameter wie Geschwindigkeit und Position in einem dreidimensionalen Koordinatensystem. Die Position (`vdyn.X`, `vdyn.Y`, `vdyn.Z`) wird an die DPU-Instanz `scn` der `DPUSCNX` weitergegeben, welche die Datenbasis zur Repräsentation des simulierten

Straßennetzes, der umgebenen Landschaft und sämtlicher Objekte (z.B. Verkehrszeichen, Ampeln, Häuser, Bäume) verwaltet. Mit der von der Fahrdynamik-DPU berechneten Position stellt die DPUSCNX (wiederum in jedem Simulationsschritt) fest, wo im Straßennetz sich das simulierte Fahrzeug befindet. Sie kann daraufhin berechnen, welche Steigung die Straße an dieser Stelle hat (PitchRoad). Auf diese Steigung muss im nächsten Simulationsschritt die Fahrdynamik-DPU reagieren. Wenn der Fahrer z.B. bei einer sehr starken Steigung nicht mehr Gas gibt, muss das Fahrzeug entsprechend langsamer werden. Auch die Visualisierungs-DPU (DPUSCNXSGE), die für die grafische Darstellung der Fahrszene zuständig ist, muss die aktuelle Position des simulierten Fahrzeugs von der Fahrdynamik-DPU bekommen. Anhand dieser Position legt sie den Blickpunkt und Blickwinkel des Fahrers fest, aus dem die Fahrszene angezeigt wird.

```

DPUConfiguration Beispiel      # Name der DPU-Konfiguration
{
  Pool Basispool              # Name des DPU-Pools
  {
    Executable = true;        # Auswahl dieses DPU-Pools

    DPUDX8Joystick joy        # Verwendete DPU und Name der DPU-In-
    {                          stanz
      Computer = {OPERATORRECH-
NER};                          # Dieser Rechner führt die Instanz aus
      Index = 10;              # Index der DPU
    };

    DPUmstVeda vdyn
    {
      Computer = {HAUPTRECHNER};
      Index = 20;
    };

    DPUSCNX scn
    {
      Computer = {HAUPTRECHNER};
      Map = "Erster Versuch";  # Map wählt die darzustellende Karte aus
      Index = 30;
    };

    DPUSCNXSGE vis
    {
      Computer = {GRAFIKRECHNER};
    };

    Connections =              # Definition der DPU-Verschaltungen
    {
      joy.SteeringWheel -> vdyn.SteeringWheel,
      joy.AcceleratorPedal -> vdyn.AcceleratorPedal,
      joy.BrakePedal -> vdyn.BrakePedal,
      vdyn.X -> scn.X,
      vdyn.Y -> scn.Y,
      vdyn.Z -> scn.Z,
      vdyn.X -> vis.X,
      vdyn.Y -> vis.Y,
      vdyn.Z -> vis.Z,
      scn.PitchRoad -> vdyn.PitchRoad
    };
  };
};

```

Code 11. Beispiel für eine DPU-Konfiguration.

Die Funktionsweise, die Ein-/Ausgänge sowie die Parameter aller DPUs sind in gesonderten Dokumenten beschrieben. Pro DPU gibt es ein Dokument mit diesen Erläuterungen. Die Bezeichnung ist DPUXXX.pdf, z.B. DPUAmplifier.pdf.

In der beispielhaften DPU-Konfiguration in Code 11 wird nur eine DPU-Instanz pro DPU erzeugt. Dies hängt damit zusammen, dass die verwendeten DPUs sehr speziell auf die Fahrsimulation zugeschnitten sind. Deshalb ergibt es z.B. keinen Sinn, eine weitere Instanz der DPU `UmsVeda` anzulegen, da der Fahrer nicht in mehr als einem Fahrzeug zur selben Zeit fahren kann. Im Folgenden wird erläutert, wie das Erzeugen von mehreren Instanzen aus einer DPU funktioniert. Instanzen von universellen DPUs können in Abhängigkeit der Parametereinstellungen und der Verschaltungen mit anderen DPUs unterschiedliche Aufgabe übernehmen.

Beispielsweise können Sie eine Instanz der DPU zur Verstärkung, Abschwächung und Umrechnung von Signalen (`DPUAmplifier`) dazu benutzen, Geschwindigkeiten von m/s in km/h umzurechnen. Sie hat einen Eingang namens `In`, einen Ausgang `Out` und (u.a.) den Parameter `Multiplicator`. In jedem Simulationsschritt berechnet diese DPU (etwas vereinfacht):

$$\text{Out} = \text{Multiplicator} * \text{In}$$

Wenn `In` die Geschwindigkeit in der Einheit m/s ist, dann ergibt sich für `Out` mit dem `Multiplicator = 3.6` die Geschwindigkeit in km/h. Die DPU-Konfiguration wird folgendermaßen erweitert:

```
DPUAmplifier ms2kmh
{
    Computer = {HAUPTRECHNER};
    Multiplicator = 3.6;
};

Connections =
{
    vdyn.v -> ms2kmh.In
};
```

Code 12. Beispiel für eine Instanziierung der DPUAmplifier.

Die aktuelle Geschwindigkeit in der Einheit km/h steht damit nach jedem Simulationsschritt als Ausgang `out` der Instanz `ms2kmh` von `DPUAmplifier` zur Verfügung. Um die Einheiten anderer Werte umzurechnen, können Sie noch beliebig viele weitere Instanzen der `DPUAmplifier` erzeugen. Sie müssen lediglich einen eindeutigen Namen für jede Instanz festlegen, den Parameter `Multiplicator` jeder Instanz entsprechend initialisieren und die Eingänge der Instanzen verknüpfen.

Weitergehende Informationen zur DPU-Konfiguration finden Sie in den Kapiteln 2.2 und 2.7 im Dokument **REFERENZ SILAB-KONFIGURATION UND BEDIENUNG**. Kapitel 2.2.2 geht im Detail auf die Parameterzuweisungen zu DPU-Instanzen ein und Kapitel 2.7.2 auf das Erstellen von Verschaltungen.

9.8.2 Integration von DPUs in eine bestehende DPU-Konfiguration

In diesem Abschnitt lernen Sie, wie Sie neue DPUs in bereits bestehende DPU-Konfigurationen integrieren können. Beim Einsatz eines Fahrsimulators passiert es häufig, dass Sie zur Untersuchung eines Assistenzsystems (z.B. Tempomat) eine zusätzliche DPU integrieren möchten, welche das Assistenzsystem repräsentiert. Aus diesem Grund wäre es mühsam, für eine solche Änderung wie die Integration eines Assistenzsystems eine von Grund auf neue DPU-Konfiguration erstellen zu müssen. Praktischerweise können Sie eine bestehende Grundkonfiguration (z.B. die aus der im Lieferumfang enthaltenen Standardkonfigurationsarchitektur, s. Abschnitt 9.5) verwenden und nur die Änderungen der neuen Konfiguration definieren. Diese Basis der DPU-Konfiguration ist bei jeder Simulation gleich und sollte daher zentral abgelegt werden, sodass sie für jeden Versuch auf diesem Fahrsimulator eingesetzt werden kann. Die neue, von der Standardkonfiguration abgeleitete DPU-Konfiguration kann die Informationen der Standardkonfiguration verändern und neue Informationen hinzufügen. Dieser Mechanismus wird in SILAB ‚Vererbung‘ genannt, weil eine neue DPU-Konfiguration alle Informationen (DPU Instanzen, Parameterinitialisierungen, Verknüpfungen) von einer bereits bestehenden Konfiguration erbt.

Im Beispiel des zu integrierenden Tempomaten (s. Code 13) definieren Sie die Grundkonfiguration in einem DPU-Pool und geben ihm einen Namen, z.B. `Basispool` (vgl. mit der Beispiel-Konfiguration in Abschnitt 9.8.1) oder Sie verwenden die mitgelieferte Standardkonfigurationen (s. Abschnitt 9.5). Sie leiten dann einen weiteren Pool namens `TempomatPool` vom Pool `Basispool` ab. In dem abgeleiteten Pool machen Sie die für den Tempomat erforderlichen Angaben.

In der Konfigurationsdatei würde dieses Vorgehen folgendermaßen aussehen (es wird auf die in Abschnitt 9.8.1 vorgestellte DPU-Konfiguration eines einfachen Fahrsimulators zurückgegriffen):

```
Pool Basispool
{
    Executable = true;

    DPUDX8Joystick joy
    {
        Computer = {OPERATORRECHNER};
        Index = 10;
    };

    DPUmstVeda vdyn
    {
        Computer = {HAUPTRECHNER};
        Index = 20;
    };

    DPUSCNX scn
    {
        Computer = {HAUPTRECHNER};
        Map = "Erster Versuch";
        Index = 30;
    };

    DPUSCNXSGE vis
    {
        Computer = {GRAFIKRECHNER};
    };

    Connections =
    {
        joy.SteeringWheel    -> vdyn.SteeringWheel,
        joy.AcceleratorPedal -> vdyn.AcceleratorPedal,
        joy.BrakePedal       -> vdyn.BrakePedal,
        vdyn.X -> scn.X,
        vdyn.Y -> scn.Y,
        vdyn.Z -> scn.Z,
        vdyn.X -> vis.X,
        vdyn.Y -> vis.Y,
        vdyn.Z -> vis.Z,
        scn.PitchRoad -> vdyn.PitchRoad
    };
};
```

Code 13. Beispiel für die Integration einer DPU in eine bestehende DPU-Konfiguration.

Durch die Ableitung des neuen DPU-Pools TempomatPool von dem Basispool entspricht der Tempomat-Pool dem Basispool bis auf diese zwei Unterschiede:

Er enthält eine zusätzliche DPU-Instanz namens `tempomat`. Der zugehörige DPU-Typ `DPUTempomat` simuliert das neue Assistenzsystem. Die Geschwindigkeit, die der Tempomat halten soll, wird durch die Parameterinitialisierung auf $v = 100.0$ gesetzt.

Die im Basispool definierte Verknüpfung

```
joy.AcceleratorPedal -> vdyn.AcceleratorPedal
```

wird durch

```
tempomat.AcceleratorPedal -> vdyn.AcceleratorPedal
```

ersetzt.

Neben der Instanziierung von zusätzlichen DPUs und der Ersetzung von Verknüpfungen haben Sie in abgeleiteten Pools einige weitere sehr praktische Möglichkeiten, den Basispool zu erweitern bzw. zu modifizieren. Eine genauere Beschreibung der vorgestellten Mechanismen finden Sie in den Kapiteln 2.7 und 3 im Dokument [REFERENZ SILAB-KONFIGURATION UND BEDIENUNG](#).

10 INTEGRATION VON HARDWARE

SILAB ermöglicht die Einbindung externer Hardwarekomponenten in die Simulation über flexible Schnittstellen. Externe Hardware wird üblicherweise über den CAN (Controlled Area Network)-Bus angesteuert. Darunter versteht man einen seriellen Datenbus, der zur Vernetzung von Steuergeräten verwendet wird. Der CAN-Bus ermöglicht den Informationsaustausch zwischen den angeschlossenen Steuergeräten über die Datenbus-Leitungen. In SILAB können CAN-Adapterkarten der Hersteller PEAK und Vector verwendet werden, die über PCI oder USB an den PC angeschlossen werden.

Im CAN-BUS-Paket von SILAB sind hierzu die universellen DPUs DPUCANBUS und DPUCANDBC enthalten, mit denen beliebige Geräte in die Simulation integriert werden können.

Die DPUCANBUS dient als Treiber für CAN-Bus-Karten, die den gesamten Zugriff auf den CAN-Bus-Kanal verwaltet. Mehrere DPUs können gleichzeitig auf den CAN-Bus zugreifen. Die DPUCANBUS empfängt CAN-Nachrichten von anderen DPUs und leitet sie an die entsprechenden DPUs weiter, die die Nachricht angefordert haben. Wichtige Informationen zu dieser DPU finden Sie in dem Dokument [DPUCANBUS](#).

Die DPUCANDBC ist dafür zuständig, Nachrichten vom CAN-Bus zu lesen oder Nachrichten auf den CAN-Bus zu schreiben, deren Format in einer DBC-Datei definiert wurde. DBC ist ein Dateiformat zum Austausch von CAN-Bus-Daten und steht für ‚Data Base CAN‘. Das Dokument [DPUCANDBC](#) enthält Informationen zur Nutzung dieser DPU.

Zusätzlich existieren noch DPUs zur Ansteuerung von Geräten über die Netzwerkprotokolle UDP oder TCP/IP und zur Ansteuerung bestimmter Modelle von USB-Messadaptern mit Analog- und Digitaleingängen.

11 DATENAUFZEICHNUNG

Dieses Kapitel beinhaltet Erklärungen zur Aufzeichnung von Messwerten mithilfe der Konfigurationsdatei und zur Erstellung einer Aufzeichnungsdatei. In Abschnitt 5.6.1 haben Sie bereits gelernt, wie Sie mithilfe des Programms SILABSessionEdit eine Standarddatenaufzeichnung von bestimmten vordefinierten Fahrparametern durchführen können. Wenn Sie andere oder weitere Parameter aufzeichnen möchten, ist die Bearbeitung der Konfigurationsdatei erforderlich.

11.1 Definition einer Aufzeichnung

Für die zeitsynchrone Aufzeichnung von Messwerten ist die DPUDataFile zuständig, welche sehr viele verschiedene Eingänge hat. Diese DPU bietet den Vorteil, dass Sie nicht alle aufzuzeichnenden Ausgänge von DPUs mit den Eingängen der DPUDataFile umständlich in einem Connections-Abschnitt verknüpfen müssen. Stattdessen können Sie im Record-Abschnitt die DPU-Ausgänge aufzählen, die Sie aufzeichnen möchten. Abschnitt 0 enthält einen Überblick über die wichtigsten Simulationsvariablen, die je nach vorliegender Fragestellung für die Aufzeichnung relevant sein könnten.

Im Folgenden finden Sie ein kommentiertes Beispiel für die Anweisung zur Datenaufzeichnung im Abschnitt der DPU-Konfiguration in der Konfigurationsdatei.

```
DPUDataFile DataFile
{
    Computer = {DATA};           # Name der Aufzeichnungsdatei
    Index = 700;
    Separator = ",";
    HeaderType = 1;
    Path = "E:\Daten\Versuch1_"; # Spalten werden durch Komma getrennt
    Record =                     # Spaltennamen erscheinen als 1. Zeile
    {                             # in Aufzeichnungsdatei
        SCNX.s,                 # Speicherort der Aufzeichnungsdatei
        SCNX.LateralDistance,   # Aufzählung der DPU-Ausgänge, die
        SCNX.LaneIdx,           # aufgezeichnet werden sollen
        SCNX.LaneCurvatureXY("Krüm-
    mung"),
    VDyn.ax,                    # Umbenennung des Parameters in der
        Mockup.SteeringWheel    # Aufzeichnungsdatei in ,Krümmung'
    };
};
```

Code 14. Beispiel für Anweisung zur Datenaufzeichnung in der DPU-Konfiguration.

Anmerkungen zu Code 14:

- Der Parameter `Separator` legt fest, wie die Spalten in der Aufzeichnungsdatei voneinander getrennt werden. In diesem Beispiel werden sie durch ein Komma voneinander getrennt. Andere Möglichkeiten sind z.B. ein Semikolon ";" oder ein Tab " ".
- Der `HeaderType` legt fest, in welcher Zeile der Aufzeichnungsdatei die Spaltennamen (= Parameternamen) erscheinen. In diesem Beispiel erscheinen die Spaltennamen in der ersten Zeile.
- Die `Path`-Anweisung legt den Speicherort der Aufzeichnungsdatei fest. Eine Verzeichnisanzeige wird mit \ abgeschlossen. Im Beispiel ist das nicht der Fall. SILAB interpretiert daher `Versuch1_` nicht als Unterverzeichnis, sondern als Prefix, das vor jeden Dateinamen einer Aufzeichnung eingefügt wird.
- Im Abschnitt `Record = { . . . }` werden die DPU-Ausgänge aufgezählt, die Sie aufzeichnen möchten. Eine Übersicht über möglicherweise interessante DPU-Ausgänge finden Sie im Abschnitt 0.
- Sie haben die Möglichkeit, die aufzuzeichnenden Parameter mit einem von Ihnen gewünschten Namen umzubenennen. Im Beispiel würde der Parameter `SCNX.LaneCurvatureXY` in der Aufzeichnungsdatei in "Krümmung" umbenannt werden. Der Spaltenname in der Aufzeichnungsdatei lautet daraufhin 'Krümmung'. Ohne diesen Zusatz erhält die Spalte in der Aufzeichnungsdatei den Namen des jeweiligen DPU-Ausgangs.

Weitere Informationen zur Datenaufzeichnung mit der `DPUDataFile` finden Sie in dem Dokument [DPUDATAFILE](#).

11.2 Größe einer Aufzeichnungsdatei

Die Größe einer Aufzeichnungsdatei steigt mit der Anzahl aufgezeichneter Variablen und der Länge der Versuchsfahrt. Als Richtwert kann dienen, dass bei einer 80-minütigen Versuchsfahrt mit 40 aufgezeichneten Variablen etwa 300 MB Daten entstehen.

11.3 Aufzeichnungsfrequenz

Die Aufzeichnungs-DPU `DPUDataFile` zeichnet in jedem Simulationsschritt des Rechners, auf dem sie läuft, eine Datenzeile in der Aufzeichnungsdatei auf. Dies entspricht der Frequency-Angabe für den Rechner in der Rechnerkonfiguration. Typischerweise liegt diese zwischen 50 Hz und 150 Hz. Frequenzen bis 1000 Hz sind möglich.

11.4 Aufbau einer Aufzeichnungsdatei

Bei den Aufzeichnungsdateien, die von der DPUDataFile erstellt werden, handelt es sich um Plain-Text-Dateien im ASCII-Zeichensatz. Diese können mit einem Programm wie Geany, Notepad++, Crimson Editor oder auch Notepad betrachtet werden. Zur Auswertung der aufgezeichneten Daten werden diese Dateien üblicherweise in ein Tabellenkalkulations- oder Statistikprogramm wie Excel, Statistica oder SPSS importiert (s. Abschnitte 11.6.1 bis 11.6.3).

Die Aufzeichnungsdateien bestehen stets aus einem Tabellenkopf und den Datenzeilen. Es wird eine Datenzeile pro Simulationsschritt generiert. Bei einer Aufzeichnungsfrequenz von 50 Hz gibt es dementsprechend 50 Datenzeilen pro Sekunde Simulationsfahrt in der Aufzeichnungsdatei. Die ersten beiden Tabellenspalten werden automatisch erzeugt. Sie bestehen stets aus dem Messzeitpunkt der Datenzeile (in ms seit dem Simulationsstart) und dem Messzeitpunktfehler (d.h. Rechenzeitverzug des Aufzeichnungsrechners in ms). Dementsprechend sind auch die ersten beiden Einträge im Tabellenkopf immer der Messzeitpunkt und der Messzeitpunktfehler. Erst danach stehen nacheinander die in der SILAB-Konfiguration definierten aufgezeichneten Parameter. Jeder Parameter wird als Spalte in die Aufzeichnungsdatei geschrieben.

Das Format des Tabellenkopfs wird durch die Parametereinstellung `HeaderType` bestimmt (s. Code 14). Verwenden Sie die Einstellung `HeaderType=1`, um eine Textdatei zu erhalten, in der alle Spaltennamen, durch das konfigurierte Trennzeichen getrennt (z.B. Komma), in einer einzigen Zeile nebeneinander ausgegeben werden, z.B.: `Messzeitpunkt,Messzeitpunktfehler,X`

Bei dieser Einstellung enthält jede Datenzeile jeweils alle aufgezeichneten Werte als Gleitkommazahlen, getrennt durch das konfigurierte Trennzeichen (mit dem Punkt '.' als Dezimaltrenner), z.B.: `37080,0,-13.351996`. Mit diesem Format des Tabellenkopfs lassen sich die Daten später am besten in andere Programme importieren.

11.5 Benennung der Aufzeichnungsdatei

Der Name, unter dem die Aufzeichnungsdatei gespeichert wird, kann über die Launch-Daten festgelegt werden. Hierbei handelt es sich um Parameter, die bei jeder Vorbereitung der Simulation in SILAB erneut grafisch abgefragt werden. Diese Launch-Daten werden in dem Abschnitt zu den SILAB-Systemeinstellungen der SILAB-Konfigurationsdatei angefordert. Für eine detaillierte Beschreibung des Vorgehens und den Aufbau der Definition in den Launch-Daten wird auf Abschnitt 9.6.2 verwiesen. Die Definition der Launch-Daten kann folgendermaßen aussehen:

```
SILAB System
{
LaunchData1 = ("Daten aufzeichnen", VAR_SAVEDATA, LD_BOOL_DEFAULT, "1");
LaunchData2 = ("Dateiname", VAR_DATAFILENAME, LD_EDIT_DEFAULT, "Data");
};
```

Code 15. Konfiguration der Launch-Daten zur Datenaufzeichnung.

11.6 Import der Aufzeichnungsdatei in andere Programme

Um die aufgezeichneten Daten statistisch auswerten zu können und Plots anzufertigen, muss die Aufzeichnungsdatei im ASCII-Format in Tabellenkalkulations- oder Statistikprogramme wie Excel, Statistica oder SPSS importiert werden. Für den Import der Aufzeichnungsdatei in jedes der Programme empfiehlt sich die Einstellung `HeaderType = 1` zu wählen (s. Abschnitt 11.4). Mit diesem Format können die Aufzeichnungsdateien dann direkt über die Textdatei-Importfunktion eingelesen werden.

Wenn Sie in der Konfigurationsdatei ein anderes Trennzeichen (Separator) als ein Komma gewählt haben, müssen Sie dementsprechend dieses Trennzeichen für den Datenimport auswählen. Dies gilt für jedes Programm, in das Sie Daten importieren.

11.6.1 Import in Excel

Für den Datenimport der Aufzeichnungsdatei in Excel befolgen Sie bitte die folgende Anleitung. Bitte übernehmen Sie die Eingaben so, wie sie in den Screenshots des Textkonvertierungsassistenten von Schritt 1 bis 3 getroffen wurden.

Tabelle 29. Anleitung zum Datenimport der Aufzeichnungsdatei in Excel

Schritt	Aktion	Anleitung
1	Excel starten	
2	Datei öffnen	1. Menüleiste: Datei → Öffnen 2. ‚Durchsuchen‘ auswählen 3. Standard-Dialogfenster zum Öffnen von Dateien: ‚Alle Dateien (*.*)‘ auswählen 4. Die entsprechende Datei mit der Endung *.asc auswählen 5. Auf ‚Öffnen‘ klicken

3

Textkonvertierungs-Assistent:
Schritt 1

Textkonvertierungs-Assistent - Schritt 1 von 3

Der Textkonvertierungs-Assistent hat erkannt, dass Ihre Daten mit Trennzeichen versehen sind.
Wenn alle Angaben korrekt sind, klicken Sie auf 'Weiter', oder wählen Sie den korrekten Datentyp.

Ursprünglicher Datentyp

Wählen Sie den Dateityp, der Ihre Daten am besten beschreibt:

☒ Getrennt - Zeichen wie z.B. Kommas oder Tabstopps trennen Felder (Excel 4.0-Standard).
☐ Feste Breite - Felder sind in Spalten ausgerichtet, mit Leerzeichen zwischen jedem Feld.

Import beginnen in Zeile: 1 Dateisprung: 65001 : Unicode (UTF-8)

☐ Die Daten haben Überschriften.

Vorschau der Datei D:\DATA\BASE\Data.1.asc

1	MeasurementTime,Measurement time error,AcceleratorPedal,BrakePedal,SteeringWheel
2	0.12,0.12,0.500000,0.450000,-0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
3	16.67,0.00,0.500000,0.450000,-0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
4	33.33,0.00,0.500000,0.450000,-0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
5	50.00,0.00,0.500000,0.450000,-0.000000,0.000000,0.000000,0.000000,0.000000,0.000000

Abbrechen < Zurück Weiter > Fertig stellen

4

Textkonvertierungs-Assistent:
Schritt 2

Textkonvertierungs-Assistent - Schritt 2 von 3

Dieses Dialogfeld ermöglicht es Ihnen, Trennzeichen festzulegen. Sie können in der Vorschau der markierten Daten sehen, wie Ihr Text erscheinen wird.

Trennzeichen

☐ Tabstopp
☐ Semikolon
☒ Komma
☐ Leerzeichen
☐ Andere:

☐ Aufeinanderfolgende Trennzeichen als ein Zeichen behandeln

Textqualifizieren:

Hochkomma auswählen

Datenvorschau

Messzeitpunkt	Messzeitpunktfehler	ModuleTemplateID	ModuleTemplateName	ModuleInstan
13.99	13.99	7001.000000	Start	1.000000
46.32	29.65	7001.000000	Start	1.000000
47.46	14.12	7001.000000	Start	1.000000
50.00	0.00	7001.000000	Start	1.000000

Abbrechen < Zurück Weiter > Fertig stellen

5

Textkonvertierungs-Assistent:
Schritt 3

Textkonvertierungs-Assistent - Schritt 3 von 3

Dieses Dialogfeld ermöglicht es Ihnen, jede Spalte zu markieren und den Datentyp festzulegen.

Datenformat der Spalten

☒ Standard
☐ Text
☐ Datum: TMJ
☐ Spalte nicht importieren (überspringen)

Die Option 'Standard' behält Datums- und Zahlenwerte bei und wandelt alle anderen Werte in Text um.
Erweitert...

Datenvorschau

Standard	Standard	Standard	Standard	Standard
Messzeitpunkt	Messzeitpunktfehler	ModuleTemplateID	ModuleTemplateName	ModuleInstan
13.99	13.99	7001.000000	Start	1.000000
46.32	29.65	7001.000000	Start	1.000000
47.46	14.12	7001.000000	Start	1.000000
50.00	0.00	7001.000000	Start	1.000000

Abbrechen < Zurück Weiter > Fertig stellen

11.6.2 Import in Statistica

Für den Datenimport der Aufzeichnungsdatei in Statistica befolgen Sie bitte die folgende Anleitung. Bitte übernehmen Sie die Eingaben so, wie sie in dem Screenshot des Textkonvertierungsassistenten getroffen wurden.

Tabelle 30. Anleitung zum Datenimport der Aufzeichnungsdatei in Statistica

Schritt	Aktion	Erklärung
1	Statistica starten	
2	Datei öffnen	1. Menüleiste: Start → Öffnen 2. Standarddialogfenster: Ordner auswählen 3. Standarddialogfenster: ‚Alle Dateien (*.*)‘ auswählen 4. Standarddialogfenster zum Öffnen von Dateien: ‚Alle Dateien (*.*)‘ auswählen 5. Die entsprechende Datei mit der Endung *.asc auswählen 6. Auf ‚Öffnen‘ klicken 7. Dialogfenster: ‚Als Tabelle importieren‘ und ‚Frei‘ auswählen 8. Auf ‚OK‘ klicken

3 Textkonvertierungs-Assistent

Text (Frei) Import-Optionen

Trennzeichen Variable
☐ Tabulator ☐ Leerz. ☒ Komma ☐ Semikolon ☐ Sonst.: , ☐ Genau ☐ Ignoriere doppelte Trennz.

Optionen Import
☒ Variablennamen aus erster Zeile übernehmen
☐ Anzahl der Fälle überspringen 0
☐ Max. Anzahl der Fälle für Import 1001
☐ Trennzeichen für Nachkommastellen ,
☐ Missing-Data-Zeichenfolge NaN
☒ Leere Zeilen überspringen
 Standards zurück Details

Var.-Optionen (für Freigabe unten klicken)
☐ Name
☒ Zahl 1
☐ Missing-Data-Wert -999999998
☐ Format
☐ Nicht-numerische Zeichen entfernen
☐ Auslassen Formatierung

Trennzeichen Fall
☐ Wagenrücklauf
☒ Zeilenverschub
☐ Sonstiges: \n
☐ Genau
 Fälle zur Vorschau 1000
☐ Vorschau (alle)

Textdatei (Data.asc)

	Messzeitpunkt	Messzeitpunktfehler	ModuleTemplateID	ModuleTemplateName	Mod
	13.99	13.99	7001.000000	Start	1.0
	46.32	29.65	7001.000000	Start	1.0
	47.46	14.12	7001.000000	Start	1.0
	50.00	0.00	7001.000000	Start	1.0
	66.67	0.00	7001.000000	Start	1.0
	83.33	0.00	7001.000000	Start	1.0
	100.00	0.00	7001.000000	Start	1.0
	116.67	0.00	7001.000000	Start	1.0
	133.33	0.00	7001.000000	Start	1.0
	150.01	0.01	7001.000000	Start	1.0
	166.67	0.00	7001.000000	Start	1.0
	183.33	0.00	7001.000000	Start	1.0
	200.00	0.00	7001.000000	Start	1.0
	216.67	0.00	7001.000000	Start	1.0
	233.34	0.00	7001.000000	Start	1.0

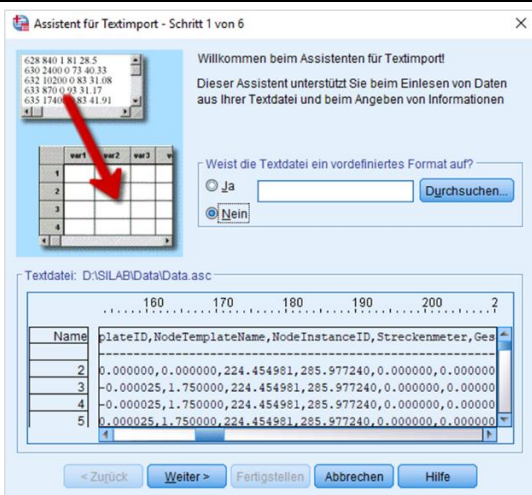
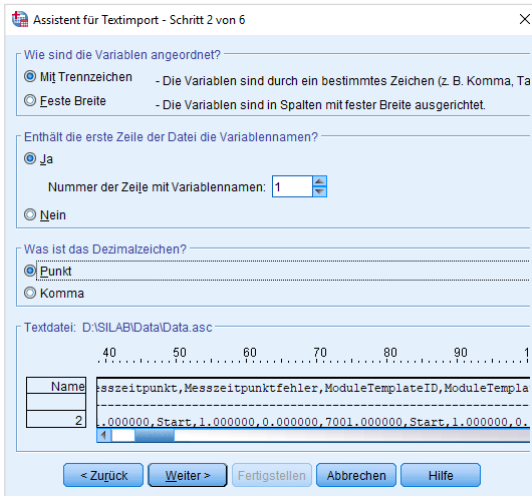
Hinweis: Klicken Sie OK, um die Datendatei unter Verwendung der gewählten Optionen als Tabelle importieren zu lassen. Die Tabellen-Cache-Größe wird als Vorschau-Puffer verwendet, um die Variablentypen zu erfassen.

☐ Automatisch Vorschau akt. OK Abbrechen

11.6.3 Import in SPSS

Für den Datenimport der Aufzeichnungsdatei in SPSS befolgen Sie bitte die folgende Anleitung. Bitte übernehmen Sie die Eingaben, so wie sie in den Screenshots des Assistenten zum Textimport von Schritt 1 bis 6 getroffen wurden.

Tabelle 31. Anleitung zum Datenimport der Aufzeichnungsdatei in SPSS.

Schritt	Aktion	Erklärung
1	SPSS starten	
2	Datei öffnen	1. Oben links im Startfenster: ‚Neue Dateien → Neues Dataset‘ 2. Menüleiste: ‚Datei → Öffnen → Daten...‘ 3. Standarddialogfenster zum Öffnen von Dateien: ‚Alle Dateien (*.*)‘ auswählen 4. Die entsprechende Datei mit der Endung *.asc auswählen 5. Auf ‚Öffnen‘ klicken
3	Assistenz für Textimport – Schritt 1	
4	Assistenz für Textimport – Schritt 2	

5

Assistenz für Textimport – Schritt 3

Assistent für Textimport - Schritt 3 von 6 (Trennzeichen)

In welcher Zeile befindet sich der erste Fall in den Daten?

Wie sind die Fälle dargestellt?

☒ Jede Zeile stellt einen Fall dar

☐ Folgende Anzahl von Variablen stellt einen Fall dar:

Wie viele Fälle sollen importiert werden?

☒ Alle Fälle

☐ Die ersten Fälle.

☐ Zufälliger Prozentwert der Fälle (ungefähr): %

Datenvorschau

Name	Messzeitpunkt, Messzeitpunktfehler, ModuleTemplateID, Module...
2	001.000000, Start, 1.000000, 0.000000, 7001.000000, Start, 1.000...
3	001.000000, Start, 1.000000, 0.000000, 7001.000000, Start, 1.000...
4	001.000000, Start, 1.000000, 0.000000, 7001.000000, Start, 1.000...
5	01.000000, Start, 1.000000, 0.000000, 7001.000000, Start, 1.000...

< Zurück Weiter > Fertigstellen Abbrechen Hilfe

6

Assistenz für Textimport – Schritt 4

Assistent für Textimport - Schritt 4 von 6 (Trennzeichen)

Welches Zeichen trennt die Variablen?

☐ Tabulator ☐ Leerzeichen

☒ Komma ☐ Semikolon

☐ Anderes:

Was ist das Texterke...

☐ Keines

☒ Hochkommas

☐ Anführungszeichen

☐ Anderes:

Führende und nachfolgende Leerzeichen

☐ Führende Leerzeichen aus Zeichenfolgert entfernen

☐ Nachfolgende Leerzeichen aus Zeichenfolgert entfernen

Datenvorschau

Messzeitp...	Messzeitp...	ModuleTe...	ModuleTe...	ModuleInst...	ModuleInst...	Nc
13.99	13.99	7001.0000...	Start	1.000000	0.000000	70
46.32	29.65	7001.0000...	Start	1.000000	0.000000	70
47.46	14.12	7001.0000...	Start	1.000000	0.000000	70
50.00	0.00	7001.0000...	Start	1.000000	0.000000	70
66.67	0.00	7001.0000...	Start	1.000000	0.000000	70
83.33	0.00	7001.0000...	Start	1.000000	0.000000	70
100.00	0.00	7001.0000...	Start	1.000000	0.000000	70
116.67	0.00	7001.0000...	Start	1.000000	0.000000	70
133.33	0.00	7001.0000...	Start	1.000000	0.000000	70
150.01	0.01	7001.0000...	Start	1.000000	0.000000	70

< Zurück Weiter > Fertigstellen Abbrechen Hilfe

7

Assistenz für Textimport – Schritt 5

Assistent für Textimport - Schritt 5 von 6

Spezifikationen für die in der Datenvorschau ausgewählten Variablen

Variablenname: Originalname:

Datenformat:

☒ Automatisch

☐ Nicht importieren

☐ Numerisch

☐ Zeichenfolge

Prozentsatz der Werte, die das automatische Datenformat festlegen

Datenvorschau

Messzeitp...	Messzeitp...	ModuleTe...	ModuleTe...	ModuleInst...	ModuleInst...	Nc
13.99	13.99	7001.0000...	Start	1.000000	0.000000	70
46.32	29.65	7001.0000...	Start	1.000000	0.000000	70
47.46	14.12	7001.0000...	Start	1.000000	0.000000	70
50.00	0.00	7001.0000...	Start	1.000000	0.000000	70
66.67	0.00	7001.0000...	Start	1.000000	0.000000	70

< Zurück Weiter > Fertigstellen Abbrechen Hilfe

8

Assistenz für Textimport – Schritt 6

Assistent für Textimport - Schritt 6 von 6

Sie haben das Format der Textdatei jetzt festgelegt.

Dateiformat für zukünftige Verwendung speichern?

☐ Ja ☐ Nein

Soll die Syntax eingefügt werden? ☒ Daten in lokalen Zwischenspeicher

☐ Ja ☐ Nein

Klicken Sie auf "Fertigstellen", um den Assistenten für Textimport zu beenden.

Datenvorschau

Messzeitp...	Messzeitp...	ModuleTe...	ModuleTe...	ModuleInst...	ModuleInst...	NodeTem...	NodeTem...
13.99	13.99	7001.0000...	Start	1.000000	0.000000	7001.0000...	Start
46.32	29.65	7001.0000...	Start	1.000000	0.000000	7001.0000...	Start
47.46	14.12	7001.0000...	Start	1.000000	0.000000	7001.0000...	Start
50.00	0.00	7001.0000...	Start	1.000000	0.000000	7001.0000...	Start
66.67	0.00	7001.0000...	Start	1.000000	0.000000	7001.0000...	Start
83.33	0.00	7001.0000...	Start	1.000000	0.000000	7001.0000...	Start
100.00	0.00	7001.0000...	Start	1.000000	0.000000	7001.0000...	Start

< Zurück Weiter > Fertigstellen Abbrechen Hilfe

12 ERSTELLUNG VON SCREENSHOTS UND VIDEOS

Mit SILAB haben Sie zusätzlich zur Aufzeichnung von DPU-Ausgängen im Textformat die Möglichkeit, die visualisierte Szenerie grafisch aufzuzeichnen. Sie können sowohl statische Screenshots als auch Videos aufnehmen.

12.1 Screenshots

Die DPUSGEView ermöglicht die Erstellung von Screenshots während der Simulation. Über den DPU-Eingang ‚Screenshot‘ können Sie ein Bildschirmfoto der aktuell sichtbaren Szenerie erzeugen. Diese werden im Format *.png in einem frei wählbaren Verzeichnis abgespeichert.

Zur Erstellung eines Screenshots während der Simulation gehen Sie bitte, wie in Tabelle 32 beschrieben, vor.

Tabelle 32. Anleitung zur Erstellung von Screenshots.

Schritt	Aktion	Erklärung
1	Simulation starten	Starten Sie wie gewohnt die Simulation mit SILAB.
2	DPUSGEView auswählen	Suchen Sie in der DPU-Liste die DPU mit dem Namen DPUSGEView und klicken Sie auf die DPU, sodass die Liste mit ihren Ein- und Ausgängen unterhalb aufgeklappt wird.
3	Schalter für DPU-Eingang ‚Screenshot‘ auswählen	Suchen Sie in der Liste der Ein- und Ausgänge unterhalb der DPUSGEView nach dem Eingang ‚Screenshot‘. Klicken Sie mit der rechten Maustaste auf den Eingang und wählen Sie aus dem Kontextmenü ‚Schalter‘ aus (Abbildung 32).
4	Screenshot erstellen	Sie erstellen einen Screenshot, indem Sie den Schalter von ‚Aus‘ (0) auf ‚An‘ (1) wechseln (Abbildung 32). Um einen wiederholten Screenshot zu erstellen, müssen Sie den Schalter zunächst erst wieder auf ‚Aus‘ zurücksetzen, um dann erneut auf ‚An‘ zu wechseln.
5	Screenshot speichern	Bei jedem Druck auf diesen Schalter wird dann ein Screenshot erzeugt und mit nummerierten, fortlaufenden Dateinamen abgelegt. Die Dateien heißen Shot0001.png usw.

Der entstandene Screenshot wird in das Verzeichnis ‚ScreenshotDir‘ abgelegt. Die Dateinamen werden aufsteigend unter dem definierten Dateinamen nummeriert (z.B. Shot0001, Shot0002, usw.). In der Konfigurationsdatei müssen folgende Angaben in der Rechnerkonfiguration gemacht werden (genauere Informationen finden Sie im Dokument DPUSGEView und in den Kapiteln 2.1.1, 2.1.2 und 5 im Dokument **REFERENZ VISUALISIERUNG SGE**).

```

VISFRONT.sgeview.ScreenshotDir = "\\SILAB\\Data\\Screenshots";           # Speicherort der Screenshots
VISFRONT.sgeview.ScreenshotBaseName = "Shot";                         # fortlaufender Dateiname
VISFRONT.sgeview.ScreenshotBaseName = "Shot";                         # die erzeugten Screenshots
VISFRONT.sgeview.WriteAtOnce = true;                                   # werden sofort abgespeichert

```

Code 16. Angaben in der Rechner-Konfiguration zur Erstellung von Screenshots.

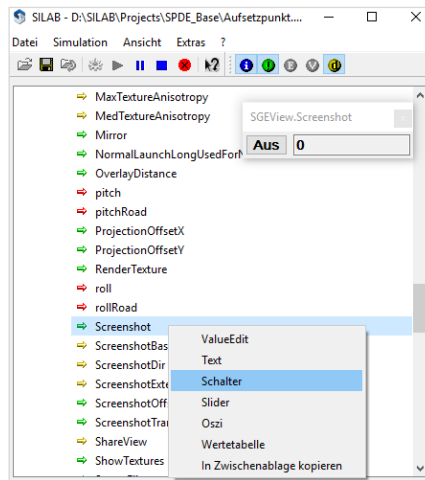


Abbildung 32. Erstellung eines Screenshots während der Simulation.

Mithilfe einer DPUCounter-Instanz können Sie in regelmäßigen Abständen automatisch Screenshots erstellen. Die entsprechenden Anweisungen, die Sie über den include-Mechanismus in Ihre Konfigurationsdatei einbinden können, befinden sich in der ‚DPUScreenshotSeries‘, welche im Verzeichnis ‚SILAB\lib\configurations\system‘ gespeichert ist.

12.2 Videos

In SILAB kann auch eine kontinuierliche Aufzeichnung der Szenarienansicht als Video durchgeführt werden. Ohne den Umweg über ein analoges Signal kann dazu während der Simulation jedes n-te Bild aufgezeichnet und mit mehreren gängigen Videocodern in eine digitale Videodatei (MP4) komprimiert und aufgezeichnet werden.

Da die Aufzeichnung des Videos zusätzlich zur Berechnung und 3D-Darstellung der Szenerie erfolgt, muss hierfür ein sehr leistungsfähiger Visualisierungsrechner (etwa doppelte CPU-Leistung wie üblich) vorhanden sein. Wir empfehlen zur digitalen Videoaufzeichnung die Integration eines zusätzlichen Videorechners in den Simulator. Seine Hardwareausstattung sollte den Grafikrechnern entsprechen. Da die aufgezeichneten Videos sehr groß werden (es ist mit einem Platzbedarf von ca. 5 GB / Stunde zu rechnen), ist ggf. der Einbau eines zusätzlichen Datenträgers notwendig.

Der Video-Rechner wird in der Rechnerkonfiguration wie ein Grafikrechner behandelt, wobei ihm zusätzlich der Aliasname RECORD zugewiesen wird. Über diesen Aliasnamen werden in der DPU-Konfiguration die entsprechenden DPUs zur Videoaufzeichnung instanziiert.

Dieser Abschnitt könnte in der Rechnerkonfiguration folgendermaßen aussehen:

```
Computer FSVIDEO
{
    IP = "10.1.2.43";
    Frequency = 60;
    Alias = {ALL,SCN,VIS,VISF,RECORD}; # Hinzufügen des Aliasnamen
RECORD
};
```

Code 17. Beispiel für die Definition des Video-Rechners in der Rechner-Konfiguration.

Für die Erstellung eines Videos wird die DPUSGECapture verwendet. Diese DPU zeichnet die Ansicht der SILAB-Visualisierung SGE auf.

```
DPUSGECapture SGECap
{
    Index = 100;
    Computer = {RECORD};

    Source = SGEView;
    Subsample = 2;           # Aufzeichnung der Visualisierung
                             SGE # mit halber Bildrate (s.u.)

    Path = "D:\Videos\MyPro- # Speicherort des Videos
ject\";
    FileName = "MyVideo";
    Extension = "mp4";       # Erstelle Video im MPEG4 Format
    VideoKbps = 8000;        # mit 8 Mbit/s Datenrate
};
```

Code 18. Beispiel für die Instanziierung der DPUSGECapture zur Erstellung von Videos.

Hierzu noch folgende Anmerkungen:

- `Subsample = 2` bedeutet, dass das Video mit der halben Bildwiederholrate des Rechners aufgezeichnet wird. Falls der RECORD-Rechner mit einer Frequency von 60 Hz konfiguriert wurde, wird das Video also mit einer Bildrate von 30 Bildern pro Sekunde ($= 60 / \text{Subsample}$) erstellt. Diese Bildrate reduziert die Anforderungen an die Prozessorleistung sowie die Dateigröße. Zudem können kurzzeitige Überlastungen der Visualisierung SGE automatisch durch die DPUSGECapture ausgeglichen werden, wenn diese nicht mit voller Bildrate aufzeichnet. Dennoch erscheint das erstellte Video flüssig.
- Die Parameter `Extension` und `VideoKbps` geben an, dass das Video im Format MPEG4 mit einer Datenrate von 8 Mbit/s aufgezeichnet wird. Dieses Format hat sich als beste Wahl bezüglich der Bildqualität und des Speicherplatzverbrauchs erwiesen.
- Die `Path`-Anweisung legt den Speicherort des Videos fest. Eine Verzeichnisangabe wird mit `\` abgeschlossen. Standardmäßig überschreibt die DPUSGECapture bereits vorhandene Dateien nicht, sondern verhindert in diesem Fall den Start der Simulation mit einer Fehlermeldung. Bei der Durchführung einer Versuchsreihe ist es empfehlenswert, bei jedem Start

der Simulation einen neuen Dateinamen zu vergeben. Dies ist am einfachsten über die Konfiguration der Launch-Daten im globalen SILAB System-Abschnitt möglich (s. Abschnitt 9.6.2), welche folgendermaßen aussehen würde:

```
SILAB System
{
LaunchData1 = ("Videodateiname", VAR_DATAFILENAME, LD_EDIT_DEFAULT,
"Data")
};
```

Code 19. Konfiguration der Launch-Daten zur Videoaufzeichnung.

Die DPUSGECapture unterstützt noch zahlreiche weitere Parameter, die in der zugehörigen DPU-Dokumentation **DPUSGECAPTURE** beschrieben sind.

Weiterführende Informationen zur Videoaufzeichnung finden Sie in den Dokumenten **REFERENZ VISUALISIERUNG SGE** sowie **TUTORIAL VIDEO-AUFZEICHNUNG**.

13 ERSTELLUNG VON DPUS MIT RUBY

SILAB kann um DPUs erweitert werden, die in einer anderen Skriptsprache erstellt wurden. Sie haben dementsprechend die Möglichkeit, DPUs als Skripte in der Skriptsprache Ruby zu realisieren. Ruby ist eine objektorientierte Programmiersprache, die umfangreiche Funktionalität in einer kompakten und übersichtlichen Syntax bietet.

Ein Anwendungsfall für die Erweiterung von SILAB um eine DPU als Ruby-Skript wäre das Einbinden eines Assistenzsystems, für das die Konfiguration aus universellen DPUs zu kompliziert ist oder die gewünschte Funktionalität durch eine Kombination universeller DPUs nicht realisierbar ist. In diesem Fall ist es hilfreich, die Erweiterung als Ruby-Skript mit der `DPURubyScript` zu realisieren.

SILAB bietet mit der `DPURubyScript` die Integration eines Ruby-Interpreters, mit dem Ruby-Skripte genauso wie im Lieferumfang vorhandene DPUs verwendet werden können, um leicht während der Simulation Berechnungen auszuführen. Der Quelltext solcher Ruby-Skripte wird beim Start der Simulation eingelesen und zur Laufzeit ausgeführt. Dadurch entfällt das Kompilieren der Skripte in eine separat einzubindende DLL (Dynamic Link Library). Ruby-Skripte können beliebige Ein- und Ausgangsvariablen definieren und darüber mit der Simulation kommunizieren.

Nähere Informationen zur Erstellung und Einbindung eines Ruby-Skripts finden Sie in den Dokumenten [DPURUBYSRIPT](#) und [REFERENZ RUBY SCRIPTING](#).

13.1 Anwendungsbeispiel

Um ein grundlegendes Verständnis für das Anwendungsgebiet von Ruby-Skripten und deren Einbindung in SILAB zu erhalten, wird in Folgenden ein Anwendungsbeispiel aufgezeigt:

Sie möchten in der Simulation ein Assistenzsystem realisieren, das den Fahrer akustisch warnt, wenn er zu weit von der Spur abkommt. Je nachdem, ob er zu weit links oder zu weit rechts von der Idealspur fährt, soll ein Warnton von links oder von rechts eingespielt werden. Ausgangspunkt sind zwei DPU-Ausgänge, die von der Datenbasis `DPUSCNX` zur Verfügung gestellt werden:

LANEWIDTH

- Breite der Spur, auf der sich der Fahrer gerade befindet (in Meter).

LATERALDISTANCE

- Lateraler Abstand des Fahrers von der exakten Spurmitte. Negative Werte bedeuten, dass der Fahrer zu weit links fährt, und positive Werte, dass der Fahrer zu weit rechts fährt. Dieser Wert liegt immer im Bereich von $-\text{LaneWidth}/2$ bis $+\text{LaneWidth}/2$.

Das System besteht zum einen aus einer Instanz der `DPUAmplifier` namens `LatDistNorm`. Damit die betrachtete Spurabweichung nicht mehr von der aktuellen Spurbreite abhängig ist, normiert diese DPU-Instanz den lateralen Abstand auf den Bereich von -1.0 bis 1.0. Der normierte Abstand dient als Eingabe für zwei Instanzen der `DPUComparator`. Die erste Instanz namens `LatDistLeftComp`

überprüft, ob der Abstand nach links größer ist als ein bestimmter Schwellwert und liefert die Ausgabe 1, falls dies der Fall ist (ansonsten erfolgt die Ausgabe 0). Analog arbeitet die andere Instanz `LatDistRightComp` für Abstände nach rechts. Das Ergebnis eines jeden Vergleichs steuert das Abspielen eines Warntons an. Hierfür wird eine Instanz `WarnSound` der `DPUWavPlayer` verwendet. Mit folgender DPU-Konfiguration wird SILAB um das zuvor beschriebene Assistenzsystem erweitert (ausschließlich unter Verwendung universeller DPUs):

```
DPUAmplifier LatDistNorm
{
  Computer = {...};
  Multiplicator = 2.0;
};

DPUComparator LatDistLeftComp
{
  Computer = {...};
  Min = -0.8;
  Max = -0.8;
};

DPUComparator LatDistRightComp
{
  Computer = {...};
  Min = 0.8;
  Max = 0.8;
};

DPUWavPlayer WarnSound
{
  Computer = {...};
  Sound1 = "NagelbandLinks.wav";
  Sound2 = "Nagelband-
Rechts.wav";
};

Connections =
{
  scn.LateralDistance ->
  LatDistNorm.In,
  scn.LaneWidth ->
  LatDistNorm.Divisor,
  LatDistNorm.Out ->
  LatDistLeftComp.In,
  LatDistLeftComp.Less->
  WarnSound.PlaySample1,
  LatDistNorm.Out ->
  LatDistRightComp.In,
  LatDistRightComp.Greater ->
  WarnSound.PlaySample2
};
```

hier wird ein Rechner Ihres
Simulators angegeben

Warnung ab einer normierten
Abweichung von -0.8 nach links

Warnung ab einer normierten
Abweichung von 0.8 nach rechts

Zuordnung der Warntöne

Normierung von LateralDistance
auf
-1 ... 1

Zu große Abweichung nach links?

Zu große Abweichung nach
rechts?

Code 20. Beispiel für die DPU-Konfiguration eines Assistenzsystems.

Das beschriebene Assistenzsystem mit seiner geringeren Komplexität zwar auch direkt durch eine Verknüpfung universeller DPUs umsetzbar, allerdings wird diese Vorgehensweise bei komplexeren Bedingungen und Formeln schnell sehr unübersichtlich. In diesem Fall empfiehlt sich eine Umsetzung als Ruby-Skript. Für das Beispiel-Assistenzsystem würde das Ruby-Skript folgendermaßen lauten:

```
class RPULatDistWarn < RPU
  def initialize
    var_in :@lane_width, :@lateral_distance
    var_out :@warn_left, :@warn_right
  end

  def trigger(time, time_error)
    norm_lat_dist = 2.0 * @lateral_distance / @lane_width
    @warn_left = (norm_lat_dist < -0.8)
    @warn_right = (norm_lat_dist > 0.8)
  end
end
```

Code 21. Beispiel für ein Ruby-Skript für die DPU-Konfiguration eines Assistenzsystems.

Dieses Ruby-Skript wird daraufhin in die SILAB-Konfiguration eingebunden. Das Skript muss in dem entsprechenden Projektordner abgelegt werden, in dem Sie aktuell arbeiten. Hierfür wird in der DPU-Konfiguration eine Instanz der DPURubySkript angelegt, in der das Skript geladen wird. Unter Connections wird diese DPU-Instanz mit dem WAV-Player und der Datenbasis verbunden.

```
DPURubyScript LatDistWarn
{
  Computer = {...};
  Script = "RPULatDistWarn.rb";           # Name des Rechners
};                                           # Name des Ruby-Skripts

DPUWavPlayer WarnSound
{
  Computer = {...};                       # Zuordnung der Warntöne
  Sound1 = "NagelbandLinks.wav";
  Sound2 = "NagelbandRechts.wav";
};

Connections =
{
  scn.LateralDistance ->
  LatDistWarn.LateralDistance,
  scn.LaneWidth ->
  LatDistWarn.LaneWidth,                 # Verbindung mit # Daten-
  LatDistWarn.WarnLeft ->                basis
  WarnSound.PlaySample1,                 # Verbindung mit WAV-
  LatDistWarn.WarnRight ->               # Player
  WarnSound.PlaySample2
```

```
};
```

Code 22. Beispiel für Einbindung und Verknüpfung eines Ruby-Skripts.

Bei sehr komplizierten Berechnungen (z.B. Operationen auf umfangreichen Arrays von Werten) kann die Auswertung eines Ruby-Skripts während der Simulation zu langsam sein. Das merken Sie, wenn der Rechner, auf dem die entsprechende Instanz der DPURubyScript läuft, Verzögerungen aufbaut (s. Abschnitt 6.5). Um solche Verzögerungen zu vermeiden, kann ein Erwerb der ENTERPRISE Edition hilfreich sein, da diese die Programmierung von DPUs in Java oder C++ ermöglicht.

14 GLOSSAR

ABTASTFREQUENZ legt die Häufigkeit fest, mit der ein Parameter im Oszilloskop dargestellt wird.

ARBEITSBEREICH (ENGL. WORKSPACE) bezeichnet das Dateiformat mit der Dateiendung *.wsp, in dem Konfigurationsdateien mit dem von Ihnen ausgewählten Layout gespeichert werden. Das Layout beinhaltet die Auswahl und Platzierung von Parameter-Anzeigen.

AUFSETZPUNKT ist die Stelle in einer Session, an der die Simulation gestartet werden kann. Aufsetzpunkte werden zuvor definiert und können vor jedem Start der Simulation mit dem Launch-Wizard ausgewählt werden.

AUFZEICHNUNGSFREQUENZ gibt an, wie viele Datenzeilen pro Sekunde von der Aufzeichnungs-DPU DPUDataFile in der Aufzeichnungsdatei aufgezeichnet werden. Sie entspricht der Takt-rate/Frequency des Rechners, auf der die DPUDataFile läuft. Die Aufzeichnungsfrequenz wird in Hz angegeben.

DPU (DATA PROCESSING UNITS) sind grundlegende funktionale Softwaremodule einer Simulation mit SILAB, die während der Simulation verschiedene Aufgaben erfüllen.

INSTANZIIERUNG beschreibt das Erzeugen eines Objekts (DPU-Instanz) aus einer bestimmten Klasse (DPU). Aus einer DPU können verschiedene DPU-Instanzen abgeleitet werden.

LAUNCH-DATEN sind Informationen, die Sie vor jedem Start einer Simulation den DPU-Instanzen mittels Eingaben in einem Dialogfenster mitteilen können, z.B. Informationen zur Datenaufzeichnung, Videoaufzeichnung oder Auswahl des Aufsetzpunktes.

LAUNCH-WIZARD bezeichnet das Dialog-Fenster, mit dem vor dem Simulationsstart die Launch-Daten abgefragt werden.

LOGAUSGABE-PARAMETER werden im Meldungsfenster von SILAB angezeigt und informieren über aktuelle Meldungen zur Simulation.

SCN/SCNX steht für ‚Scenery‘ ist die Datenbasis für die SILAB-Fahrsimulation, die Informationen über den Verlauf, die Gestaltung und den Aufbau des befahrbaren Streckennetzes sowie das Aussehen und Höhenprofil der Umgebung enthält. SCNX ist die weiterentwickelte Version von SCN (X steht für Extended).

SCNXSGE ist das Programmmodul der SILAB-Visualisierung, das für eine realistische Darstellung des simulierten Szenarios auf den Anzeigegeräten des Fahrsimulators (Monitore oder Projektoren) sorgt. SGE steht für ‚Simulator Graphics Enhanced‘.

SILABADMIN ist ein Programm des SILAB-Softwarepakets, mit dem die an der Simulation beteiligten Rechner gesteuert und überwacht werden.

SILABEXPLORER ist ein Programm des SILAB-Softwarepakets, mit dem die Simulation von anderen Rechnern ferngesteuert werden kann.

SILABSESSIONEDIT ist ein Programm des SILAB-Softwarepakets, mit dem Sessions aus mitgelieferten Szenarien-Datenbanken zusammengestellt werden können.

Session bezeichnet eine Simulationsstrecke in SILAB, die aus einer Abfolge von verschiedenen Szenarien besteht.

Szenarien-Datenbank bezeichnet die Sammlung von Szenarien in unterschiedlichen Verkehrsumgebungen, aus denen mit dem Programm SILABSessionEdit nach einem Baukasten-Prinzip eine Session erstellt werden kann.

PARCOUR eine Abfolge an Szenarien die von SILABSessionEdit in das SILAB Konfigurationsdateiformat übersetzt wurden.

TAKTRATE (ENGL. FREQUENCY) gibt an, wie viele Berechnungsschritte pro Sekunde durchgeführt werden sollen. Die Taktrate wird in Hz angegeben.

TRF/TRFX steht für ‚Traffic‘ und ist ein Verbund aus verschiedenen DPU's, die in SILAB für die Simulation anderer Verkehrsteilnehmer zuständig sind. TRFX ist die weiterentwickelte Version von TRF (X steht für Extended).

WATCHDOG-Parameter sind Funktionen, die Zeitüberschreitungen von Rechnerabläufen überwachen. Die Überwachung der Remoterechner bei SILAB erfolgt periodisch vom Operatorrechner aus.